

Robust Non-Interactive Zero-Knowledge Combiners*

Michele Ciampi¹, Lorenzo Magliocco², Daniele Venturi², and Yu Xia¹

¹The University of Edinburgh, UK
{michele.ciampi, yu.xia}@ed.ac.uk

²Sapienza University of Rome, Italy
{magliocco, venturi}@di.uniroma1.it

April 20, 2026

Abstract

A t -out-of- n robust non-interactive zero-knowledge (NIZK) combiner is a construction that, given access to n candidate instantiations of a NIZK for some language, itself implements a NIZK for the same language. Moreover, the combiner is secure, assuming at least t of the given candidates are secure. In this work, we provide the first definition of robust combiners for NIZK, and prove that no robust NIZK combiner exists assuming $t \leq \lfloor n/2 \rfloor$ (unless the polynomial hierarchy collapses). On the positive side, we provide different constructions of robust NIZK combiners for $t > \lfloor n/2 \rfloor$. In particular, we show how to obtain:

- A black-box combiner working for a special class of *homomorphic* languages where n, t are polynomial and $t > \lfloor n/2 \rfloor$.
- A non-black-box combiner working for any language, where n, t are constant and $t > \lfloor n/2 \rfloor$.
- A non-black-box combiner working for any language, where n, t are polynomial and $t > \lfloor 2n/3 \rfloor$.

1 Introduction

A short story. Once upon a time, there was a little girl called Cryptess. She was smart and had a passion for crypto puzzles. She loved walking through her favourite forest, especially while thinking about the next hard problem to attack. One day, while wandering through the forest without thinking about where to go, she found herself in front of a beautiful glade. She had the impression¹ she had been already there before, but could not remember, so she entered the glade.

Once inside, she found a fairy flapping her wings and staring at her. The fairy said: “Welcome back, little Cryptess. I am the fairy Cryptophia and since you have found my magical glade, I grant you one wish.” Cryptess thought about it for a second, then she asked: “Do you know about zero-knowledge proofs?”. “Of course I do!”, replied the fairy. Then, Cryptess continued: “Here is my wish then. I give you two non-interactive zero-knowledge proof systems, out of which at least one is secure but you don’t know which one. I wish you design a non-interactive zero-knowledge proof system, based on the two I gave you, that is always secure.”

*Full version of an extended abstract that will appear in Proceedings of Eurocrypt 2026, Springer. Available from the IACR Cryptology ePrint Archive as Report 2025/240.

¹Indeed, she was right [Mit13], but for some reason she had forgotten about it.

Cryptophia immediately replied: “I am sorry. Unfortunately, what you ask for is impossible.” Cryptess was surprised at first, but after some moment of thought something kicked in her head. Then, the glade disappeared and the little girl found herself on the ground, in the middle of the forest. “Maybe, if I assume three proof systems instead of two...” Then, she smiled, and ran home. Now, she knew what to do.

1.1 Robust NIZK combiners

A combiner $\mathcal{C}$ for a primitive $\mathcal{P}$ is a cryptographic construction that, given access to a set of candidate instantiations of $\mathcal{P}$, itself instantiates the primitive $\mathcal{P}$. In particular, a t -out-of- n combiner $\mathcal{C}$ is *secure* assuming at least $t < n$ of the candidates $\Pi_1, \dots, \Pi_n$ are *secure*. This notion finds applicability in several real-world applications, as one may combine constructions relying on assumptions of different flavours (*e.g.*, classical vs post-quantum), distribute the trust placed on setup assumptions, and even provide meaningful security in the presence of compromised implementations. Of course, depending on the actual primitive $\mathcal{P}$, security can be formalized in different ways, and, indeed, combiners have been studied in the literature for a plethora of cryptographic primitives including hash functions, oblivious transfer, functional encryption schemes, and more. We review some of these results in Section 1.4.

Robust combiners. In spite of the simple intuition, capturing combiners by means of a formal definition is a non-trivial task. Indeed, one cannot just say that a combiner $\mathcal{C}$ is secure assuming at least t of the n candidates $\Pi_1, \dots, \Pi_n$ are secure. This is because $\mathcal{C}$ could always ignore all of the n candidates and construct the primitive $\mathcal{P}$ from scratch. To address this shortcoming, Harnik *et al.* [HKN⁺05] introduced the notion of *robust* combiners, which makes the reductions to the underlying candidates explicit by asking that, for every adversary $\mathcal{A}$ breaking security of $\mathcal{C}$, there exist $n - t + 1$ reductions $\{\text{Red}_i\}_{i \in [n-t+1]}$ (with running time at most a polynomial factor slower than the running time of $\mathcal{A}$) that break security of at least $n - t + 1$ distinct candidates. This way, we have a formal guarantee that $\mathcal{C}$ is secure assuming at least t of the candidates are secure (rather than some external computational assumption), and moreover $\mathcal{C}$ cannot construct $\mathcal{P}$ from scratch (as ensured by the existence of the above reductions). Within this composition, we focus on characterizing *robust* combiners for non-interactive zero-knowledge (NIZK).

A review of NIZK proof systems. A NIZK proof system for a language $\mathcal{L} \subseteq \{0, 1\}^*$ allows a prover to convince a verifier (both modelled as PPT algorithms) about the veracity of a shared statement $x \in \mathcal{L}$ using a single message π (called the proof). The prover is additionally given as input a witness w to $x \in \mathcal{L}$, and both the prover and the verifier are also given a CRS—denoted σ —that is initialized at setup. A NIZK proof system is said to be *complete* if for all $x \in \mathcal{L}$, and for all honestly sampled CRSs, the honest prover always convinces the honest verifier about the veracity of the statement. As for security, NIZK proof systems are required to satisfy different² properties. We recall the most important ones below.

Soundness: No computationally³ unbounded malicious prover (given the CRS σ) can produce an accepting proof π^* for a *false* statement $x \in \{0, 1\}^*$ that is accepted by the verifier.

Knowledge soundness: For some languages, false statements simply do not exist, thus making soundness trivial to achieve. In such cases, a stronger form of soundness (called *knowledge soundness*) is required, which roughly says that any successful prover must in

²In fact, one can consider further security properties (such as witness indistinguishability or simulation soundness), but in this paper we only focus on the properties above.

³When soundness holds computationally, we speak of *arguments* instead of proofs.

fact *know* a witness. The latter is formalized by the existence of a PPT extractor $\mathcal{E}(\xi, x, \pi)$ that for every (possibly adversarial) accepting proof π outputs a valid witness $\tilde{w}$ to $x \in \mathcal{L}$ with high probability; the extractor is facilitated by a trapdoor ξ that is generated along with the CRS σ .

Zero knowledge: For every true statement $x \in \mathcal{L}$, honestly computed proofs π reveal nothing about the witness. The latter is formalized by the existence of a PPT simulator $\mathcal{S}(\tau, x)$ that outputs a proof $\tilde{\pi}$ that is computationally indistinguishable from an honestly computed proof π ; the simulator is facilitated by a trapdoor τ that is generated along with the CRS σ . This corresponds to so-called black-box zero knowledge; in the non-black-box case, for every adversary there exists an adversary-dependent simulator.

Combiners for NIZK. In prior literature, NIZK was captured in the multi-string setting [GO07], *i.e.*, NIZKs that remain secure as long as a subset of the CRSs is secure. This technique can be regarded as a restricted class of combiners in which all candidates run the same (uncompromised) prover and verifier algorithms.

More general combiners were also discussed in the context of NIZK amplification [GJS19, BKP⁺24, BG24], and in a concurrent and independent work [AK25]. However, none of these works explicitly focus on formally defining combiners. Moreover, the constructions therein contained assume some external computational assumptions (one-way functions or polynomially-secure PKEs), making them incompatible with robustness. We provide a detailed discussion of how our results compare to the above in Section 1.4.

1.2 Our contributions

In this composition, we put forth the first comprehensive characterization of *robust* NIZK combiners, including definitions, impossibility results, and constructions. This requires us to overcome several technical pitfalls and introduce novel proof techniques.

Definitions. Our definitions capture the spirit of robust combiners of [HKN⁺05], with non-trivial adaptations due to NIZK being a multi-property primitive that admits computational and statistical security properties (more in the technical overview). At the same time, we consider different levels of black-boxness along three dimensions: (i) whether the combiner $\mathcal{C}$ accesses $\Pi_1, \dots, \Pi_n$ as a black-box; (ii) whether the zero-knowledge simulator accesses the adversary as a black-box; (iii) whether the knowledge extractor (in the case of knowledge soundness) accesses the adversary as a black-box. See Section 2 for the formal definitions. For simplicity, in the rest of this introduction, we only focus on dimension (i) and always assume black-box extraction and simulation (however, our results also cover non-black-box extraction and simulation).

Negative results. After having formally defined the notion of robust NIZK combiners, in Section 3, we argue that there exists no t -out-of- n (even non-black-box) robust NIZK combiner for $t \leq \lfloor n/2 \rfloor$, unless the polynomial hierarchy collapses. This impossibility is unconditional and is sufficient to rule out robust *multi-property* NIZK combiners. We prove this impossibility by showing that every combiner would fail if run on input half NIZK schemes that are zero-knowledge (but not sound) and half NIZK schemes that are sound (but not zero-knowledge). One may see our counterexample as contrived, and rightfully wonder whether it is possible that there exists a combiner that works if we assume that half of the input schemes are both zero-knowledge and sound (whereas no security at all is guaranteed by the remaining input schemes).

Combiner	Threshold	Type	Input/Output Language	Proof Size
§4	$t > \lfloor n/2 \rfloor$	Black-box	Homomorphic	$n \cdot \pi $
§5	$t > \lfloor n/2 \rfloor, n, t = O(1)$	Non-black-box	NP	$\text{poly}(\pi)$
§6	$t > \lfloor 2n/3 \rfloor$	Non-black-box	NP	$n \cdot \pi + \text{poly}(x , w , \lambda)$

Table 1: Comparing our results for robust NIZK combiners in terms of threshold parameters, type of black-boxness, supported language, and proof size. In particular, by *Input/Output Language*, we refer to both the language that must be supported by the input candidates and the language supported by the NIZK returned from the combiner. From the table, it can be seen that all our combiners are *language preserving*. We assume that all the input protocols issue proofs of size at most $|\pi|$ (where $|\pi|$ is at least polynomial in the security parameter λ). We note that the first and the second constructions preserve the succinctness of the input candidates.

We show that no *black-box* combiner can work in this setting either. However, this latter impossibility is conditioned on the existence of (i) NIZK with statistical soundness (and computational zero-knowledge), (ii) NIZK with statistical zero-knowledge (and computational soundness), and (iii) dual-mode commitments⁴.

Positive results. To complement the above negative results, we provide constructions of robust NIZK combiners. In particular, we obtain three different (and incomparable) combiners (see Table 1):

- Our first construction yields a t -out-of- n black-box robust NIZK combiner assuming $t > \lfloor n/2 \rfloor$, where $n, t = \text{poly}(\lambda)$. This construction supports both soundness as well as knowledge soundness, and works for so-called *homomorphic* languages [Cra96], as defined by Maurer [Mau09], which essentially means that the witness w belongs to a group $(G, \star)$ and the statement x belongs to a group $(H, \otimes)$, and moreover there is a function $f : G \rightarrow H$ such that $f(x) \otimes f(y) = f(x \star y)$ and f is one-way function (*i.e.*, it is infeasible to compute x from $f(x)$ for a randomly chosen x). Concrete examples of such languages include the language for proving the knowledge of a discrete logarithm, that a tuple is Diffie-Hellman, the equality of embedded values, and more. These languages have already been proven to be well prone to black-box transformations [CPS⁺16b, CPS⁺16a].
- Our second construction yields a t -out-of- n non-black-box robust NIZK combiner assuming $t > \lfloor n/2 \rfloor$, where $n, t = O(1)$. This construction works for arbitrary languages.
- Our third construction yields a t -out-of- n non-black-box robust NIZK combiner assuming $t > \lfloor 2n/3 \rfloor$, where $n, t = \text{poly}(\lambda)$. This construction also works for arbitrary languages.

Due to our impossibility results, our first and second constructions are essentially tight in terms of achievable parameters. We leave it as an interesting open problem to construct a black-box robust NIZK combiner supporting all of NP for the parameters regime $t > \lfloor n/2 \rfloor$ with $n, t = \text{poly}(\lambda)$.

1.3 Technical overview

Below, we give an overview of the main technical ideas behind our results.

⁴A dual-mode commitment scheme is a commitment scheme defined in the CRS mode, where the CRS can be sampled in two modes, in such a way that the commitment is statistical binding or statistical hiding. This primitive can be instantiated from a variety of assumptions, including DDH [Lin15].

1.3.1 Defining robust NIZK combiners

A NIZK scheme should enjoy at least two security properties: zero-knowledge and soundness, which can either be computational or statistical. We discuss how these characteristics are embedded in our definitions.

Capturing the multi-property nature of NIZK. The first design choice stems from the multi-property nature of NIZK: in order for the combiner to be secure, should t candidates satisfy all the security properties at once, or does it suffice for t candidates to be secure with respect to each *individual* property? In this work, we follow the latter (more general) approach. Indeed, for our combiners, it suffices that t candidates are (knowledge) sound and t candidates are zero-knowledge, but these two subsets may not be identical. Hence, ours are *multi-property combiners* as defined in [FL08, FLP08, FLP14]. On the other hand, we implicitly assume that all of the candidates have a negligible completeness error: a common assumption in the literature on combiners.⁵

Characterizing robustness. As remarked in the introduction, in order to establish robustness of a combiner C , for every adversary breaking the security of C , one has to design $n - t + 1$ reductions, each breaking the security of a distinct candidate. The same rationale applies also in the context of a multi-property primitive: if an adversary successfully attacks *any* of the security properties of the combiner, there should exist $n - t + 1$ reductions to one of the security properties of the candidates. We capture this requirement by having two distinct definitions: one for soundness, and one for zero-knowledge. Roughly, we say that a robust combiner is sound if, for every adversary $\mathcal{A}$ breaking the soundness of C , there exist at least $n - t + 1$ reductions that break either the zero-knowledge of at least $n - t + 1$ distinct candidates *or* the soundness of at least $n - t + 1$ distinct candidates. The definition for zero-knowledge follows similarly. We stress that this definition allows, for example, to reduce the soundness of the combiner to the zero-knowledge of the input primitives. Despite this looking like an artificial way to make our definition more general, this characterization perfectly captures the spirit of robust combiners: the combiner should be allowed to leverage all the security properties of its candidates to itself implement the desired primitive. Later in this section, we show that this characterization is indeed instrumental to the design of certain combiners.

Capturing computational and statistical properties. Our characterization of robustness states that, for any adversary attacking the combiner in time τ , there exist reductions running in time $\text{poly}(\tau)$ attacking at least $n - t + 1$ distinct candidates. Hence, our combiners are general enough to accept as input candidates that have computational or statistical properties while outputting a NIZK that has computational or statistical properties.

A relation with standard security properties. As a sanity check, throughout the formal discussion in Section 2, we also argue that any robust NIZK combiner satisfying our definition, when instantiated with at least t secure NIZK candidates, yields a NIZK satisfying the standard definitions of (knowledge) soundness and zero-knowledge. Thus, any robust combiner satisfying our definitions can be employed directly in any application requiring these properties.

⁵But note that in the setting of NIZK proof systems, one can always amplify non-negligible completeness to overwhelming completeness (at the price of a larger running time for the prover). Alternatively, one can apply solutions à la [HKN⁺05] by having the combiner pre-emptively “testing” the correctness of each candidate.

Why is our definition the right one? Our definitions capture exactly what one should expect from a *robust* combiner - namely, that the security of the combiner only relies on the input candidates. In particular, in our definition, we state that the only way to attack the security of the combiner is to attack the security of the input candidates, meaning that the combiner cannot *create security*. We stress that this implies that the combiner cannot rely on additional computational assumptions (that are not directly implied by the input candidates), but also that no information-theoretic security should be created. For example, a combiner that, on input only NIZK candidates that enjoy computational zero-knowledge and computational soundness, returns a NIZK scheme that is statistically zero-knowledge would not satisfy our security definition. Indeed, in such a case, an adversary attacking the zero-knowledge (hence, unbounded) can never be meaningfully turned into an adversary attacking any of the properties of the input schemes, as they are guaranteed to hold only against polynomial-time adversaries. We believe that this feature captures well the fact that a combiner should be *simple* and should not create security without relying on the input schemes.

This is, for example, in contrast with [HKNY23] where the authors construct a combiner for commitment schemes, which always return a statistical binding commitment (regardless of the security of the input schemes). We refer to Appendix A for a more detailed comparison.

1.3.2 Impossibility results

Next, we explain how to prove that no robust combiner exists unless assuming the majority of the candidates are *good*. In all our proof sketches, we will consider the case where $n = 2$ and $t = 1$. Assume by contradiction that there exists a 1-out-of-2 robust NIZK combiner C . Consider the following dummy NIZK candidates: (i) the prover in Π_1 , upon input (x, w) , always outputs 0 and the verifier only accepts 0 as a proof; (ii) the prover in Π_2 , upon input (x, w) , always outputs w and the verifier accepts if and only if w is a valid witness for x . Clearly, Π_1 is unconditionally zero-knowledge but not sound, whereas Π_2 is unconditionally sound but not zero-knowledge.

Yet, the definition of robust combiner implies that C must work for Π_1 and Π_2 . In particular, given any unbounded adversary that breaks either the zero-knowledge or the soundness property of C , there should exist two reductions, with at most a polynomial slowdown, that break either the zero-knowledge or the soundness of both Π_1 and Π_2 ; but this is impossible, as Π_1 is unconditionally zero-knowledge (resp. Π_2 is unconditionally sound). It follows that no (even unbounded) adversary can break C , and in particular that C must be both unconditionally sound and unconditionally zero-knowledge. But such a combiner cannot exist, unless the polynomial hierarchy collapses. Note that, for the above argument, it is completely irrelevant whether the combiner is black-box or not.

Ruling out more types of combiners: Step 1. We note that the proof we have just sketched crucially relies on the fact that C is a multi-property combiner. This, informally, requires that the combiner works even if one instance is zero-knowledge (but not sound) and the other instance is sound (but not zero-knowledge). In particular, our impossibility proof does not rule out the existence of a combiner that retains its security when receiving as input one scheme that is both zero-knowledge and sound (while the other scheme has no security at all).

We show that even in this case, no combiner can exist. Assume there exists a NIZK protocol Π_{SSND} that is statistically sound and computationally zero-knowledge, and a protocol Π_{SZK} that is computationally sound and statistically zero-knowledge. Suppose now by contradiction that a 1-out-of-2 combiner exists. Such a combiner, in particular, must retain its security when

executed with the input candidates Π_{SSND} and Π_{SZK} .

The security of the combiner guarantees that if there exists an adversary that breaks the soundness of the combiner $\mathcal{C}$, then it is possible to construct a reduction that either breaks the soundness or the zero-knowledge of both input candidates. However, because we have assumed Π_{SSND} to be statistically sound, no (even unbounded) reduction can break the soundness of both Π_{SSND} and Π_{SZK} . Similarly, because we have assumed that Π_{SZK} is statistical zero-knowledge, then no (even unbounded) reduction can break the zero-knowledge of both Π_{SSND} and Π_{SZK} .

Hence, this means that it cannot be that an (unbounded) adversary breaks the soundness of $\mathcal{C}$ (*i.e.*, the combined scheme is statistically sound). Using a similar argument, we can claim that the zero-knowledge of $\mathcal{C}$ is statistical. But as recalled above, no NIZK can be at the same time statistically sound and statistically zero-knowledge unless the polynomial hierarchy collapses.

Note that our impossibility is based solely on the existence of Π_{SSND} and Π_{SZK} , hence, it relies on the assumption that it is possible to design statistical zero-knowledge (computationally sound) and statistically sound (computationally zero-knowledge) NIZK protocols, but we do not need to rely on any specific instantiation of Π_{SSND} and Π_{SZK} .

Ruling out more types of combiners: Step 2. The impossibility we have sketched above does not rule out the existence of a combiner that always produces a NIZK scheme that is only computational zero-knowledge and computational sound, as long as one of the input candidates is both computational sound and zero-knowledge.

Hence, one may wonder whether such a combiner exists. We show that this is again not the case, albeit only for black-box combiners. Suppose by contradiction that such a combiner $\mathcal{C}$ does exist. By assumption, the combiner must also work when it is run on input two schemes Π_1 and Π_2 , both computationally zero-knowledge and computationally sound. The idea of our proof is to consider special instantiations of Π_1 and Π_2 , which we gradually modify into two new protocols Π_{SZK} and Π_{SSND} , such that (as before) Π_{SZK} is statistical zero-knowledge (but computational sound), and Π_{SSND} is statistical sound (but computationally zero-knowledge). If the behaviour of (Π_1, Π_2) is computationally indistinguishable from the behaviour of $(\Pi_{\text{SSND}}, \Pi_{\text{SZK}})$, then $\mathcal{C}$ keeps working when it is executed on Π_{SZK} and Π_{SSND} . But now, following the arguments of our previous impossibility, we can again use $\mathcal{C}$ to construct a NIZK scheme that is statistical zero-knowledge and statistical sound (which cannot exist), thus reaching a contradiction. We refer to Section 3 for the precise descriptions of (Π_1, Π_2) and $(\Pi_{\text{SSND}}, \Pi_{\text{SZK}})$, and for all the formal details of our three impossibility proofs.

1.3.3 Combiner for homomorphic languages

We start with the combiner for homomorphic languages. For simplicity, we describe the combiner for proving knowledge of the discrete logarithm $w \in \mathbb{Z}_q$ of a group element $x = g^w \in \mathbf{G}$ (where g is the generator of the cyclic group $\mathbf{G}$ with order q). In the formal part of the paper, we give a more general construction that works for a large class of homomorphic languages, as defined by Maurer [Mau09]. Concrete examples of such languages include Diffie-Hellman, Guillou-Quisquater, proofs for equality of embedded values, and more.

Assume we have n NIZK argument systems $\Pi_1, \dots, \Pi_n$, each for proving knowledge of the discrete logarithm of a group element. We aim at constructing a combiner that returns a NIZK argument system Π for the language of discrete logarithms and which guarantees security (*i.e.*, zero-knowledge and knowledge soundness) assuming that t out of the n input schemes are secure.⁶ The prover takes a random polynomial p of degree $t - 1$ such that $p(0) = w$, and

⁶Here, for simplicity, when we say that a scheme is secure, we mean that it enjoys both zero-knowledge and (knowledge) soundness. However, we remark that our construction also works in case the subsets of candidates

computes the shares $(i, p(i))$ for each $i \in [n]$, thus obtaining a t -out-of- n Shamir's secret sharing of the witness w . Then, the prover derives n sub-statements $x_i \leftarrow g^{p(i)}$ and runs the prover algorithm of the scheme Π_i on input the sub-statement x_i and its witness $p(i)$ for each $i \in [n]$, obtaining a proof π_i . The final proof π consists of all the generated proofs $\{\pi_i\}_{i \in [n]}$ and all the sub-statements $\{x_i\}_{i \in [n]}$. The verifier first checks that all the proofs are accepting (with respect to each individual scheme), and then runs the reconstruction of Shamir's secret sharing "in the exponent" in order to check that $g^{p(0)}$ equals x and that $g^{p(i)}$ equals x_i for all $i \in [n]$. The latter is possible because Shamir's reconstruction is based on Lagrange interpolation, which basically just requires the verifier to sum t polynomial evaluations (multiplied by some constant), and the latter can be done "in the exponent" by exploiting the homomorphic properties of the function $f(\alpha) = g^\alpha$ between the groups G and H (with $H = G$ in this case).

Zero-knowledge is guaranteed because the witnesses w_i of at least t of the sub-statements x_i are protected (thanks to the assumption that t candidates are zero-knowledge and that $t > \lfloor n/2 \rfloor$), hence the privacy property of Shamir's secret sharing implies that the witness w is protected. Knowledge soundness is guaranteed by the fact that, for at least t of the sub-statements x_i , the proof will be generated using a knowledge sound candidate, hence, it will be possible to extract t shares (polynomial evaluations), which we can use to reconstruct a candidate witness w' . It remains to argue that w' corresponds to w . But this must be the case, given that $g^{p(0)} = g^{w'}$.

1.3.4 Recursive proof combiner

The combiner we have just described works for a limited class of languages. Next, we propose a NIZK combiner that supports all of NP, assuming the underlying candidates also support all of NP. To simplify the description of our scheme, let us consider here the case where $n = 3$ and $t = 2$. The prover performs a recursive proof, using all possible combinations of the given candidates. In other words, the prover computes a proof that $x \in \mathcal{L}$ using the i -th candidate, thus obtaining a proof π_i ; then, using the scheme j (with $j > i$) it proves the following: "I know a proof π_i for the statement $x \in \mathcal{L}$ which the verifier of the scheme Π_i would accept". We denote this latter proof with $\pi_{i,j}$. The prover proceeds as above for each $i, j \in \{1, 2, 3\}$ with $j > i$, and finally it sends to the verifier $\pi_{1,2}, \pi_{1,3}, \pi_{2,3}$. The verifier accepts if all three proofs are accepting.

The above scheme is zero-knowledge, because every proof $\pi_{i,j}$ hides the witness: either the i -th candidate is zero-knowledge (and hence w is protected), or the j -th candidate is zero-knowledge (and hence the proof π_i , that could have leaked something about the witness, remains hidden to the adversary). Note that this holds given that $t > \lfloor n/2 \rfloor$. Knowledge soundness holds, because there exists at least one recursive proof $\pi_{i,j}$, generated using the i -th and the j -th candidates which are both knowledge sound (this again follows from the fact that $t > \lfloor n/2 \rfloor$). The construction can be extended to the case where n and t are arbitrary, but to keep the running time of the prover polynomial, we must require that n and t are constant.

1.3.5 MPC-in-the-head combiner

With our third construction we want to remove the limitation on n and t , while still supporting all of NP. The construction is inspired by the MPC-in-the-head approach proposed in [IKOS07], and a similar approach has been used in [GJS19] to construct amplifiers for zero-knowledge from sub-exponentially secure public-key encryption.

that are either zero-knowledge or (knowledge) sound are not identical.

Our combiner is based on an n -party information-theoretic multi-party computation (MPC) protocol for the following function g : *given as input n shares of a t -out-of- n secret sharing scheme, perform the reconstruction thus obtaining a value w , and return 1 if and only if w is a valid witness for x .* The MPC protocol guarantees security as long as k parties are corrupted (with $k < n/2$).

To simplify the description of the combiner, let us first assume that the combiner has access to a non-interactive commitment scheme; we will explain later how to remove this assumption. Let $\mathbf{pt}_1, \dots, \mathbf{pt}_n$ be the n parties of the MPC protocol. The prover of our combiner computes a t -out-of- n secret sharing of the witness w , thus obtaining the shares $w_1, \dots, w_n$, and executes the MPC protocol for the above function g in its head simulating the role of each party $\mathbf{pt}_i$ with input w_i . This process generates, for each party $\mathbf{pt}_i$, a set of ingoing and outgoing messages that we denote with in-out_i for $i \in [n]$. The prover then, for each $i \in [n]$, commits to these messages and uses the candidate Π_i to prove that the messages in in-out_i correspond to the view generated by running party $\mathbf{pt}_i$ of the above MPC protocol. The prover finally sends all the generated proofs and all the commitments to the values in-out_i .

The verifier first checks that there is a match between the commitments corresponding to the ingoing messages of a party $\mathbf{pt}_i$ and the outgoing messages of all the other parties, and accepts the proof if all the proofs are valid. Intuitively, the zero-knowledge property is guaranteed given that t shares are protected by the zero-knowledge property of the t secure candidates, and moreover, the security of the MPC protocol guarantees that, even if the messages of k parties are controlled by the adversary (hence, the adversary knows the input of these parties), then the shares of the *honest parties* are still protected thanks to the hiding property of the commitment. Hence, due to the privacy of the secret sharing scheme, the witness itself is protected.

To argue (knowledge) soundness, ideally we would like to rely on the correctness of the MPC protocol, similarly to what is done in [IKOS07]. Unfortunately, we cannot do that, given that a dishonest prover can compute some of the messages in a malicious way, which rules out relying on the correctness of the MPC protocol. To solve this problem we require the MPC protocol to be *robust*. This property was used also in [IKOS07] to prove some of their results. The robustness property requires that even if the randomness used by the honest parties is chosen adversarially, and $n - t$ parties are fully controlled by the adversary, then the output of the honest parties is always correct. One possible instantiation of such an MPC protocol is [BGW88], which tolerates a corruption threshold of $k < \lfloor \frac{n-1}{3} \rfloor$, and, as claimed in [GMO16], is also robust.

It remains to explain how to remove the assumption that the combiner is given access to a secure non-interactive commitment scheme. We do that by having the combiner construct a non-interactive commitment scheme using the given NIZK candidates themselves. More precisely, we rely on [HN24, LMP24], in which the authors prove that one-way functions exist if: (i) every language in NP has a zero-knowledge argument (*i.e.*, $\text{NP} \subseteq \text{ZKA}$), and (ii) ZKA contains non-trivial languages (*i.e.*, $\text{ZKA} \not\subseteq \text{ioP/poly}$). Hence, our combiner can first obtain a one-way function candidate f_i using the NIZK candidate Π_i , combine⁷ the one-way function candidates $\{f_i\}_{i \in [n]}$ into a secure one-way function construction f , and finally use f to obtain⁸ a two-round commitment using Naor's construction [Nao91], that we can make non-interactive by plugging the first round of the commitment in the CRS. This makes our combiner unconditional under assumptions (i) and (ii); however, note that if assumption (i) does not hold, there are languages in NP that do not admit zero-knowledge arguments (and hence, the problem of designing NIZK

⁷Robust combiners for one-way functions are well-known [Her05], for instance the classical concatenation combiner with input splitting would work.

⁸Naor's construction requires a pseudorandom generator, which can be constructed from one-way functions [HILL99].

combiners becomes less interesting). Moreover, if (ii) does not hold, we have that $\text{NP} \subseteq \text{ioP/poly}$ (as $\text{NP} \subseteq \text{ZKA}$ must hold, and $\text{ZKA} \subseteq \text{ioP/poly}$). We stress that the above transformation fundamentally relies on both soundness *and* zero-knowledge of the candidates. In particular, while binding is unconditional, hiding relies on the security of the OWF, which in turn relies on the assumption that at least one candidate is simultaneously sound and zero-knowledge. This occurs if at least $n/2 + 1$ candidates are sound and $n/2 + 1$ candidates are zero-knowledge.

This fact is instrumental in proving the security of the overall combiner. In particular, note that the zero-knowledge property of the combiner relies on the hiding of the commitment, which, as just mentioned, relies also on the soundness of the input candidates. Hence, our combiner represents a concrete example where the zero-knowledge property surprisingly relies on the soundness of the input candidates. More precisely, when formally proving the zero-knowledge property of the combiner, we will show that if an adversary breaks the zero-knowledge of the combined scheme, then this adversary could be used to break either the *soundness* or the zero-knowledge of at least $n - t + 1$ input candidates.

We finally note that, since the prover runs the NIZK candidates to prove a statement involving the commitments (which, in turn, are obtained using the NIZK candidates themselves), the combiner is non-black-box in the use of the candidates.

1.4 Related works

Combiners. Several previous works have considered and applied the concept of combiners, either implicitly or explicitly. The first formal treatment of robust combiners is due to Harnik *et al.* [HKN⁺05], in the setting of oblivious transfer and key agreement protocols. In particular, this was the first work pointing out the necessity of making the reduction explicit in the definition, in order to rule out trivial solutions that ignore the underlying candidates.

Most relevant to our work is Sommer’s Master Thesis [Som06] that studies robust combiners for interactive proof systems. This work rules out very specific forms of robust combiners (*e.g.*, those that are deterministic or that work by splitting the statement into multiple parts).

A long line of research studies robust combiners for hash functions [FL08, FLP08, Mit13, MP14]. In this context, it is particularly challenging to obtain *short* combiners, in which the output length of the combiner is significantly shorter than the concatenation of the candidates’ outputs. While there are lower bounds for short robust hash functions combiners [BB06, CRS⁺07, Pie07, Pie08, Mit12], recent work has shown that such lower bounds can be overcome in the random oracle model [DFG⁺23]. Robust combiners have also been considered for other cryptographic primitives, including public-key encryption [HKN⁺05, DK05], oblivious transfer [HKN⁺05, Som06, MPW07, CDFR17, FR24], commitment schemes [Som06], private information retrieval [MP06], key encapsulation mechanisms [GHP18], authenticated encryption with associated data [PR20], obfuscators [AJS17, FHNS16] and functional encryption [AJN⁺16, ABJ⁺19, JMS20].

Amplifiers. A recent line of work studies amplification of NIZK proof and argument systems [GJS19, BKP⁺24, BG24]. Intuitively, a NIZK amplifier takes as input an $(\varepsilon_s, \varepsilon_z)$ -weak NIZK—where the soundness and zero-knowledge errors are, respectively, $\varepsilon_s, \varepsilon_z$ for $\varepsilon_s + \varepsilon_z < 1$ —and turns it into a NIZK (for the same language) with negligible soundness and zero-knowledge errors. NIZK amplifiers imply NIZK combiners: by selecting at random one of the NIZK proof systems given to the combiner, we get a weak NIZK that we can then amplify. Following this approach, one obtains [BG24]:

- A robust combiner for NIZK arguments assuming polynomially-secure public-key encryption.

- A robust combiner for NIZK proofs assuming polynomially-secure one-way functions.

In contrast, our combiners are unconditional in the sense that they do not require any additional assumptions besides the fact that t out of n of the given NIZK proofs systems are secure. Another limitation is that the approach based on amplifiers currently does not seem to preserve the proof/argument of knowledge property of the input protocols, while all of our combiners do. We also note that if we want to design a combiner that works when the input candidates are *proof* systems (with unconditional soundness) and we do not care about the property of PoK/AoK, then we can obtain the following combiner that works for any threshold $t > \lfloor n/2 \rfloor$. On input n NIZK proof system candidates the combiner constructs a OWF using the technique described in Section 1.3 (§MPC-in-the-head combiner), picks a random NIZK input candidate and applies the amplifier of [BG24] (we can use this amplifier given that it only requires the existence of OWFs and a weak NIZK as input). However, such an approach does not work if the input candidates are argument systems, as in this case the best-known amplifier [BG24] requires the existence of public-key encryption. In summary, the combiner we have just sketched works only for proof systems, while in this paper we propose (unconditional) combiners that work for both argument and proof systems while preserving the PoK/AoK property (if any) of the input protocols.

Concurrent and independent work. In a concurrent and independent work [AK25], assuming one-way functions, the authors show how to obtain a t -out-of- n NIZK combiner for all of NP, with optimal parameters $t > \lfloor n/2 \rfloor$ and $t, n = \text{poly}(\lambda)$. This combiner, compared to our first two constructions, does not preserve the succinctness of the input schemes, and moreover relies on one-way functions (however, this assumption can be avoided using our approach sketched above, at the cost of making the combiner non-black-box). The authors also provide a combiner that preserves the succinctness of the input scheme, assuming collision-resistant hash functions. In comparison, the first two constructions we propose do not rely on any additional assumptions, but instead support a restricted class of languages or work under the assumption that n, t are constant.

1.5 Notation

We denote the set of natural numbers with $\mathbb{N}$, and the set of integers of prime order q with $\mathbb{Z}_q$. We denote “ $\leftarrow$ ” as the assigning operator (*e.g.*, to assign to a the value of b we write $a \leftarrow b$), and “ $\stackrel{\$}{\leftarrow}$ ” as the sample operator (*e.g.*, $x \stackrel{\$}{\leftarrow} Q$ denotes the sampling of x from a distribution Q). We use “ $=$ ” to check equality of two different elements. We use $\text{poly}(\cdot)$ to indicate a generic polynomial function, $\text{negl}(\cdot)$ to denote a negligible function, and $\text{nonnegl}(\cdot)$ to denote a non-negligible function. We use $\approx_c$ (resp. $\not\approx_c$) to denote computational indistinguishability (resp. distinguishability) of two ensembles. We use $\approx_p$ to denote that two ensembles are identical. While referring to a set T , we denote T_j as the subset containing the first j indices of T . With respect to an NP language $\mathcal{L}$, we denote $\mathcal{R}_{\mathcal{L}}(x, w)$ as its corresponding NP-relation.

2 Defining combiners

Our combiner definitions follow the spirit of the notion for hash-function combiners proposed in [Pie08]. At a high level, in [Pie08], the authors defines black-box combiners for collision resistant hash functions as a tuple of algorithms $(\mathcal{C}, \mathcal{A})$, where $\mathcal{C}$ is the algorithm combining n candidates (the hash function protocols in the case of [Pie08]). Intuitively, the security required from a t -out-of- n combiner requires that if an adversary breaks the security of the combiner, then it is possible to define reductions that breaks the security of at least $n - t + 1$ candidates.

This captures the fact that the security of the combiner relies only on the security of at least t input candidates, and no additional cryptographic assumption is used (*i.e.*, it prevents the combiner from just ignoring the input candidate and creating from scratch a hash function).

In this work, we follow a similar blueprint and extend it to the case of non-interactive zero-knowledge. Intuitively, a robust NIZK combiner takes n input candidates, where t of these schemes are sound and t are zero-knowledge (the subset of secure schemes for each property may differ), to itself achieve soundness and zero-knowledge. In proving security for each of these properties, the combiner could very well use *all* the security properties provided by the candidates. More concretely, a crafty combiner could use the soundness of the candidates as part of its proof for zero-knowledge (we cover this case in the proof of zero-knowledge of our third construction, detailed in Section 6).

The definitions we put forward are general enough to capture whether the combiner has *black-box* or *non-black-box* access to the underlying primitives, and whether it considers black-box or non-black-box zero-knowledge. Moreover, whenever a combiner is secure according to our definitions, it is also secure according to the *standard* notions of zero-knowledge and soundness. Throughout this section, we provide formal definitions and the formal arguments of the above claims.

We conclude the section by also introducing a variation of the above definitions that allows us to capture combiners that consider knowledge soundness in place of soundness.

2.1 Formalizing correctness

From a correctness standpoint, our goal consists of combining non-interactive complete protocols, which we formally define as follows.

Definition 1 (Set of Non-Interactive Complete Protocols $\mathcal{F}_{\text{NI}}$ for the language $\mathcal{L}$). *Let $\Pi = (\text{setup}, \mathcal{P}, \mathcal{V})$ be a tuple of algorithms defined as follows:*

- $\text{setup}(1^n, 1^\lambda)$ takes as input a statement of length n and the unary description of the security parameter λ . It outputs a public parameter pp .
- $\mathcal{P}(pp, x, w)$ takes as the input a public parameter pp , a statement x and a witness w , s.t. $(x, w) \in \mathcal{R}_{\mathcal{L}}$. It outputs a proof π .
- $\mathcal{V}(pp, x, \pi)$ takes as the input a public parameter pp , a statement x and a proof π . It outputs 1 to accept and 0 to reject.

Then, $\Pi \in \mathcal{F}_{\text{NI}}$ if, for all $\lambda \in \mathbb{N}$ and all $(x, w) \in \mathcal{R}_{\mathcal{L}}$, where $\mathcal{L}$ is an NP language, it holds that:

$$\Pr \left[\mathcal{V}(pp, x, \mathcal{P}(pp, x, w)) = 1 \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda) \right] \geq 1 - \text{negl}(\lambda).$$

With that, we can properly characterize correctness for $\mathcal{F}_{\text{NI}}$ combiners. Roughly, whenever one defines a combiner $\mathcal{C}$ that combines $\mathcal{F}_{\text{NI}}$ candidates, $\mathcal{C}$ itself has to be a $\mathcal{F}_{\text{NI}}$.

Definition 2 (Correctness of an n -candidate $\mathcal{F}_{\text{NI}}$ -combiner). *Let $\mathcal{C}$ be a PPT algorithm. $\mathcal{C}$ is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner if the following holds:*

$$\forall PPT f_1, \dots, f_n \in \mathcal{F}_{\text{NI}}, \mathcal{C}^{f_1, \dots, f_n} \in \mathcal{F}_{\text{NI}}.$$

Note that, by our definition of $\mathcal{F}_{\text{NI}}$, it is permissible to design combiners for some language $\mathcal{L}$ while using candidates for different languages $\mathcal{L}_1, \dots, \mathcal{L}_n$. This approach makes our definition more general while strengthening our impossibility results.

2.2 Security breaks for soundness and ZK

Following the introduction of this section, our goal consisting in designing *robust* NIZK combiners, meaning that security should *exclusively* derive from the security of the candidates. Intuitively, in order to state that a security property (*i.e.*, soundness or ZK) of the combiner is satisfied, it suffices to show that any adversary breaking the soundness of the combiner *must* have broken soundness or ZK for at least $n - t + 1$ of the candidates. Before moving to the formal definitions of security for our combiners, we clearly formalize what it means to “break” our security properties of interest (*i.e.*, soundness, ZK), and how these breaks relate to standard definitions of such properties.

2.2.1 Breaking soundness

In order to capture soundness, we find it convenient to define the relation $\mathcal{R}_{\text{Sound}}$. This relation contains all the tuples (*protocol, adversary*) such that the adversary outputs an accepting proof for a false statement. More formally:

Definition 3 ($\mathcal{R}_{\text{Sound}}$). *Let $\Pi \in \mathcal{F}_{\text{NI}}$, and let $\mathcal{A}$ (*i.e.*, the adversary) be an algorithm. We say that $(\Pi, \mathcal{A}) \in \mathcal{R}_{\text{Sound}}$ if the following holds:*

$$\Pr \left[\mathcal{V}(pp, x, \pi) = 1 \wedge x \notin \mathcal{L} \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda); (x, \pi) \xleftarrow{\$} \mathcal{A}(pp) \right] \geq \text{nonnegl}(\lambda).$$

Notice that $\mathcal{R}_{\text{Sound}}$ is, by itself, sufficient to capture a security break for soundness in both its computational and statistical flavours. For example, by the standard definition of computational soundness (reported in Definition 16), a protocol Π is secure if there exists no efficient adversary $\mathcal{A}$ that can craft an accepting proof for a false statement with better non-negligible probability (*i.e.*, $\forall PPT\mathcal{A}, (\Pi, \mathcal{A}) \notin \mathcal{R}_{\text{Sound}}$). Hence, as long as $\exists PPT\mathcal{A}$ s.t. $(\Pi, \mathcal{A}) \in \mathcal{R}_{\text{Sound}}$, the computational soundness of Π is broken. By lifting the *PPT* requirement for $\mathcal{A}$, one obtains statistical soundness.

2.2.2 Breaking zero-knowledge

We now move to zero-knowledge. First, we define the relation $\mathcal{R}_{\text{ZK}}$. A tuple (*protocol, (simulator, adversary)*) belongs to $\mathcal{R}_{\text{ZK}}$ if the adversary distinguishes between proofs generated using the honest prover of the protocol and proofs generated from the simulator. More formally:

Definition 4 ($\mathcal{R}_{\text{ZK}}$). *Let $\Pi \in \mathcal{F}_{\text{NI}}$, let $\mathcal{A}$ be an algorithm, and let $\mathcal{S} = (\mathcal{S}_0, \mathcal{S}_1)$ be a PPT algorithm. Let $\text{REAL} = \{(pp, x, w, \pi) \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{P}(pp, x, w)\}_{\lambda \in \mathbb{N}}$ and $\text{IDEAL}_{\mathcal{S}} = \{(pp, x, w, \pi) \mid (pp, \tau) \xleftarrow{\$} \mathcal{S}_0(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{S}_1(pp, \tau, x)\}_{\lambda \in \mathbb{N}}$. We say that $(\Pi, (\mathcal{A}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}}$ if the following holds:*

$$\Pr \left[b = b' \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\} \\ b' \leftarrow \mathcal{A}(pp, x, w, \pi) \end{array}; (pp, x, w, \pi) \xleftarrow{\$} \begin{cases} \text{REAL} & \text{if } b = 0 \\ \text{IDEAL}_{\mathcal{S}} & \text{if } b = 1 \end{cases} \right] \geq \frac{1}{2} + \text{nonnegl}(\lambda).$$

Differently from soundness, $\mathcal{R}_{\text{ZK}}$ does not capture a security break for zero-knowledge by itself, as it only refers to a specific adversary breaking a specific simulator. In order to obtain a break for either black-box or non-black-box zero-knowledge, we need to introduce quantifiers appropriately. We discuss black-box zero-knowledge in what follows, deferring non-black-box zero-knowledge to Appendix D.

Breaking black-box zero-knowledge. The standard definition of computational black-box zero-knowledge (reported in Definition 17) states that a protocol Π is secure if there exists a (universal) simulator $\mathcal{S}$ that is secure against every possible efficient adversary $\mathcal{A}$ (i.e., $\exists \mathcal{S} \text{ s.t. } \forall PPTA, (\Pi, (\mathcal{A}, \mathcal{S})) \notin \mathcal{R}_{\text{ZK}}$). Breaking black-box zero-knowledge therefore means that such a simulator cannot exist (which, in turn, means that *every* possible simulator can be attacked). This is formalized as follows: $\forall \mathcal{S} \exists PPTA \text{ s.t. } (\Pi, (\mathcal{A}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}}$. Similarly to the case of soundness, it is possible to obtain statistical zero-knowledge by considering unbounded adversaries.

2.3 Combiners for soundness and ZK

In what follows, we provide a definition of combiners for soundness and zero-knowledge, with a label $\alpha\beta \in \{\text{B}, \text{N}\}^2$ that specifies whether the combiner has black-box access ($\alpha = \text{B}$) or non-black-box access ($\alpha = \text{N}$) to its candidates, and whether the combiner relies on black-box zero-knowledge ($\beta = \text{B}$) or non-black-box zero-knowledge ($\beta = \text{N}$) of the candidates. For instance, a NB (ZK, Sound)-combiner has non-black-box access to its candidates, and considers candidates satisfying black-box zero-knowledge (i.e., the secure candidates are such that there exists a universal simulator). For convenience, we discuss soundness and zero-knowledge separately.

2.3.1 Soundness of (ZK, Sound)-combiners

Let $\mathcal{C}$ be a *correct* combiner that works having as input n candidates $f_1, \dots, f_n \in \mathcal{F}_{\text{NI}}$. Informally, we say that a (ZK, Sound)-combiner $\mathcal{C}$ is sound if the following holds: if there exists $\mathcal{A}$ running in time τ such that $(\mathcal{C}, \mathcal{A}) \in \mathcal{R}_{\text{Sound}}$ (i.e., $\mathcal{C}$ is not sound), then there must exist $n - t + 1$ adversaries (reductions) running in time $\text{poly}(\tau)$ to either soundness or ZK of distinct candidates. More specifically, if the combiner is not sound, then one of the below must hold:

1. There exists a sequence of adversaries $\text{Red}_{i_1}, \dots, \text{Red}_{i_{n-t+1}}$ such that $(f_{i_j}, \text{Red}_{i_j}) \in \mathcal{R}_{\text{Sound}}$ for all $j \in [n - t + 1]$, with $\{i_1, \dots, i_{n-t+1}\} \subseteq [n]$ (i.e., there are at least $n - t + 1$ candidates for which soundness is broken, as per Section 2.2).
2. For all possible choices of simulators $\mathcal{S}_{i_1}, \dots, \mathcal{S}_{i_{n-t+1}}$, $\exists \text{Red}_{i_1}, \dots, \text{Red}_{i_{n-t+1}}$ such that $(f_{i_j}, (\mathcal{S}_{i_j}, \text{Red}_{i_j})) \in \mathcal{R}_{\text{ZK}}$ for all $j \in [n - t + 1]$ (i.e., there are at least $n - t + 1$ candidates for which black-box zero-knowledge is broken, as per Section 2.2).

Below, we provide a formal definition:

Definition 5 (Soundness of t -out-of- n type $\in \{\text{B}, \text{N}\}^2$ (ZK, Sound)-combiners). *Let $\mathcal{C}$ be a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner, and let $\mathcal{A}$ be an algorithm running in time τ . We say that a t -out-of- n type $\in \{\text{B}, \text{N}\}^2$ (ZK, Sound)-combiner $\mathcal{C}$ is sound if, after ordering the quantifiers according to type in Tables 2,3, the following holds:*

$$(\mathcal{C}^{f_1, \dots, f_n}, \mathcal{A}^{f_1, \dots, f_n}) \in \mathcal{R}_{\text{Sound}} \implies \text{any of the following:}$$

- $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to the soundness of distinct candidates, i.e.:

$$\exists I = \{i_1, \dots, i_{n-t+1}\} \subseteq [n] \text{ s.t., } \forall i_j \in I, \exists \text{Red}_{i_j} \text{ s.t. } (f_{i_j}, \text{Red}_{i_j}) \in \mathcal{R}_{\text{Sound}}.$$

- $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to the black-box (resp. non-black-box) zero-knowledge of distinct candidates if type = *B (resp. type = *N), i.e.:

$$\begin{aligned} &\exists I = \{i_1, \dots, i_{n-t+1}\} \subseteq [n] \text{ s.t. the following holds:} \\ &\forall i_j \in I, \text{candZKQuant s.t. } (f_{i_j}, (\text{Red}_{i_j}, \mathcal{S}_{i_j})) \in \mathcal{R}_{\text{ZK}}. \end{aligned}$$

type	Structural quantifiers	
B*	$\exists C$	$\forall f_1, \dots, f_n \in \mathcal{F}$
N*	$\forall f_1, \dots, f_n \in \mathcal{F}$	$\exists C$

Table 2: Classification of soundness for (ZK, Sound)-combiners.

type	candZKQuant
*B	$\forall \mathcal{S}_{i_j} \exists \text{Red}_{i_j}$
*N	$\exists \text{Red}_{i_j} \forall \mathcal{S}_{i_j}$

Table 3: Quantifiers for ZK of the candidates of (ZK, Sound)-Combiners. $\mathcal{S}_{i_j}$ is a *PPT* algorithm. Red_{i_j} runs in time $\text{poly}(\tau)$.

Connections with the *standard* notion of soundness. The above definition implies that, if the combiner runs with sufficiently-many candidates that are secure (*i.e.*, at least t are sound and t are zero-knowledge), then it must be that C satisfies the standard notion of soundness. This holds because, by the above definition, the existence of an adversary $\mathcal{A}$ running in time τ that can break soundness of the combiner C (*i.e.*, $\exists \mathcal{A}$ s.t. $(C, \mathcal{A}) \in \mathcal{R}_{\text{Sound}}$) implies the existence of $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to one of the security properties of the candidates. However, if t of the candidates are secure (according to each security property), it is impossible to craft successful reductions for these schemes, which means that at most $n - t$ successful reductions can exist to each individual security property. Hence, no such $\mathcal{A}$ can exist (*i.e.*, $\forall \mathcal{A}, (C, \mathcal{A}) \notin \mathcal{R}_{\text{Sound}}$). As discussed in Section 2.2, this means that the combiner is secure according to the standard definition of soundness.

2.3.2 Zero-knowledge of (ZK, Sound)-combiners

Following a similar approach, we characterize zero-knowledge. For example, in the case of *B (ZK, sound)-combiners (*i.e.*, combiners considering *black-box* zero-knowledge), we say that C running on input n candidates $f_1, \dots, f_n$ is (black-box) zero-knowledge if the following happens: if for all simulators $\mathcal{S}$ there exists an adversary $\mathcal{A}$ running in time τ such that $(C, (\mathcal{S}, \mathcal{A}))$ belongs to $\mathcal{R}_{\text{ZK}}$ (*i.e.*, C is not black-box zero-knowledge), then it is possible to design at least $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to one of the security properties of distinct candidates. In other words, if the combiner is not zero-knowledge, then it must be that at least $n - t + 1$ input candidates are not sound or zero-knowledge. The formal definition follows.

Definition 6 (Non-Interactive Zero-Knowledge of t -out-of- n type $\in \{B, N\}^2$ (ZK, Sound)-combiners). *Let C be a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner. We say a t -out-of- n type $\in \{B, N\}^2$ (ZK, Sound)-combiner C is zero-knowledge if, after ordering the quantifiers according to type in Table 4, the following holds (combZKQuant is also defined in Table 4):*

combZKQuant s.t. $(C^{f_1, \dots, f_n}, (\mathcal{A}^{f_1, \dots, f_n}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}} \implies$ any of the following:

- $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to soundness of the candidates, as in Definition 5.
- $n - t + 1$ reductions running in time $\text{poly}(\tau)$ to black-box (resp. non-black-box) zero-knowledge of the candidates if type = *B (resp. type = *N), as in Definition 5.

type	Structural quantifiers	combZKQuant
BB	$\exists C \forall f_1, \dots, f_n \in \mathcal{F}$	$\forall \mathcal{S} \exists \mathcal{A}$
BN	$\exists C \forall f_1, \dots, f_n \in \mathcal{F}$	$\exists \mathcal{A} \forall \mathcal{S}$
NB	$\forall f_1, \dots, f_n \in \mathcal{F} \exists C$	$\forall \mathcal{S} \exists \mathcal{A}$
NN	$\forall f_1, \dots, f_n \in \mathcal{F} \exists C$	$\exists \mathcal{A} \forall \mathcal{S}$

Table 4: Classification of zero-knowledge for (ZK, Sound)-combiners.

Connections with the *standard ZK* notion. Let us focus on the black-box zero-knowledge case (a similar argument holds for the non-black-box case). The above definition implies that, if the combiner runs with sufficiently-many candidates that are secure, then it must be that C satisfies the standard notion of black-box zero-knowledge. This holds because, by the above definition, if the black-box zero-knowledge of the combiner C is broken (*i.e.*, $\forall \mathcal{S} \exists \mathcal{A}$ s.t. $(C, (\mathcal{A}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}}$), it means that there exist $n - t + 1$ reductions to one of the security properties of the candidates. However, if t of the candidates are secure (according to each security property), it is impossible to craft successful reductions for these schemes, which means that at most $n - t$ successful reductions can exist. Hence, it is impossible to break zero-knowledge for the combiner (*i.e.*, $\exists \mathcal{S}$ s.t. $\forall \mathcal{A}, (C, (\mathcal{A}, \mathcal{S})) \notin \mathcal{R}_{\text{ZK}}$). As discussed in Section 2.2, this means that the combiner is secure according to the standard definition of black-box zero-knowledge, and in particular that it admits a universal ZK simulator. Therefore, our combiner can be used in any application in which a black-box zero-knowledge protocol is required.

2.4 Combiners for knowledge soundness and zero-knowledge

We observe that it is possible to adapt the definition provided in the previous subsection to a different set of security properties in a straight-forward manner. Indeed, throughout this composition we also consider $\text{type} \in \{\text{B}, \text{N}\}^3$ (ZK, AoK)-combiners, *i.e.*, combiners that achieve ZK and knowledge soundness by relying on the same properties for their candidates. Note that, in this case, type has a third literal, highlighting whether the combiner considers black-box or non-black-box knowledge soundness. The definition follows the same strategy we discussed for (ZK, Sound)-combiners: first, we formalize what it means to break knowledge soundness, and then provide security definitions for each property. In particular, breaking knowledge soundness or ZK of the combiner implies breaking either knowledge soundness or ZK of $n - t + 1$ of the candidates. We defer the complete formal treatment to Appendix C.

3 Impossibility Results for $t \leq \lfloor \frac{n}{2} \rfloor$

In this section, we show that no t -out-of- n (ZK, Sound)-combiner exists for NP for $t \leq \lfloor \frac{n}{2} \rfloor$, according to our Definitions 5, 6, unless the polynomial hierarchy collapses.

Theorem 1. *Let $\{\Pi_1, \dots, \Pi_n\}$ be a set of non-interactive complete candidates (*i.e.*, $\forall i \in [n], \Pi_i \in \mathcal{F}_{\text{NI}}$). If there exists a t -out-of- n type (ZK, Sound)-combiner $C^{\Pi_1, \dots, \Pi_n}$ for the language $\mathcal{L}$, with $\text{type} \in \{\text{B}, \text{N}\}^2$ (according to Definitions 5, 6) (*i.e.*, the combiner returns a protocol that is both sound and zero-knowledge), then either $t > \lfloor \frac{n}{2} \rfloor$ or $\mathcal{L} \subseteq \text{coAM} \cap \text{AM}$.*

Proof. In what follows, we exhibit a proof for $(n, t) = (2, 1)$, which can be trivially extended to the general case of (n, t) with $t \leq \lfloor \frac{n}{2} \rfloor$.

Let C be a 1-out-of-2 (ZK, Sound)-combiner for some NP-language $\mathcal{L}$. This combiner works assuming that the input protocols are for two languages $\mathcal{L}_{\text{zk}}$ and $\mathcal{L}_{\text{Sound}}$. Let us consider the following two NIZK candidates: $\Pi_{\text{zk}} = (\text{setup}_{\text{zk}}, \mathcal{P}_{\text{zk}}, \mathcal{V}_{\text{zk}})$ and $\Pi_{\text{Sound}} = (\text{setup}_{\text{Sound}}, \mathcal{P}_{\text{Sound}},$

$\mathcal{V}_{\text{Sound}}$), respectively for the languages $\mathcal{L}_{\text{zk}}$ and $\mathcal{L}_{\text{Sound}}$. The setup algorithms of both Π_{zk} and Π_{Sound} return nothing. The prover $\mathcal{P}_{\text{zk}}$, on input a statement and a witness, returns 0. The verifier accepts a proof π_{zk} for a statement x only if $\pi_{\text{zk}} = 0$. The prover $\mathcal{P}_{\text{Sound}}$ on input a statement x and a witness w returns the witness w . The verifier accepts a proof π only if w is a valid witness for x . In summary, we have that Π_{zk} is unconditional zero-knowledge (but not sound), and Π_{Sound} is unconditional sound (but not zero-knowledge). By assumption, $\mathcal{C}$ is a valid combiner that on input Π_{zk} and Π_{Sound} returns a new scheme Π . By the definition of secure combiner, we have that if there exists an adversary, running in time τ , that attacks successfully the zero-knowledge (the soundness) of Π , then we can design $n - t + 1 = 2$ reductions to either the zero-knowledge or the soundness properties of both schemes, each (the reductions) running in time $\text{poly}(\tau)$. We are going to prove that it is impossible to construct these two reductions. Proving this implies that the scheme produced by $\mathcal{C}$ is unconditional sound and unconditional zero-knowledge, which implies that $\mathcal{L} \subseteq \text{coAM} \cap \text{AM}$ due to [Ps05]. Assume that an unbounded adversary breaks either the zero-knowledge or the soundness of $\mathcal{C}$. By the definition of t -out-of- n (ZK, Sound)-combiners, this must imply the existence of two (unbounded) reductions to zero-knowledge or two (unbounded) reductions to soundness of the input schemes. We first consider zero-knowledge. Having two reductions to zero-knowledge of the candidates implies having a reduction against zero-knowledge of Π_{Sound} , and a reduction against zero-knowledge of Π_{zk} . Let us focus on the reduction to zero-knowledge for Π_{zk} . A successful reduction Red_{zk} needs to tell apart if a proof of Π_{zk} is simulated or not. However, no such Red_{zk} can exist, as the zero-knowledge property of Π_{zk} is unconditional. Hence, it is impossible to have two (even unbounded) reductions to the zero-knowledge of the candidates. Let us now turn our attention to soundness. Similarly, in this case, we have a reduction $\text{Red}_{\text{Sound}}$ to the soundness of Π_{Sound} and a reduction to the soundness of Π_{zk} . We observe that the soundness of Π_{Sound} is unconditional, hence, no such reduction $\text{Red}_{\text{Sound}}$ can exist.

Given that we can never obtain $n - t + 1 = 2$ (even unbounded) reductions to either soundness or zero-knowledge of the candidates, it follows that the scheme obtained by running $\mathcal{C}$ on inputs Π_{zk} and Π_{Sound} is both unconditional sound and unconditional zero-knowledge. But due to [Ps05], such a scheme exists only for languages in $\text{coAM} \cap \text{AM}$. This concludes the proof. $\square$

Lifting the multi-property requirement. The previous impossibility result is sufficient to rule out robust combiners that satisfy our definitions. However, our proof crucially relies on the fact that one scheme is unconditionally sound (but not ZK), while the other is unconditionally ZK (but not sound). It is a natural question to investigate whether t -out-of- n (ZK, Sound)-combiners could exist if we restrict our attention to schemes that are simultaneously sound and zero-knowledge, with $t \leq \lfloor \frac{n}{2} \rfloor$.

In what follows, we show that this is indeed impossible, assuming the existence of any statistically sound and computationally zero-knowledge NIZK scheme, and the existence of any computationally sound and statistically zero-knowledge NIZK scheme. The strong evidence towards the existence of such protocols [BFM88, FLS90, FLS99, CHK03, GOS06, GOS12, SW14, LPWW20] helps us rule out the existence of these stronger combiners.

Theorem 2. *Let $T = \{\Pi_1, \dots, \Pi_n\}$ with $\Pi_i \in \mathcal{F}_{\text{NI}}$ for each $i \in [n]$, and let $K \subseteq T$ with $|K| = t$ be such that each $\Pi \in K$ is both sound and zero-knowledge. Assume the existence of statistically sound and computationally zero-knowledge NIZK schemes, and computationally sound and statistically zero-knowledge NIZK schemes. If there exists a t -out-of- n type (ZK, Sound)-combiner $\mathcal{C}^{\Pi_1, \dots, \Pi_n}$ for the language $\mathcal{L}$ (according to Definition 5, Definition 6), with $\text{type} \in \{\text{B}, \text{N}\}^2$ (i.e., the combiner returns a protocol that is both sound and zero-knowledge), then either $t > \lfloor \frac{n}{2} \rfloor$ or $\mathcal{L} \subseteq \text{coAM} \cap \text{AM}$.*

Proof. Let C be a 1-out-of-2 (ZK, Sound)-combiner for some NP-language $\mathcal{L}$. This combiner works assuming that the input protocols are for two languages $\mathcal{L}_{\text{zk}}$ and $\mathcal{L}_{\text{Sound}}$. Let us consider a computationally sound and statistically zero-knowledge scheme $\Pi_{\text{zk}} = (\text{setup}_{\text{zk}}, \mathcal{P}_{\text{zk}}, \mathcal{V}_{\text{zk}})$, and a statistically sound and computationally zero-knowledge scheme $\Pi_{\text{Sound}} = (\text{setup}_{\text{Sound}}, \mathcal{P}_{\text{Sound}}, \mathcal{V}_{\text{Sound}})$, respectively for the languages $\mathcal{L}_{\text{zk}}$ and $\mathcal{L}_{\text{Sound}}$. These schemes exist by assumption.

Similarly to Theorem 1, we argue that, by running C on input Π_{zk} and Π_{Sound} , we obtain a new scheme Π for which we cannot design $n - t + 1 = 2$ (even unbounded) reductions to either the zero-knowledge or the soundness of the candidates. By our definition of (ZK, Sound)-combiners, this implies that there exists no (even unbounded) adversary who successfully attacks the combiner. Hence, the scheme produced by C is both statistically sound and statistically zero-knowledge, implying that $\mathcal{L} \subseteq \text{coAM} \cap \text{AM}$ due to [Ps05].

Assume that an unbounded adversary breaks either the zero-knowledge or the soundness of C . Having two reductions to zero-knowledge of the candidates implies having a reduction against zero-knowledge of Π_{Sound} , and a reduction Red_{zk} against zero-knowledge of Π_{zk} . However, no such Red_{zk} can exist, as the zero-knowledge property of Π_{zk} is statistical (by assumption). Hence, it is impossible to have two (even unbounded) reductions to the zero-knowledge of the candidates. Similarly, having two reductions to soundness of the candidates implies having a reduction $\text{Red}_{\text{Sound}}$ to the soundness of Π_{Sound} and a reduction to the soundness of Π_{zk} . We observe that the soundness of Π_{Sound} is statistical (by assumption); hence, no such reduction $\text{Red}_{\text{Sound}}$ can exist. $\square$

Computationally-secure input candidates. The aforementioned impossibility result rules out a restricted class of robust t -out-of- n combiners, with $t \leq \lfloor n/2 \rfloor$, where the input candidates are simultaneously sound and zero-knowledge. However, our result crucially relies on a counterexample depending on candidates with statistical properties. Hence, robust t -out-of- n NIZK combiners could still exist if we restrict the input candidates to be *exclusively* computationally sound and computationally zero-knowledge, with $t \leq \lfloor n/2 \rfloor$. With the next impossibility result, we further rule out this restricted class of combiners under standard cryptographic assumptions, albeit only for black-box combiners.

Theorem 3. *Let $T = \{\Pi_1, \dots, \Pi_n\}$ with $\Pi_i \in \mathcal{F}_{\text{NI}}$ for each $i \in [n]$, and let $K \subseteq T$ with $|K| = t$ be such that each $\Pi \in K$ is both computationally sound and computationally zero-knowledge. Assume the existence of dual-mode commitments, one-way functions, statistically sound and computationally zero-knowledge NIZK schemes, and computationally sound and statistically zero-knowledge NIZK schemes. If there exists a t -out-of- n type (ZK, Sound)-combiner $C^{\Pi_1, \dots, \Pi_n}$ for the language $\mathcal{L}$ (according to Definition 5, Definition 6), with $\text{type} = \text{B}^*$ (i.e., the combiner returns a protocol that is both sound and zero-knowledge while having black-box access to its candidates), then either $t > \lfloor \frac{n}{2} \rfloor$ or $\mathcal{L} \subseteq \text{coAM} \cap \text{AM}$.*

Proof. We first list the ingredients that will be required for the proof:

- $\Pi_{\text{czkp}} = (\text{setup}_{\text{czkp}}, \mathcal{P}_{\text{czkp}}, \mathcal{V}_{\text{czkp}})$ is a computational zero-knowledge proof system.
- $\Pi_{\text{szka}} = (\text{setup}_{\text{szka}}, \mathcal{P}_{\text{szka}}, \mathcal{V}_{\text{szka}})$ is a statistical zero-knowledge argument system.
- $\text{dmcom}^{\text{mode}} = (\text{setup}_{\text{dmcom}}^{\text{mode}}, \text{commit}_{\text{dmcom}}^{\text{mode}}, \text{open}_{\text{dmcom}}^{\text{mode}})$ is a dual-mode commitment scheme. If $\text{mode} = \text{hiding}$, the commitment scheme is statistically hiding and computationally binding. If $\text{mode} = \text{binding}$, the commitment scheme is computationally hiding and statistically binding. The two modes are computationally indistinguishable. This primitive admits instantiations from standard cryptographic assumptions, including DDH [Lin15].
- PRG is a pseudo-random generator.

Next, we construct two schemes $\Pi_{\text{Sound}} = (\text{setup}_{\text{Sound}}, \mathcal{P}_{\text{Sound}}, \mathcal{V}_{\text{Sound}})$ and $\Pi_{\text{zk}} = (\text{setup}_{\text{zk}}, \mathcal{P}_{\text{zk}}, \mathcal{V}_{\text{zk}})$, working respectively for languages $\mathcal{L}_{\text{Sound}}$ and $\mathcal{L}_{\text{zk}}$, by using the ingredients above.

- $\text{setup}_{\text{Sound}}$ outputs the concatenation of $\text{setup}_{\text{Sound}}$ and a uniformly random string. $\mathcal{P}_{\text{Sound}}, \mathcal{V}_{\text{Sound}}$ are defined as $\mathcal{P}_{\text{czkp}}, \mathcal{V}_{\text{czkp}}$ with respect to the following language: $\mathcal{L}_{\text{czkp}} = \{(\text{CRS}, x) \mid \exists w \text{ s.t. } (x, w) \in \mathcal{R}_{\mathcal{L}_{\text{Sound}}} \vee \exists r \text{ s.t. } \text{CRS} = \text{PRG}(r)\}$.
- setup_{zk} outputs the concatenation of $\text{setup}_{\text{szka}}$ and $\text{setup}_{\text{dmcom}}^{\text{hiding}}$. $\mathcal{P}_{\text{zk}}$ computes a commitment of the witness com with opening o and a proof π by running $\mathcal{P}_{\text{szka}}$ for the following language: $\mathcal{L}_{\text{szka}} = \{(com, x) \mid \exists (w, o) \text{ s.t. } (x, w) \in \mathcal{R}_{\mathcal{L}_{\text{zk}}} \wedge w = \text{open}_{\text{dmcom}}^{\text{mode}}(com, o)\}$. It then outputs (com, π) . $\mathcal{V}_{\text{zk}}$ runs $\mathcal{V}_{\text{szka}}$.

It is easy to see that Π_{Sound} is still a CZKP. Indeed, Π_{czkp} is statistically sound, and the CRS is generated as a uniformly random string. Moreover, Π_{zk} is still a SZKA: the output of the prover is (com, π) , with com being a statistically hiding commitment and π being a statistically zero-knowledge argument.

Finally, we construct two CZKA schemes Π_1, Π_2 by “downgrading” the statistical properties of $\Pi_{\text{Sound}}, \Pi_{\text{zk}}$ to computational. We do that as follows:

- Π_1 is the same as Π_{Sound} , except that the setup algorithm evaluates the common random string by sampling a random r and evaluating $\text{PRG}(r)$.
- Π_2 is the same as Π_{zk} , except that the dual-mode commitment scheme is initialized in binding mode.

We are now ready for the proof. By contradiction, let us assume that $\mathcal{C}$ is a 1-out-of-2 (ZK, Sound)-combiner for some NP-language $\mathcal{L}$. This combiner works assuming that the input protocols are for two languages $\mathcal{L}_{\text{zk}}$ and $\mathcal{L}_{\text{Sound}}$.

We first argue that the combiner, upon input Π_1 and Π_2 , outputs a CZKA. Since Π_1 and Π_2 are both secure CZKA, we can never have $n - t + 1 = 2$ *efficient* reductions to either the zero-knowledge or the soundness of the candidates. Since $\mathcal{C}$ is a robust combiner, there exists no efficient adversary who successfully attacks the zero-knowledge or the soundness of the combiner. This, in turn, means that the scheme output by the combiner is a CZKA for the NP-language $\mathcal{L}$.

Next, we exhibit a hybrid chain by which we replace Π_1 and Π_2 with Π_{Sound} and Π_{zk} , respectively. Intuitively, we argue that the combiner still outputs a CZKA for $\mathcal{L}$ even if we switch computational properties of the candidates with statistical properties. Moreover, the CZKA output by the combiner upon input Π_1 and Π_2 is such that the joint distribution of its setup, prover, and verifier algorithms are computationally indistinguishable from the CZKA output by the combiner upon input Π_{Sound} and Π_{zk} . Essentially, this ensures that the combiner cannot detect the introduction of candidates with statistical properties.

- $\mathcal{H}_0$: The combiner runs with input candidates Π_1 and Π_2 .
- $\mathcal{H}_1$: We replace Π_1 with Π_{Sound} . We observe that the combiner still outputs a CZKA, given that it is impossible to find $n - t + 1 = 2$ reductions to computational zero-knowledge or soundness of the candidates (as Π_{Sound} is a secure CZKP and Π_2 is a secure CZKA). Moreover, the distribution of the algorithms of the CZKA output by the combiner is computationally indistinguishable from $\mathcal{H}_0$. In particular, if we have a distinguisher telling apart $\mathcal{H}_0$ from $\mathcal{H}_1$, we can design a reduction to the security of the PRG. Since the PRG is secure by assumption, there can never exist such a distinguisher.
- $\mathcal{H}_2$: We replace Π_2 with Π_{zk} . With a similar argument as in the previous hybrid, the combiner still outputs a CZKA. Moreover, the distribution of the algorithms of the CZKA output by the combiner is computationally indistinguishable from $\mathcal{H}_1$, as otherwise it

would be possible to design a reduction to the indistinguishability of modes of the dual-mode commitment scheme.

In $\mathcal{H}_2$, we have that C is a 1-out-of-2 combiner that outputs a CZKA for an NP-language $\mathcal{L}$ even when running with Π_{Sound} and Π_{ZK} as inputs (which are, respectively, a CZKA and a SZKP). But by Theorem 2, such a combiner cannot exist for any NP-language (unless the polynomial hierarchy collapses). Hence, we have a contradiction. $\square$

4 Combiner for Homomorphic Languages

In this section, we define a t -out-of- n BB (ZK, Sound)-combiner, with $t > \lfloor \frac{n}{2} \rfloor$, that may be used to prove the class of languages described in [Mau09] which are denoted as *homomorphic languages* [Cra96]. In particular, a homomorphic language $\mathcal{L}_{\text{HL}}$ is defined with respect to the groups $(G, \star)$, $(H, \otimes)$ and a homomorphism $f : G \rightarrow H$ (i.e., $f(x) \otimes f(y) = f(x \star y)$), and is represented by the set $\{x \mid \exists w \text{ s.t. } f(w) = x\}$. We conclude the section by showing that our combiner is also a t -out-of- n BBB (ZK, AoK)-combiner.

4.1 Preliminaries

Notationally, within this scheme we use hyperoperations as in [Goo47]. Let w be an element in group G . We denote $w\{2\star\}v$ as the result of $\underbrace{w \star w \star \dots \star w}_{v \text{ copies of } w}$, with $v \in \mathbb{Z}_q$. We compactly

represent $a_0 \star a_1 \star \dots \star a_n$ with the symbol $\bigstar_{i=0}^n a_i$, and $a_0 \otimes a_1 \otimes \dots \otimes a_n$ with the symbol $\bigotimes_{i=0}^n a_i$.

Finally, if f is a group homomorphism as defined above, the following facts hold:

$$f(w)\{2\otimes\}v = f(w\{2\star\}v) \quad (1)$$

$$\bigotimes_{j \in [n]} f(w_j) = f\left(\bigstar_{j \in [n]} w_j\right) \quad (2)$$

Fact 1 holds, as:

$$f(w)\{2\otimes\}v = \underbrace{f(w) \otimes f(w) \otimes \dots \otimes f(w)}_{v \text{ copies of } f(w)} = f\left(\underbrace{w \star w \star \dots \star w}_{v \text{ copies of } w}\right) = f(w\{2\star\}v)$$

Fact 2 holds, as:

$$\bigotimes_{j \in [n]} f(w_j) = f(w_1) \otimes \dots \otimes f(w_n) = f(w_1 \star \dots \star w_n) = f\left(\bigstar_{j \in [n]} w_j\right)$$

Shamir's secret sharing scheme and homomorphic interpolation. We propose a version of Shamir's secret sharing scheme that works in $(G, \star)$, which allows performing the standard sharing and reconstruction operations. Moreover, we design a special reconstruction algorithm that we denote with $\overline{\text{reconstruct}}_H$, which allows performing the reconstruction *homomorphically*, exploiting the properties of the homomorphic language. More details follow.

The sharing algorithm share_G takes as input w , where w is an element of group G , and samples $t - 1$ random group elements $a_1, \dots, a_{t-1}$ by using the generation algorithm $\text{Gen}_G(1^\lambda)$ (this algorithm just samples uniformly at random elements from G). A polynomial p of degree $t - 1$ is defined by using coefficient a_i , and by setting $a_0 = w$. The shares are computed by evaluating $p(i)$, for $i \in [n]$:

$\text{share}_{\mathbf{G}}(w)$: $a_1, \dots, a_{t-1} \xleftarrow{\$} \text{Gen}_{\mathbf{G}}(1^\lambda)$. Output $\{w_i \leftarrow w \star \bigstar_{j=1}^{t-1} (a_j \{2\star\} i^j)\}_{i \in [n]}$

The $\text{reconstruct}_{\mathbf{G}}$ algorithm takes as input a subset $S = \{s_i\}_{i \in I}$ of t shares, where $I = i_1, \dots, i_t \in \mathbb{Z}_q^t$ is a set of evaluation points of the polynomial, and uses Lagrange interpolation to recompute $p(0)$ (*i.e.*, the secret):

$\text{reconstruct}_{\mathbf{G}}(\{s_i\}_{i \in I})$: Output $w \leftarrow \bigstar_{j \in I} (s_i \{2\star\} (\prod_{k \in I, k \neq j} \frac{k}{k-j}))$

Since Shamir's secret sharing scheme relies on Lagrange interpolation, it is possible to reconstruct points in the polynomial that are different from the secret (*i.e.*, points evaluated in $l \neq 0$). To this extent, we denote the modified version of $\text{reconstruct}_{\mathbf{G}}$ as $\overline{\text{reconstruct}}_{\mathbf{G}}$. $\overline{\text{reconstruct}}_{\mathbf{G}}$ takes as input a subset $S = \{s_i\}_{i \in I}$ of t shares and an index $l \notin I$ of the point to be reconstructed, and works as follows:

$\overline{\text{reconstruct}}_{\mathbf{G}}(\{s_i\}_{i \in I}, l)$: Output $w_l \leftarrow \bigstar_{j \in I} (s_i \{2\star\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k}))$.

We remark that $\overline{\text{reconstruct}}_{\mathbf{G}}(\cdot, 0) = \text{reconstruct}_{\mathbf{G}}(\cdot)$.

Finally, if f is a group homomorphism as defined above, having access to a subset S of size t of functions of the shares (*i.e.*, $\{f(s_i)\}_{i \in I}$) allows the reconstruction of a function of the point in the polynomial (*i.e.*, $x_l = f(s_l)$ for $l \notin I$) as follows:

$$x_l \leftarrow \bigotimes_{j \in I} (f(s_i) \{2\otimes\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k}))$$

We denote the reconstruction operation above as $\overline{\text{reconstruct}}_{\mathbf{H}}(\{x_i\}_{i \in I}, l)$. The correctness of this operation follows by applying Fact 1 and Fact 2:

$$\begin{aligned} x_l \leftarrow \bigotimes_{j \in I} (f(s_i) \{2\otimes\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) &= f(\bigstar_{j \in I} (s_i \{2\star\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k}))) \\ &= f(\overline{\text{reconstruct}}_{\mathbf{G}}(\{s_i\}_{i \in I}, l)) = f(w_l) \end{aligned}$$

In the rest of the paper, whenever we talk about homomorphic languages $\mathcal{L}_{\mathbf{HL}}$ with respect to the groups $(\mathbf{G}, \star)$, $(\mathbf{H}, \otimes)$ and a homomorphism $f : \mathbf{G} \rightarrow \mathbf{H}$, we implicitly require that the algorithms $(\text{share}_{\mathbf{G}}, \text{reconstruct}_{\mathbf{G}})$, as defined here, satisfy correctness and perfect security of Shamir's secret sharing scheme (respectively, Definition 14 and Definition 15). Looking ahead, the correctness property will be used within the proof of Theorem 4, and the perfect security property will be used within the proof of Theorem 6. In Appendix E we give a few concrete examples of such homomorphic languages and associated sharing and reconstruction operations.

4.2 Combiner description

Let $\{\Pi_k = (\text{setup}_k, \mathcal{P}_k, \mathcal{V}_k)\}_{k \in [n]}$ be non-interactive protocols for the homomorphic language $\mathcal{L}_{\mathbf{HL}}$ defined above. These protocols represent the input primitives of the combiner. Let x be the theorem the verifier wants to verify. At a high level, in our scheme the prover secret shares the witness w in $\mathbf{G}$ by running $\text{share}_{\mathbf{G}}$, and computes n sub-statements by evaluating f on each obtained share. Then the prover proves that each of these sub-statements belongs to the homomorphic language, using the input candidates (the k -th input candidate Π_k is used to prove the k -th sub-statement). The verifier picks the first t sub-statements and homomorphically reconstructs the polynomial in $\mathbf{H}$ by running $\overline{\text{reconstruct}}_{\mathbf{H}}$ (as defined in Section 4.1). The verifier then accepts if all the proofs are accepting, and the homomorphic reconstruction matches with the theorem x and with all of the sub-statements. We formally describe our combiner $\mathbf{C}_{\mathbf{HL}} = (\mathbf{C}_{\text{setup}}, \mathbf{C}_{\text{prove}}, \mathbf{C}_{\text{verify}})$ in Figure 4.1:

Figure 4.1: Combiner C_{HL}

- $C_{\text{setup}}(1^l, 1^\lambda)$: Compute $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$, for $k \in [n]$. Output $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- $C_{\text{prove}}(pp, x, w)$:
 - Compute $(w_1, \dots, w_n) \xleftarrow{\$} \text{share}_G(w)$.
 - Compute $x_k \leftarrow f(w_k)$, for $k \in [n]$.
 - Compute $\pi_k \leftarrow \mathcal{P}_k(pp_k, x_k, w_k)$, for $k \in [n]$.
 - Output $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in [n]})$.
- $C_{\text{verify}}(pp, x, \pi)$:
 - Parse (pp, π) as $(\{pp_k\}_{k \in [n]}, \{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in [n]})$. Set $x_0 \leftarrow x$.
 - If $\mathcal{V}_k(pp_k, x_k, \pi_k)$ outputs 0 for some $k \in [n]$, output 0. Otherwise, for each $k \in [n] \setminus [t] \cup \{0\}$, behave as follows:
 - Compute $x'_k \leftarrow \overline{\text{reconstruct}}_H(\{x_j\}_{j \in [t]}, k)$.
 - Check whether $x'_k = x_k$. If the check fails then return 0.
 - Return 1.

4.3 Proof sketch for the (ZK, Sound) case

Next, we provide a high-level overview of how our proofs for soundness and zero-knowledge work, deferring formal theorems and complete proofs to Appendix F.1, including the analysis of correctness.

Soundness. To show that C_{HL} is sound (as per Definition 5), we argue that the existence of an adversary that produces an accepting proof for a statement $x \notin \mathcal{L}_{HL}$ implies having sufficiently-many reductions (*i.e.*, $n - t + 1$) to soundness of the underlying candidates. Suppose by contradiction that a prover can make a verifier accept a proof for a false statement (*i.e.*, x is s.t. $f(w) \neq x$ for all w) and that only $n - t$ reductions to the soundness are possible. This means that there are at least t sub-statements that are valid (*i.e.*, they belong to $\mathcal{L}_{HL}$). Moreover, if the verifier of our combiner has accepted the proof, this means that the witnesses for all the sub-statements represent points that belong to the same polynomial of degree $t - 1$. In particular, this polynomial is univocally determined by the witnesses of the t valid sub-statements (that exist by contradiction). Given that this polynomial evaluated on 0 is equal to x (this holds because the verifier has accepted the proof) then the correctness of our secret sharing scheme and the homomorphic property of the language ensures that x belongs to the language.

Zero-knowledge. Proving the zero-knowledge of this (and of the other schemes we propose) requires some particular care. The traditional way to prove the zero-knowledge of our protocol would be to design a zero-knowledge simulator that executes the simulators of t of the candidates. To prove that the output of this simulator is indistinguishable from the output of an adversarial verifier when interacting with an honest prover, we design a chain of indistinguishable hybrid experiments, the first corresponding to the experiment where the proof is generated via an honest prover, and the last corresponding to the simulated experiment. If by contradiction our simulator is not good (*i.e.*, zero-knowledge does not hold) this would imply that a pair of adjacent hybrids is not indistinguishable, hence, a reduction to a cryptographic primitive can be performed.

In the proof we just sketched, we have that if by contradiction zero-knowledge does not hold, then we have *one* reduction. However, our security definition states that if zero-knowledge does not hold, then it must be possible to create $n - t + 1$ reductions. Clearly, the proof approach

we have discussed does not achieve what we want. Indeed, the above approach does not really make sense as our simulator does not know for which of the candidates the simulator should be run. We recall that our definition of combiner (Definition 6) states that if the combiner is not zero-knowledge then it should be possible to perform $n - t + 1$ reductions to soundness or $n - t + 1$ reductions to zero-knowledge of the candidates. In particular, if the combiner is not zero-knowledge, then every zero-knowledge simulator we design can be attacked. The intuition is that to prove the security of our scheme, we need to design multiple simulators, such that any time an adversary wins against one simulator, we can create a new reduction to different a input candidate. We construct these multiple simulators as follows. Each simulator $\mathcal{S}^T$ is parameterized by a set T containing t distinct indices in $[n]$, and runs the simulators for the candidates whose indices are in T . We design a sequence of hybrid arguments in which we gradually replace all the sub-proofs generated by all the candidates in T , using real shares of the witness, with simulated sub-proofs. After these sub-proofs are simulated, we replace the $n - t$ remaining shares of the witness with random group elements. Because there cannot exist a good simulator, the output of $\mathcal{S}^T$ must be distinguishable, for all $T \subseteq [n]$ with $|T| = t$. But this means that there exists an adversary $\mathcal{A}$ that is able to distinguish two adjacent hybrids of the hybrid chain (say, $\mathcal{H}_{i^*-1}$ and $\mathcal{H}_{i^*}$), and we can therefore state that there exists one reduction to zero-knowledge of candidate Π_{i^*} . Intuitively, whenever some $\mathcal{A}$ breaks zero-knowledge of candidate Π_{i^*} in $\mathcal{S}^T$ by distinguishing the output of a specific protocol Π_{i^*} , it also breaks all the other $\mathcal{S}^{T'}$ for $T \neq T'$ that contain candidate Π_{i^*} within their chain of hybrids. However, there still exist simulators that do not include Π_{i^*} as part of their hybrid chain. Therefore, (a possibly different) $\mathcal{A}$ has to break zero-knowledge of another candidate. By a combinatorics argument, the only way *all* $\binom{n}{t}$ simulators can be broken is by breaking zero-knowledge for at least $n - t + 1$ candidates.

4.4 Proof sketch for the (ZK, AoK) case

We conclude the section by arguing that C_{HL} is also a (ZK, AoK)-combiner. Notice that, in proving zero-knowledge of the (ZK, Sound) case, we relied exclusively on zero-knowledge of the candidates. Hence, we can reuse exactly the same proof here. Therefore, we only need to prove that our combiner is indeed AoK. We provide a sketch of this proof in what follows, deferring the formal details to Appendix F.2.

Argument-of-Knowledge sketch. The proof for the AoK property is similar in spirit to the zero-knowledge one: we argue that, if AoK of the combiner is broken, we can design $n - t + 1$ reductions to AoK of the candidates. To do that, we construct an extractor $\mathcal{E}^T$ which is parameterized by a set T containing t distinct indices in $[n]$, and runs the extractors for the candidates in T . Then, we show that either $\mathcal{E}^T$ successfully extracts a witness, or there exists at least one reduction to AoK of one of the candidates whose indices are in T .

The first portion of the proof consists of an hybrid argument in which we gradually replace all the public parameters generated by setup algorithms of the candidates with ones generated by trapdoor setup algorithms. As for zero-knowledge, if some adversary detects any variation within the hybrid chain, we have a reduction to argument-of-knowledge of a specific candidate. After that, the extractor attempts to run all the extractors for the primitives indexed by T to recompute a witness from an accepting proof (x, π) supplied by the adversary. For that, we argue that the only way the extractor may output the witness is by successfully running the extractor for all the candidates whose indices are in T . Hence, it suffices for one extractor of the candidates to fail to cause $\mathcal{E}^T$ to fail. For this scheme this is indeed the case, as each extractor extracts a share of the witness w_i , and the only way w can be extracted is by running

the reconstruction procedure of Shamir's secret sharing scheme with these t shares.

The proof concludes with the same combinatorics argument used for the proof of zero-knowledge, with the only difference being that reductions may exist either because some adversary distinguishes the setup generation algorithm of one of the candidates, or because some adversary produces an accepting proof (x, π) for which a candidate fails to extract a share of the witness.

5 Recursive proof combiner

In this section we define a t -out-of- n NBB (ZK, AoK)-combiner, with $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in O(1)$, that may be used for all of NP and generates proofs by “nesting” proofs of the candidates. This procedure fundamentally entails two operations:

- Computing the “innermost” proof. The innermost proof is generated by using a proof system to prove knowledge of a witness w for a statement x with respect to language $\mathcal{L}$.
- Computing an “intermediate” proof. An intermediate proof consists of using a proof system to prove knowledge of an accepting proof π generated by another (possibly different) proof system. More formally, the NP-language for a scheme Π_j that proves knowledge of an accepting proof generated with scheme Π_{j-1} is defined as $\mathcal{L}_j = \{(\mathcal{V}_{j-1}, pp_{j-1}, x_{j-1}) \mid \exists \pi_{j-1} \text{ s.t. } \mathcal{V}_{j-1}(pp_{j-1}, x_{j-1}, \pi_{j-1}) = 1\}$.

The second operation can be applied recursively, and the last intermediate proof will be output in the clear. Intuitively, if such a proof is accepting, it proves that the entire chain of proofs was generated correctly. Moreover, the presence of intermediate proofs is what makes the combiner non-black-box in its use of its candidates, as the construction is quantified after the specific instantiations of the candidates $\Pi_1, \dots, \Pi_n$ are quantified.

Combiner description. The combiner takes as input n non-interactive protocols $\Pi_k = (\text{setup}_k, \mathcal{P}_k, \mathcal{V}_k) \in \mathcal{F}_{\text{NI}}$ as per Definition 1, for $k \in [n]$, which works for any NP language $\mathcal{L}$. From a high-level perspective, the prover generates $\binom{n}{t}$ proofs, each with a different subset of t out of n candidates. For each proof, the candidate with the lowest index generates the innermost proof by using statement x and witness w . After that, intermediate proofs are generated recursively by using all the other candidates in the specific subset. We formally describe our combiner in Figure 5.1.

Figure 5.1: Combiner $\mathcal{C}_{\text{PoP}}$

- $\mathcal{C}_{\text{setup}}(1^l, 1^\lambda)$: Compute $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$, for $k \in [n]$. Output $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- $\mathcal{C}_{\text{prove}}(pp, x, w)$:
 - Index all possible subsets containing t of the n candidates with set $S = \{S^i\}_{i \in \binom{n}{t}}$.
 - For $S^i \in S$, denote $\{s_{i,1}, \dots, s_{i,t}\}$ as the indices of the candidates for this specific subset sorted in lexicographic order. Then, compute proof π_i^{PoP} as follows:
 - * Compute the innermost proof $\pi_{s_{i,1}} \leftarrow \mathcal{P}_{s_{i,1}}(pp_{s_{i,1}}, x, w)$. The language associated to this proof is $\mathcal{L}_{s_{i,1}} = \{x \mid \exists w \text{ s.t. } (x, w) \in \mathcal{R}_{\mathcal{L}}\}$.
 - * For $j \in [2, \dots, t]$, let $x_{s_{i,j}} \leftarrow (\mathcal{V}_{s_{i,j-1}}, pp_{s_{i,j-1}}, x_{s_{i,j-1}})$. Compute intermediate proof $\pi_{s_{i,j}} \leftarrow \mathcal{P}_{s_{i,j}}(pp_{s_{i,j}}, x_{s_{i,j}}, \pi_{s_{i,j-1}})$. The language associated to the j -th proof is $\mathcal{L}_j = \{x_{s_{i,j}} \mid \exists \pi_{s_{i,j-1}} \text{ s.t. } \mathcal{V}_{s_{i,j-1}}(pp_{s_{i,j-1}}, x_{s_{i,j-1}}, \pi_{s_{i,j-1}}) = 1\}$.
 - * Set $\pi_i^{\text{PoP}} \leftarrow \pi_{s_{i,t}}$.
 - Set $\pi \leftarrow \{\pi_i^{\text{PoP}}\}_{i \in \binom{n}{t}}$. Output π .

- $C_{\text{vrfy}}(pp, x, \pi)$:
 - Parse π as $\{\pi_i^{\text{PoP}}\}_{i \in \binom{n}{t}}$.
 - Index all possible subsets containing t of the n candidates with set $S = \{S^i\}_{i \in \binom{n}{t}}$.
 - For $S^i \in S$, denote $\{s_{i,1}, \dots, s_{i,t}\}$ as the indices of the candidates for this specific subset sorted in lexicographic order. Then, run $\mathcal{V}_{s_{i,t}}(pp_{s_{i,t}}, x_{s_{i,t}}, \pi_i)$.
 - If any of the verifier runs above outputs 0, output 0. Otherwise, output 1.

The formal treatment of this protocol follows the same ideas presented in the sketch of the previous section, and is deferred to Appendix G.

6 MPC-in-the-head approach

In this section we exhibit a t -out-of- n NB (ZK, Sound)-combiner, with $t > \lfloor \frac{2n}{3} \rfloor$, that relies on MPC-in-the-head techniques and may be used for all of NP. Similarly to Section 4, this combiner can also be instantiated as a t -out-of- n NBB (ZK, AoK)-combiner.

6.1 Preliminaries

We start by summarizing Secure Multiparty Computation (MPC), following the definitions of [IKOS07]. An n -party protocol $\Pi = (P_1, \dots, P_n)$ is an n -tuple of probabilistic polynomial-time (PPT) interactive Turing machines (ITMs). Each party P_i is initialized with a local input $w_i \in \{0, 1\}^*$ and random coins $r_i \in \{0, 1\}^*$. All the parties share a public input $x \in \{0, 1\}^*$. In this paper, we mainly focus on R -round MPC protocols that securely realize an n -party functionality f , where f maps the input $(x, w_1, \dots, w_n)$ to an n -tuple of outputs (when only a single output is specified, this output is assumed to be given to all parties). Moreover, instead of letting every party share the input x , we hardcode x in the function f . All the MPC protocols we use have this structure; hence, this will be implicit.

Notation for MPC. Rather than viewing a protocol Π as an n -tuple of interactive Turing machines, it is convenient to view each Turing machine as a sequence of multiple algorithms: **first-msg_{*i*}**, to compute P_i 's first messages to its peers; **nxt-msg_{*i*}^{*k*}**, to compute P_i 's k -th round messages for $2 \leq k \leq R$; and **output_{*i*}**, to compute P_i 's output. Thus, a protocol Π can be defined as $\{(\text{first-msg}_i, \text{nxt-msg}_i^k, \text{output}_i)\}_{i \in [n], k \in [R] \setminus \{1\}}$. The syntax of the algorithms is as follows:

- **first-msg_{*i*}**($w_i; r_i$) $\rightarrow (\text{msg}_{i \rightarrow 1}^1, \dots, \text{msg}_{i \rightarrow n}^1)$ produces the first-round messages of party P_i to all parties. If the message is sent over a broadcast channel, it holds that $\text{msg}_{i \rightarrow 1}^1 = \text{msg}_{i \rightarrow 2}^1 = \dots = \text{msg}_{i \rightarrow n}^1$. Note that a party's message to itself can be considered to be its state.
- **nxt-msg_{*i*}^{*k*}**($w_i, \{\text{msg}_{j \rightarrow i}^l\}_{j \in [n], l \in [k-1]}; r_i$) $\rightarrow (\text{msg}_{i \rightarrow 1}^k, \dots, \text{msg}_{i \rightarrow n}^k)$ produces the k -th round messages of party P_i to all parties.
- **output_{*i*}**($w_i, \{\text{msg}_{j \rightarrow i}^k\}_{j \in [n], k \in [R]}; r_i$) $\rightarrow y_i$ produces the output returned to party P_i .

We note that, unless needed, to not overburden the notation, we do not pass the random coin r as an explicit input of the cryptographic algorithms. In addition, we define the view of party P_i and the consistency of views as follows:

Definition 7 (View). The view view_i of party P_i is defined as the input w_i , the randomness r_i , and the all the messages P_i sends (denoted as $\text{msg}_{i, \text{out}}$, with $\text{msg}_{i, \text{out}} \leftarrow \{\text{msg}_{i \rightarrow j}^k\}_{j \in [n] \setminus \{i\}, k \in [R]}$)

and receives (denoted as $\text{msg}_{i,\text{in}}$, with $\text{msg}_{i,\text{in}} \leftarrow \{\text{msg}_{j \rightarrow i}^k\}_{j \in [n] \setminus \{i\}, k \in [R]}$) during the execution of the MPC protocol.

Definition 8 (View Consistency). *A pair of views view_i and view_j are consistent (with respect to the protocol Π realizing f_x) if $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]}$ is identical to $\{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]}$ and vice versa.*

Finally, we report properties of MPC protocols from [IKOS07] that are needed within our construction.

Definition 9 (Correctness). *Let $f_x(w_1, \dots, w_n) : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ with a hardcoded value $x \in \{0, 1\}^*$ be an n -party function. A protocol Π realizes the function f_x with perfect correctness if for all inputs $w_1, \dots, w_n$, the probability that the output of some player is different from the output of f_x is 0, where the probability is over the independent choices of the randomness $r_1, \dots, r_n$.*

Definition 10 ($\mathcal{K}$ -Privacy). *Let $f_x(w_1, \dots, w_n) : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ with a hardcoded value $x \in \{0, 1\}^*$ be an n -party function. We denote the joint view $\text{view}_{\mathcal{I}}(w_1, \dots, w_n)$ as the views of parties P_i ($i \in \mathcal{I}$), corresponding to the execution where every P_i 's input is w_i . We denote $f_{x,\mathcal{I}}(w_1, \dots, w_n)$ the output of function $f_x(w_1, \dots, w_n)$ received by the parties whose indices are in the set $\mathcal{I}$. A protocol Π realizes the function f_x with perfect $\mathcal{K}$ -Privacy if there is a PPT simulator $\mathcal{S}$ s.t. for all inputs $(w_1, \dots, w_n)$, and every set of corrupted players $\mathcal{I} \subseteq [n]$ with at most size $\mathcal{K}$, the joint view $\text{view}_{\mathcal{I}}(w_1, \dots, w_n)$ of parties in $\mathcal{I}$ is identically distributed to $\mathcal{S}(\mathcal{I}, x, \{w_k\}_{k \in [\mathcal{I}]}, f_{x,\mathcal{I}}(w_1, \dots, w_n))$. We note that when we say “corrupted”, we mean semi-honest, i.e., parties will follow the protocol specification.*

Definition 11 ($\mathcal{K}$ -Robustness). *Let $f_x(w_1, \dots, w_n) : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ with a hardcoded value $x \in \{0, 1\}^*$ be an n -party function. A protocol Π realizes the function f_x with perfect $\mathcal{K}$ -Robustness if it is perfectly correct (Definition 9) in the presence of a semi-honest adversary, and moreover for any computational unbounded malicious adversary corrupting a set $\mathcal{I}$ with at most size $\mathcal{K}$, and for any inputs $(w_1, \dots, w_n)$, if there is no $(w'_1, \dots, w'_n)$ s.t. $f_x(w'_1, \dots, w'_n) = 1$, then the probability that some honest parties output 1 in an execution of Π realizing f_x is 0.*

6.2 Building blocks

This combiner requires the following primitives:

- n non-interactive protocols $\Pi_k = (\text{setup}_k, \mathcal{P}_k, \mathcal{V}_k) \in \mathcal{F}_{\text{NI}}$ as per Definition 1, for $k \in [n]$, which work for any NP language $\mathcal{L}$.
- A t -out-of- n secret sharing scheme $\text{SS} = (\text{share}, \text{reconstruct})$, as per Definition 13.
- A statistically binding and computationally hiding non-interactive string commitment $\text{NIC} = (\text{setup}, \text{commit}, \text{open})$, as per Definition 12.
- An MPC protocol $\Pi_{\text{MPC}} = \{(\text{first-msg}_i, \text{nxt-msg}_i^k, \text{output}_i)\}_{i \in [n], k \in [R] \setminus \{1\}}$ that realizes the function f_x (described below) with perfect $\mathcal{K}$ -robustness (as per Definition 11) in the malicious model and perfect $\mathcal{K}$ -privacy (as per Definition 10) in the semi-honest model, where $\mathcal{K} = (n - t)$. f_x has hard-coded a statement x , takes as inputs $w_1, \dots, w_n$, and outputs 1 if $(x, \text{reconstruct}(\{w_j\}_{j \in [t]})) \in \mathcal{R}_{\mathcal{L}}$; otherwise, it outputs 0.

6.3 From NIZK candidates to non-interactive commitments

Among the primitives for this construction, we require a commitment scheme that is computationally hiding and statistically binding. This is incompatible with our formalization of

combiners, given that an adversary breaking the security of the combiner can now yield a reduction to the newly-introduced (computational) assumption, rather than $n - t + 1$ reductions to the security of the underlying candidates. In what follows, we show that it is possible to instantiate statistically binding and computationally hiding non-interactive string commitments by exclusively relying on the security of the candidates. Formally, we prove the following lemma:

Lemma 1. *Let $\{\Pi_k = (\text{setup}_k, \mathcal{P}_k, \mathcal{V}_k)\}_{k \in [n]} \in \mathcal{F}_{\text{NI}}$ be non-interactive protocols, as per Definition 1, for any NP language $\mathcal{L}$. Let t of the candidates be sound and t of the candidates be zero-knowledge, with $t > \lfloor \frac{n}{2} \rfloor$. Assuming that $\text{NP} \subseteq \text{ZKA}$ and that $\text{ZKA} \not\subseteq \text{ioP/poly}$, there exists an efficient construction of statistically binding and computationally hiding non-interactive string commitments in the CRS model.*

Proof. The high-level idea of the proof consists in expanding the following chain of implications: t -out-of- n NIZK candidates $\implies$ 1-out-of- n OWFs $\implies$ OWF $\implies$ PRG $\implies$ non-interactive commitments.

- **t -out-of- n NIZK candidates $\implies$ 1-out-of- n OWF candidates.** Let $\mathcal{S}_{\text{snd}}$ be the set of sound candidates and $\mathcal{S}_{\text{zk}}$ be the set of ZK candidates. Since $t > \lfloor \frac{n}{2} \rfloor$, $\mathcal{S}_{\text{snd}} \cap \mathcal{S}_{\text{zk}} \neq \emptyset$. Hence, at least one candidate is both sound and zero-knowledge. Given that we assume $\text{NP} \subseteq \text{ZKA}$ and that $\text{ZKA} \not\subseteq \text{ioP/poly}$, we can invoke [LMP24, Theorem 1.1] on each of the candidates, obtaining a set of n candidate functions $\{f_i\}_{i \in [n]}$. Since at least one of the candidates is both sound and zero-knowledge, at least one of the functions will be a OWF.
- **1-out-of- n OWF candidates $\implies$ OWF.** [Her05, Lemma 4.1] shows a combiner that splits the input among the function candidates (e.g., by XOR), and then simply concatenates the output of all the candidates. More explicitly, as long as there exists an index i for which f_i is a OWF, $f(x) = f_1(x_1) || \dots || f_n(x_n)$ is also a OWF, with $x = x_1 \oplus x_2 \oplus \dots \oplus x_n$.
- **OWF $\implies$ PRG.** By [HILL99], there exists a PRG that may be instantiated by a OWF.
- **PRG $\implies$ NI-Com.** Following [MP12], Naor’s bit commitment scheme [Nao91] can be made non-interactive by moving the randomness of the receiver to the CRS that is statistically binding and computationally hiding. Once we have a non-interactive bit commitment scheme, we also have a non-interactive string commitment scheme.

□

Further remarks. Proving this lemma requires that $\text{NP} \subseteq \text{ZKA}$ and that $\text{ZKA} \not\subseteq \text{ioP/poly}$. As hinted throughout the technical overview, if the first assumption does not hold, it means that there exists a language in NP that does not admit zero-knowledge arguments. Hence, the problem of designing NIZK combiners becomes less interesting. From here, if the second assumption is falsified, it would imply that $\text{NP} \subseteq \text{ioP/poly}$.

6.4 Combiner description

We are now ready to define our combiner. The construction is very similar to [GJS19]. Intuitively, the prover secret shares the witness w , runs the MPC protocol Π_{MPC} to generate views of all parties, uses the non-interactive commitment scheme to commit to each of these views (which, again, include the input, randomness, and sent/received messages of the party they relate to). Then, for each party, it uses a NIZK candidate to prove that the commitments related to that party are computed correctly and the committed values are from an execution of Π_{MPC} for which the party’s output is 1. The verifier checks whether all the sub-proofs are

accepting, and checks whether the commitment corresponding to outgoing messages are consistent with the commitment corresponding to inbound messages. If all the checks succeed, the verifier outputs 1. Formally, we define our combiner $C_{\text{MPC}} = (C_{\text{setup}}, C_{\text{prove}}, C_{\text{vrfy}})$ in Figure 6.1.

Figure 6.1: Combiner C_{MPC}

- $C_{\text{setup}}(1^l, 1^\lambda)$: Compute $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$, for $k \in [n]$. Compute $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$. Output $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.
- $C_{\text{prove}}(pp, x, w)$:
 - Compute $(w_1, \dots, w_n) \xleftarrow{\$} \text{share}(w)$.
 - Run MPC protocol Π_{MPC} , where every party P_i ($i \in [n]$) takes w_i as input, and uses uniformly randomly sampled value r_i as the randomness. Obtain $\text{view}_1, \dots, \text{view}_n$ from the execution of Π_{MPC} .
 - For $i \in [n]$, $j \in [n]$, compute $(\text{com}_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(\text{com}_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
 - For $i \in [n]$, set $x'_i \leftarrow (\{\text{com}_{i,j}\}_{j \in [n]}, \{\text{com}_{j,i}\}_{j \in [n]})$ and $w'_i \leftarrow (\text{view}_i, \{o_{i,j}\}_{j \in [n]}, \{o_{j,i}\}_{j \in [n]})$.
 - For $k \in [n]$, compute $\pi_k = \mathcal{P}_k(pp_k, (pp_{\text{NIC}}, x'_k), w'_k)$. The language associated to the sub-proofs π_i is $\mathcal{L}_{i,\text{MPC}} = \left\{ (pp_{\text{NIC}}, \{\text{com}_{i,j}\}_{j \in [n]}, \{\text{com}_{j,i}\}_{j \in [n]}) \mid \forall k \in [R] \setminus \{1\}, \exists (\text{view}_i, \{o_{i,j}\}_{j \in [n]}, \{o_{j,i}\}_{j \in [n]}), \{\text{msg}_{i \rightarrow j}^1\}_{j \in [n]} = \text{frst-msg}_i(w_i; r_i) \wedge \{\text{msg}_{i \rightarrow j}^k\}_{j \in [n]} = \text{nxt-msg}_i^k(w_i, \{\text{msg}_{j \rightarrow i}^l\}_{j \in [n], l \in [k-1]}; r_i) \wedge 1 = \text{output}_i(w_i, \text{msg}_{i,\text{in}}; r_i) \wedge \forall j \in [n], \text{open}(\text{com}_{i,j}, o_{i,j}) = \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} \wedge \text{open}(\text{com}_{j,i}, o_{j,i}) = \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]} \right\}$.
 - Output $\pi = (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
- $C_{\text{vrfy}}(pp, x, \pi)$: Output 1 if all following conditions hold:
 - Compute $\mathcal{V}_k(pp_k, (pp_{\text{NIC}}, x'_k), \pi_k)$ for $k \in [n]$, and all $\mathcal{V}_k$ outputs 1.
 - Parse $\{x'_k\}_{k \in [n]}$ as $\{(\{\text{com}_{k,j}\}_{j \in [n]}, \{\text{com}_{j,k}\}_{j \in [n]})\}_{k \in [n]}$. For $k \in [n], j \in [n]$, there exists $j \neq k$ s.t. $\text{com}_{k,j} = \text{com}_{j,k}$.

We remark that, even though all the proofs for the remainder of this section work for $t > \lfloor \frac{n}{2} \rfloor$, we rely on the instantiation of Π_{MPC} as in BGW [BGW88]. In [GMO16], the authors claim that such a protocol is $\lfloor \frac{n-1}{3} \rfloor$ -private, perfect $\lfloor \frac{n-1}{3} \rfloor$ -robust protocol, and use it to instantiate the protocol of [IKOS07]. The formal treatment of this protocol is deferred to Appendix H.

Acknowledgements

Daniele Venturi is member of the Gruppo Nazionale Calcolo Scientifico Istituto Nazionale di Alta Matematica (GNCS-INdAM). His research was supported by project SERICS (PE00000014) and by project PARTHENON (B53D23013000006), under the MUR National Recovery and Resilience Plan funded by the European Union—NextGenerationEU, and by project BEAT, funded by Sapienza University of Rome. Lorenzo Magliocco was also supported by project PARTHENON and by project BEAT. Michele Ciampi was supported by the Sunday Group, Inc. and by the Input Output Research Hub (IORH) of the University of Edinburgh.

References

- [ABJ⁺19] Prabhanjan Ananth, Saikrishna Badrinarayanan, Aayush Jain, Nathan Manohar, and Amit Sahai. From FE combiners to secure MPC and back. In Dennis Hofheinz

- and Alon Rosen, editors, *TCC 2019: 17th Theory of Cryptography Conference, Part I*, volume 11891 of *Lecture Notes in Computer Science*, pages 199–228, Nuremberg, Germany, December 1–5, 2019. Springer, Cham, Switzerland.
- [AJN⁺16] Prabhanjan Ananth, Aayush Jain, Moni Naor, Amit Sahai, and Eylon Yogev. Universal constructions and robust combiners for indistinguishability obfuscation and witness encryption. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 491–520, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
 - [AJS17] Prabhanjan Ananth, Aayush Jain, and Amit Sahai. Robust transforming combiners from indistinguishability obfuscation to functional encryption. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part I*, volume 10210 of *Lecture Notes in Computer Science*, pages 91–121, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
 - [AK25] Benny Applebaum and Eliran Kachlon. How to share an NP statement or combiners for zero-knowledge proofs. Cryptology ePrint Archive, Paper 2025/334, 2025.
 - [BB06] Dan Boneh and Xavier Boyen. On the impossibility of efficiently combining collision resistant hash functions. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 570–583, Santa Barbara, CA, USA, August 20–24, 2006. Springer Berlin Heidelberg, Germany.
 - [BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *20th Annual ACM Symposium on Theory of Computing*, pages 103–112, Chicago, IL, USA, May 2–4, 1988. ACM Press.
 - [BG24] Nir Bitansky and Nathan Geier. Amplification of non-interactive zero knowledge, revisited. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part IX*, volume 14928 of *Lecture Notes in Computer Science*, pages 361–390, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
 - [BGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th Annual ACM Symposium on Theory of Computing*, pages 1–10, Chicago, IL, USA, May 2–4, 1988. ACM Press.
 - [BKP⁺24] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron D. Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th Annual ACM Symposium on Theory of Computing*, pages 435–443, Vancouver, BC, Canada, June 24–28, 2024. ACM Press.
 - [CDFR17] Ignacio Cascudo, Ivan Damgård, Oriol Farràs, and Samuel Ranellucci. Resource-efficient OT combiners with active security. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part II*, volume 10678 of *Lecture Notes in Computer Science*, pages 461–486, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.

- [CHK03] Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In Eli Biham, editor, *Advances in Cryptology – EUROCRYPT 2003*, volume 2656 of *Lecture Notes in Computer Science*, pages 255–271, Warsaw, Poland, May 4–8, 2003. Springer Berlin Heidelberg, Germany.
- [CPS⁺16a] Michele Ciampi, Giuseppe Persiano, Alessandra Scafuro, Luisa Siniscalchi, and Ivan Visconti. Improved OR-composition of sigma-protocols. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part II*, volume 9563 of *Lecture Notes in Computer Science*, pages 112–141, Tel Aviv, Israel, January 10–13, 2016. Springer Berlin Heidelberg, Germany.
- [CPS⁺16b] Michele Ciampi, Giuseppe Persiano, Alessandra Scafuro, Luisa Siniscalchi, and Ivan Visconti. Online/offline OR composition of sigma protocols. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 63–92, Vienna, Austria, May 8–12, 2016. Springer Berlin Heidelberg, Germany.
- [Cra96] Ronald Cramer. Modular design of secure yet practical cryptographic protocols. *Ph. D.-thesis, CWI and U. of Amsterdam*, 2, 1996.
- [CRS⁺07] Ran Canetti, Ronald L. Rivest, Madhu Sudan, Luca Trevisan, Salil P. Vadhan, and Hoeteck Wee. Amplifying collision resistance: A complexity-theoretic treatment. In Alfred Menezes, editor, *Advances in Cryptology – CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 264–283, Santa Barbara, CA, USA, August 19–23, 2007. Springer Berlin Heidelberg, Germany.
- [DFG⁺23] Yevgeniy Dodis, Niels Ferguson, Eli Goldin, Peter Hall, and Krzysztof Pietrzak. Random oracle combiners: Breaking the concatenation barrier for collision-resistance. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 514–546, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
- [DK05] Yevgeniy Dodis and Jonathan Katz. Chosen-ciphertext security of multiple encryption. In Joe Kilian, editor, *TCC 2005: 2nd Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 188–209, Cambridge, MA, USA, February 10–12, 2005. Springer Berlin Heidelberg, Germany.
- [FHNS16] Marc Fischlin, Amir Herzberg, Hod Bin Noon, and Haya Shulman. Obfuscation combiners. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part II*, volume 9815 of *Lecture Notes in Computer Science*, pages 521–550, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- [FL08] Marc Fischlin and Anja Lehmann. Multi-property preserving combiners for hash functions. In Ran Canetti, editor, *TCC 2008: 5th Theory of Cryptography Conference*, volume 4948 of *Lecture Notes in Computer Science*, pages 375–392, San Francisco, CA, USA, March 19–21, 2008. Springer Berlin Heidelberg, Germany.
- [FLP08] Marc Fischlin, Anja Lehmann, and Krzysztof Pietrzak. Robust multi-property combiners for hash functions revisited. In Luca Aceto, Ivan Damgård, Leslie Ann Goldberg, Magnús M. Halldórsson, Anna Ingólfssdóttir, and Igor Walukiewicz, editors, *ICALP 2008: 35th International Colloquium on Automata, Languages and*

- Programming, Part II*, volume 5126 of *Lecture Notes in Computer Science*, pages 655–666, Reykjavik, Iceland, July 7–11, 2008. Springer Berlin Heidelberg, Germany.
- [FLP14] Marc Fischlin, Anja Lehmann, and Krzysztof Pietrzak. Robust multi-property combiners for hash functions. *Journal of Cryptology*, 27(3):397–428, July 2014.
 - [FLS90] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple non-interactive zero knowledge proofs based on a single random string (extended abstract). In *31st Annual Symposium on Foundations of Computer Science*, pages 308–317, St. Louis, MO, USA, October 22–24, 1990. IEEE Computer Society Press.
 - [FLS99] Uriel Feige, Dror Lapidot, and Adi Shamir. Multiple noninteractive zero knowledge proofs under general assumptions. *SIAM Journal on computing*, 29(1):1–28, 1999.
 - [FR24] Oriol Farràs and Jordi Ribes-González. One-out-of-q OT combiners. *IEEE Trans. Inf. Theory*, 70(4):2984–2998, 2024.
 - [GHP18] Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018: 21st International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 10769 of *Lecture Notes in Computer Science*, pages 190–218, Rio de Janeiro, Brazil, March 25–29, 2018. Springer, Cham, Switzerland.
 - [GJS19] Vipul Goyal, Aayush Jain, and Amit Sahai. Simultaneous amplification: The case of non-interactive zero-knowledge. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 608–637, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
 - [GMO16] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for Boolean circuits. In Thorsten Holz and Stefan Savage, editors, *USENIX Security 2016: 25th USENIX Security Symposium*, pages 1069–1083, Austin, TX, USA, August 10–12, 2016. USENIX Association.
 - [GO07] Jens Groth and Rafail Ostrovsky. Cryptography in the multi-string model. In Alfred Menezes, editor, *Advances in Cryptology – CRYPTO 2007*, volume 4622 of *Lecture Notes in Computer Science*, pages 323–341, Santa Barbara, CA, USA, August 19–23, 2007. Springer Berlin Heidelberg, Germany.
 - [Goo47] Reuben Louis Goodstein. Transfinite ordinals in recursive number theory. *The Journal of Symbolic Logic*, 12(4):123–129, 1947.
 - [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *Advances in Cryptology – EURO-CRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 339–358, St. Petersburg, Russia, May 28 – June 1, 2006. Springer Berlin Heidelberg, Germany.
 - [GOS12] Jens Groth, Rafail Ostrovsky, and Amit Sahai. New techniques for noninteractive zero-knowledge. *J. ACM*, 59(3), jun 2012.
 - [Her05] Amir Herzberg. On tolerant cryptographic constructions. In Alfred Menezes, editor, *Topics in Cryptology – CT-RSA 2005*, volume 3376 of *Lecture Notes in Computer*

- Science*, pages 172–190, San Francisco, CA, USA, February 14–18, 2005. Springer Berlin Heidelberg, Germany.
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.
 - [HKN⁺05] Danny Harnik, Joe Kilian, Moni Naor, Omer Reingold, and Alon Rosen. On robust combiners for oblivious transfer and other primitives. In Ronald Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 96–113, Aarhus, Denmark, May 22–26, 2005. Springer Berlin Heidelberg, Germany.
 - [HKNY23] Taiga Hiroka, Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. Robust combiners and universal constructions for quantum cryptography. Cryptology ePrint Archive, Report 2023/1772, 2023.
 - [HN24] Shuichi Hirahara and Mikito Nanashima. One-way functions and zero knowledge. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *56th Annual ACM Symposium on Theory of Computing*, pages 1731–1738, Vancouver, BC, Canada, June 24–28, 2024. ACM Press.
 - [IKOS07] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In David S. Johnson and Uriel Feige, editors, *39th Annual ACM Symposium on Theory of Computing*, pages 21–30, San Diego, CA, USA, June 11–13, 2007. ACM Press.
 - [JMS20] Aayush Jain, Nathan Manohar, and Amit Sahai. Combiners for functional encryption, unconditionally. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 141–168, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.
 - [Lin15] Yehuda Lindell. An efficient transform from sigma protocols to NIZK with a CRS and non-programmable random oracle. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *TCC 2015: 12th Theory of Cryptography Conference, Part I*, volume 9014 of *Lecture Notes in Computer Science*, pages 93–109, Warsaw, Poland, March 23–25, 2015. Springer Berlin Heidelberg, Germany.
 - [LMP24] Yanyi Liu, Noam Mazon, and Rafael Pass. A note on zero-knowledge for NP and one-way functions. *Electron. Colloquium Comput. Complex.*, TR24-095, 2024.
 - [LPWW20] Benoît Libert, Alain Passelègue, Hoeteck Wee, and David J. Wu. New constructions of statistical NIZKs: Dual-mode DV-NIZKs and more. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 410–441, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland.
 - [Mau09] Ueli M. Maurer. Unifying zero-knowledge proofs of knowledge. In Bart Preneel, editor, *AFRICACRYPT 09: 2nd International Conference on Cryptology in Africa*, volume 5580 of *Lecture Notes in Computer Science*, pages 272–286, Gammarth, Tunisia, June 21–25, 2009. Springer Berlin Heidelberg, Germany.

- [Mit12] Arno Mittelbach. Hash combiners for second pre-image resistance, target collision resistance and pre-image resistance have long output. In Ivan Visconti and Roberto De Prisco, editors, *SCN 12: 8th International Conference on Security in Communication Networks*, volume 7485 of *Lecture Notes in Computer Science*, pages 522–539, Amalfi, Italy, September 5–7, 2012. Springer Berlin Heidelberg, Germany.
- [Mit13] Arno Mittelbach. Cryptophia’s short combiner for collision-resistant hash functions. In Michael J. Jacobson, Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 2013: 11th International Conference on Applied Cryptography and Network Security*, volume 7954 of *Lecture Notes in Computer Science*, pages 136–153, Banff, AB, Canada, June 25–28, 2013. Springer Berlin Heidelberg, Germany.
- [MP06] Remo Meier and Bartosz Przydatek. On robust combiners for private information retrieval and other primitives. In Cynthia Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 555–569, Santa Barbara, CA, USA, August 20–24, 2006. Springer Berlin Heidelberg, Germany.
- [MP12] Mohammad Mahmoody and Rafael Pass. The curious case of non-interactive commitments - on the power of black-box vs. non-black-box use of primitives. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 701–718, Santa Barbara, CA, USA, August 19–23, 2012. Springer Berlin Heidelberg, Germany.
- [MP14] Bart Mennink and Bart Preneel. Breaking and fixing cryptophia’s short combiner. In Dimitris Gritzalis, Aggelos Kiayias, and Ioannis G. Askoxylakis, editors, *CANS 14: 13th International Conference on Cryptology and Network Security*, volume 8813 of *Lecture Notes in Computer Science*, pages 50–63, Heraklion, Crete, Greece, October 22–24, 2014. Springer, Cham, Switzerland.
- [MPW07] Remo Meier, Bartosz Przydatek, and Jürg Wullschleger. Robuster combiners for oblivious transfer. In Salil P. Vadhan, editor, *TCC 2007: 4th Theory of Cryptography Conference*, volume 4392 of *Lecture Notes in Computer Science*, pages 404–418, Amsterdam, The Netherlands, February 21–24, 2007. Springer Berlin Heidelberg, Germany.
- [Nao91] Moni Naor. Bit commitment using pseudorandomness. *Journal of Cryptology*, 4(2):151–158, January 1991.
- [Pie07] Krzysztof Pietrzak. Non-trivial black-box combiners for collision-resistant hash-functions don’t exist. In Moni Naor, editor, *Advances in Cryptology – EURO-CRYPT 2007*, volume 4515 of *Lecture Notes in Computer Science*, pages 23–33, Barcelona, Spain, May 20–24, 2007. Springer Berlin Heidelberg, Germany.
- [Pie08] Krzysztof Pietrzak. Compression from collisions, or why CRHF combiners have a long output. In David Wagner, editor, *Advances in Cryptology – CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 413–432, Santa Barbara, CA, USA, August 17–21, 2008. Springer Berlin Heidelberg, Germany.
- [PR20] Bertram Poettering and Paul Rösler. Combiners for AEAD. *IACR Transactions on Symmetric Cryptology*, 2020(1):121–143, 2020.

- [Ps05] Rafael Pass and abhi shelat. Unconditional characterizations of non-interactive zero-knowledge. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 118–134, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany.
- [Som06] Christian Sommer. *Robust Combiners for Cryptographic Primitives*. Master thesis, ETH Zurich, 2006.
- [SW14] Amit Sahai and Brent Waters. How to use indistinguishability obfuscation: deniable encryption, and more. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 475–484, New York, NY, USA, May 31 – June 3, 2014. ACM Press.

A A comparison with [HKNY23]

In [HKNY23], the authors exhibit a 1-out-of-2 robust combiner for quantum commitments - an inherently multi-property primitive that admits computational and statistical variants. By adapting our characterization of robustness to commitments, one would expect to be able to reuse the same proof strategies adopted for our impossibility results to rule out such combiners, hence ruling out the existence of their positive result. The reason of this mismatch lies in a fundamentally different characterization of robustness in [HKNY23], which is weaker compared to ours as it grants the combiner the power to *create* security. We provide more details in what follows, shedding some further light on why this is the case.

The definition of [HKNY23] requires the combined commitment to *always* be statistically binding and computationally hiding, regardless of whether the input schemes have statistical or computational security. Our definition (in the case of commitments) instead would not put any restriction on whether the binding or the hiding is statistical/computational. This indeed will depend on the input candidates. In particular, our notion allows the design of a combiner that, for example, on input statistically hiding commitments, yields a statistically hiding commitment.

The reason why the combiner of [HKNY23] does not satisfy our security notion is that the combiner of [HKNY23] always returns a statistical binding commitment, even if the input candidates are computationally binding and computationally hiding. This means that an unbounded adversary breaking the statistical binding of the combined scheme can never be turned into an adversary meaningfully breaking the security of any of the input schemes (given that the security of the candidates is only computational). This, in turn, means that the combiner can create security (statistical soundness); something that should not be captured by the spirit of robustness. Indeed, as per our definitions, the security of a combiner *must* derive from the security of the input primitives.

B Deferred definitions

In what follows, we summarize relevant definitions that are used throughout the composition.

Definition 12 (Non-interactive string commitment). *A non-interactive string-commitment scheme $\text{com} = (\mathsf{S}, \mathsf{R})$ for message space $\mathcal{M}$ is composed of an efficient sender S and an efficient receiver R for which the following conditions hold:*

- **Completeness.**

$$\Pr \left[\text{open}(pp, \text{com}, o) = m \mid pp \xleftarrow{\$} \text{setup}(1^\lambda); (\text{com}, o) \leftarrow \text{commit}(pp, m) \right] \geq 1 - \text{negl}(\lambda).$$

- **Computational Hiding.** *Let $pp \xleftarrow{\$} \text{setup}(1^\lambda)$. Then, for all $m_0 \neq m_1$, with $m_0, m_1 \in \mathcal{M}$:*

$$\text{commit}(pp, m_0) \approx_c \text{commit}(pp, m_1).$$

- **Computational Binding.** *For all PPT algorithms S' and all $m_0 \neq m_1$, with $m_0, m_1 \in \mathcal{M}$:*

$$\Pr \left[\text{open}(pp, \text{com}, o_0) = m_0 \wedge \text{open}(pp, \text{com}, o_1) = m_1 \mid \begin{array}{l} pp \xleftarrow{\$} \text{setup}(1^\lambda) \\ (\text{com}, o_0, o_1) \leftarrow S'(pp) \end{array} \right] \leq \text{negl}(\lambda).$$

If the properties above hold against unbounded adversaries, we refer to statistical hiding and statistical binding, respectively.

Secret sharing. We provide a generic definition of a t -out-of- n secret sharing scheme (that we use in Section 6).

Definition 13 (t -out-of- n secret sharing scheme). *A t -out-of- n secret sharing scheme is a pair of algorithms (**share**, **reconstruct**) that behaves as follows:*

- **share** is a randomized algorithm that, on input a secret m , outputs a set of n shares $(s_1, \dots, s_n)$.
- **reconstruct** is a deterministic algorithm that, on input t shares $(s_1, \dots, s_t)$, outputs the secret m .

The scheme satisfies the correctness property if, $\forall m, \forall S = \{i_1, \dots, i_t\} \subseteq \{1, \dots, n\}$ of size t , it holds that:

$$\Pr_{\text{share}(m) \rightarrow (s_1, \dots, s_n)} [\text{reconstruct}(s_{i_1}, \dots, s_{i_t}) = m] = 1.$$

In addition, the scheme satisfies perfect security if, $\forall m, m', \forall S \subseteq \{1, \dots, n\}$, where $|S| < t$, the following holds:

$$\{(s_i : i \in S) \mid (s_1, \dots, s_n) \leftarrow \text{share}(m)\} \approx_p \{(s'_i : i \in S) \mid (s'_1, \dots, s'_n) \leftarrow \text{share}(m')\}.$$

Additionally, we specify properties we require from the secret sharing scheme in Section 4 (which are similar to some of the basic properties of the Shamir's secret sharing).

Definition 14 (Correctness). *We say the scheme satisfy the correctness property if, $\forall m, \forall S = \{i_1, \dots, i_t\} \subseteq \{1, \dots, n\}$ of size t , it holds that:*

$$\Pr_{\text{share}_G(m) \rightarrow (s_1, \dots, s_n)} [\text{reconstruct}_G(s_{i_1}, \dots, s_{i_t}) = m] = 1.$$

Additionally, we require that for any set $S = \{i_1, \dots, i_t\} \subseteq [n]$, the following holds: $\forall k \in [n] \setminus S$,

$$\Pr_{\text{share}_G(m) \rightarrow (s_1, \dots, s_n)} [\text{reconstruct}_G((s_{i_1}, \dots, s_{i_t}), k) = s_k] = 1.$$

Definition 15 (Perfect security). *We say that $(\text{share}_G, \text{reconstruct}_G)$ satisfies perfect security if, $\forall m, m', \forall S \subseteq \{1, \dots, n\}$, where $|S| < t$, the following holds:*

$$\{(s_i : i \in S) \mid (s_1, \dots, s_n) \leftarrow \text{share}_G(m)\} \approx_p \{(s'_i : i \in S) \mid (s'_1, \dots, s'_n) \leftarrow \text{share}_G(m')\}.$$

Standard security properties of non-interactive proof systems. Next, we report standard counterparts of the security definitions we consider for our combiners (i.e., soundness, zero-knowledge, and argument-of-knowledge).

Definition 16 (Soundness). *Let $\Pi \in \mathcal{F}_{\text{NI}}$. We say Π is sound if, for all PPT algorithms $\mathcal{A}$, the following holds:*

$$\Pr \left[\mathcal{V}(pp, x, \pi) = 1 \wedge x \notin \mathcal{L} \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda); (x, \pi) \xleftarrow{\$} \mathcal{A}(pp) \right] < \text{negl}(\lambda).$$

Definition 17 (Black-box Non-Interactive Zero Knowledge). *Let $\Pi \in \mathcal{F}_{\text{NI}}$. Let $\text{REAL} = \{(pp, x, w, \pi) \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{P}(pp, x, w)\}_{\lambda \in \mathbb{N}}$ and $\text{IDEAL}_S = \{(pp, x, w, \pi) \mid (pp, \tau) \xleftarrow{\$} \mathcal{S}^0(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{S}^1(pp, \tau, x)\}_{\lambda \in \mathbb{N}}$. We say that Π satisfies black-box zero-knowledge if there exists a PPT algorithm $\mathcal{S} = (\mathcal{S}^0, \mathcal{S}^1)$ such that, for all PPT algorithms $\mathcal{A}$, the following holds:*

$$\Pr \left[b = b' \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\} \\ b' \leftarrow \mathcal{A}(pp, x, w, \pi) \end{array} ; (pp, x, w, \pi) \xleftarrow{\$} \begin{cases} \text{REAL} & \text{if } b = 0 \\ \text{IDEAL}_S & \text{if } b = 1 \end{cases} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Definition 18 (Black-Box Argument-of-Knowledge). *Let $\Pi \in \mathcal{F}_{\text{NI}}$. We say that Π satisfies black-box argument-of-knowledge if there exists a PPT algorithm $\mathcal{E} = (\mathcal{E}^0, \mathcal{E}^1)$ such that, for all PPT algorithms $\mathcal{A}$, the following holds:*

$$\Pr \left[\mathcal{A}(pp) = 1 \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda) \right] \approx_c \Pr \left[\mathcal{A}(pp) = 1 \mid (pp, \xi) \xleftarrow{\$} \mathcal{E}^0(1^{|x|}, 1^\lambda) \right],$$

and

$$\Pr \left[(x, w) \in \mathcal{R}_{\mathcal{L}}, \mathcal{V}(pp, x, \pi) = 1 \mid \begin{array}{l} (pp, \xi) \xleftarrow{\$} \mathcal{E}^0(1^n, 1^\lambda); (x, \pi) \leftarrow \mathcal{A}(pp) \\ w \leftarrow \mathcal{E}^1(pp, \xi, x, \pi) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

C Combiners for argument-of-knowledge and zero-knowledge

In this section, we expand the informal definition of (ZK, AoK)-robust combiners sketched in Section 2.4.

C.1 Breaking argument-of-knowledge

Following Section 2.2, we define $\mathcal{R}_{\text{AoK}}$ in a similar fashion to $\mathcal{R}_{\text{ZK}}$. In particular, a tuple $(\text{protocol}, (\text{extractor}, \text{adversary}))$ belongs to $\mathcal{R}_{\text{AoK}}$ if the adversary is able to either distinguish the introduction of a trapdoor setup by the extractor or craft an accepting proof for which the extractor fails to extract any witness. More formally:

Definition 19 ($\mathcal{R}_{\text{AoK}}$). *Let $\Pi \in \mathcal{F}_{\text{NI}}$, let $\mathcal{A}$ be an algorithm, and let $\mathcal{E} = (\mathcal{E}_0, \mathcal{E}_1)$ be a PPT algorithm. We say that $(\Pi, (\mathcal{A}, \mathcal{E})) \in \mathcal{R}_{\text{AoK}}$ if the following holds:*

$$\Pr \left[\mathcal{A}(pp) = 1 \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda) \right] \not\approx_c \Pr \left[\mathcal{A}(pp) = 1 \mid (pp, \xi) \xleftarrow{\$} \mathcal{E}_0(1^{|x|}, 1^\lambda) \right],$$

or

$$\Pr \left[(x, w) \in \mathcal{R}_{\mathcal{L}}, \mathcal{V}(pp, x, \pi) = 1 \mid \begin{array}{l} (pp, \xi) \xleftarrow{\$} \mathcal{E}_0(1^n, 1^\lambda); (x, \pi) \leftarrow \mathcal{A}(pp) \\ w \leftarrow \mathcal{E}_1(pp, \xi, x, \pi) \end{array} \right] < 1 - \text{negl}(\lambda).$$

$\mathcal{R}_{\text{AoK}}$ by itself is insufficient to capture AoK, as it only refers to a specific adversary breaking a specific extractor. We proceed similarly to ZK, and discuss black-box AoK in what follows (non-black-box AoK is reported in Appendix D).

Breaking black-box argument-of-knowledge. The standard definition of black-box argument-of-knowledge (reported in Definition 18) states that a protocol Π is secure if there exists a (universal) extractor $\mathcal{E}$ that is secure against every efficient adversary $\mathcal{A}$ (i.e., $\exists \mathcal{E} \text{ s.t. } \forall PPT \mathcal{A}, (\Pi, (\mathcal{A}, \mathcal{E})) \notin \mathcal{R}_{\text{AoK}}$). Breaking black-box argument-of-knowledge therefore means that there cannot exist any such extractor (which, in turn, means that *every* possible extractor can be attacked). This is formalized as follows:

$$\forall \mathcal{E} \exists PPT \mathcal{A} \text{ s.t. } (\Pi, (\mathcal{A}, \mathcal{E})) \in \mathcal{R}_{\text{AoK}}.$$

Similarly to Section 2.2, the same rationale can be applied to proof-of-knowledge by allowing unbounded adversaries.

C.2 Combiners for argument-of-knowledge

In what follows, we describe (ZK, AoK)-combiners. All our definitions have a label $\alpha\beta\gamma \in \{\mathbf{B}, \mathbf{N}\}^3$ that specifies whether the combiner has black-box access ($\alpha = \mathbf{B}$) or non-black-box access ($\alpha = \mathbf{N}$) to its candidates, whether the combiner relies on black-box zero-knowledge ($\beta = \mathbf{B}$) or non-black-box zero-knowledge ($\beta = \mathbf{N}$) of the candidates, and whether the combiner relies on black-box argument-of-knowledge ($\gamma = \mathbf{B}$) or non-black-box argument-of-knowledge ($\gamma = \mathbf{N}$). For instance, a NBN combiner has non-black-box access to its candidates, and considers candidates satisfying black-box zero-knowledge (*i.e.*, the secure candidates are such that there exists a universal simulator) and non-black-box argument of knowledge (*i.e.*, the secure candidates are such that for every adversary there exists an extractor). For a cleaner presentation, we again present separate definitions for argument-of-knowledge and zero-knowledge.

C.2.1 Argument-of-knowledge of (ZK, AoK)-combiners

Argument-of-knowledge uses exactly the same structure as our combiners for zero-knowledge of our (ZK, Sound)-combiners (intuitively, with extractors in place of simulators). For instance, for $**\mathbf{B}$ (ZK, AoK)-combiners, we say that $\mathbf{C}$ is black-box argument-of-knowledge if the following holds: if $\mathbf{C}$ is not black-box argument-of-knowledge (*i.e.*, no universal extractor exists), then it is possible to design $n - t + 1$ reductions to one of the security properties of distinct candidates. Hence, if the combiner is not argument-of-knowledge, it means that a security property is violated for at least $n - t + 1$ of the candidates.

Definition 20 (Argument-of-knowledge of t -out-of- n type $\in \{\mathbf{B}, \mathbf{N}\}^3$ (ZK, AoK)-combiners). *Let $\mathbf{C}$ be a correct n -candidate $\mathcal{F}_{\mathbf{N}}$ -combiner. We say a t -out-of- n type $\in \{\mathbf{B}, \mathbf{N}\}^3$ (ZK, AoK)-combiner $\mathbf{C}$ is argument-of-knowledge if, after ordering the quantifiers according to type in Tables 5, 6, the following holds (combAoKQuant is also defined in Table 5):*

combAoKQuant s.t. $(\mathbf{C}^{f_1, \dots, f_n}, (\mathcal{A}^{f_1, \dots, f_n}, \mathcal{E})) \in \mathcal{R}_{\mathbf{AoK}} \implies$ any of the following:

- $n - t + 1$ reductions to black-box (resp. non-black-box) zero-knowledge of the candidates if type = $*\mathbf{B}*$ (resp. type = $*\mathbf{N}*$), *i.e.*:

$$\begin{aligned} \exists I = \{i_1, \dots, i_{n-t+1}\} \subseteq [n] \text{ s.t. the following holds:} \\ \forall i_j \in I, \text{candZKQuant s.t. } (f_{i_j}, (\text{Red}_{i_j}, \mathcal{S}_{i_j})) \in \mathcal{R}_{\mathbf{ZK}}. \end{aligned}$$

- $n - t + 1$ reductions to black-box (resp. non-black-box) argument-of-knowledge of the candidates if type = $**\mathbf{B}$, *i.e.*:

$$\begin{aligned} \exists I = \{i_1, \dots, i_{n-t+1}\} \subseteq [n] \text{ s.t. the following holds:} \\ \forall i_j \in I, \text{candAoKQuant s.t. } (f_{i_j}, (\text{Red}_{i_j}, \mathcal{E}_{i_j})) \in \mathcal{R}_{\mathbf{AoK}}. \end{aligned}$$

type	Structural quantifiers	combAoKQuant
$\mathbf{B} * \mathbf{B}$	$\exists \mathbf{C} \forall f_1, \dots, f_n \in \mathcal{F}$	$\forall \mathcal{E} \exists \mathcal{A}$
$\mathbf{B} * \mathbf{N}$	$\exists \mathbf{C} \forall f_1, \dots, f_n \in \mathcal{F}$	$\exists \mathcal{A} \forall \mathcal{E}$
$\mathbf{N} * \mathbf{B}$	$\forall f_1, \dots, f_n \in \mathcal{F} \exists \mathbf{C}$	$\forall \mathcal{E} \exists \mathcal{A}$
$\mathbf{N} * \mathbf{N}$	$\forall f_1, \dots, f_n \in \mathcal{F} \exists \mathbf{C}$	$\exists \mathcal{A} \forall \mathcal{E}$

Table 5: Classification of argument-of-knowledge for (ZK, AoK)-combiners.

type	candZKQuant	type	candAoKQuant
B	$\forall \mathcal{S}_{i_j} \exists \text{Red}_{i_j}$	**B	$\forall \mathcal{E}_{i_j} \exists \text{Red}_{i_j}$
N	$\exists \text{Red}_{i_j} \forall \mathcal{S}_{i_j}$	**N	$\exists \text{Red}_{i_j} \forall \mathcal{E}_{i_j}$

Table 6: Quantifiers for ZK and AoK of candidates. $\mathcal{S}_{i_j}$, $\mathcal{E}_{i_j}$ are *PPT* algorithms. Red_{i_j} runs in time $\text{poly}(\tau)$.

Connections with the *standard* AoK notion. Following the same arguments used for the zero-knowledge case, we can claim that an AoK combiner executed on input t primitives that satisfies the *standard* notion of AoK also satisfies the *standard* AoK notion. For the case of black-box AoK, this implies that a secure combiner admits a universal extractor, and hence it may be used in any application in which a black-box AoK protocol is required.

C.2.2 Zero-knowledge of (ZK, AoK)-combiners

The definition of zero-knowledge of a (ZK, AoK)-combiner is essentially the same as the one described for (ZK, Sound) combiner, with the only difference being that we can obtain reductions to ZK or AoK of the candidates.

Definition 21 (Zero-knowledge of t -out-of- n type $\in \{\text{B}, \text{N}\}^3$ (ZK, AoK)-combiners). *Let $\mathcal{C}$ be a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner. We say a t -out-of- n type $\in \{\text{B}, \text{N}\}^3$ (ZK, AoK)-combiner $\mathcal{C}$ zero-knowledge if, after ordering the quantifiers according to type in Tables 6, 7, the following holds (combZKQuant is also defined in Table 7):*

combZKQuant s.t. $(\mathcal{C}^{f_1, \dots, f_n}, (\mathcal{A}^{f_1, \dots, f_n}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}} \implies$ any of the following:

- $n - t + 1$ reductions to black-box (resp. non-black-box) zero-knowledge of the candidates if type = *B* (resp. type = *N*), as in Definition 20.
- $n - t + 1$ reductions to black-box (resp. non-black-box) argument-of-knowledge of the candidates if type = **B, as in Definition 20.

type	Structural quantifiers	combZKQuant
BB*	$\exists \mathcal{C} \forall f_1, \dots, f_n \in \mathcal{F}$	$\forall \mathcal{S} \exists \mathcal{A}$
BN*	$\exists \mathcal{C} \forall f_1, \dots, f_n \in \mathcal{F}$	$\exists \mathcal{A} \forall \mathcal{S}$
NB*	$\forall f_1, \dots, f_n \in \mathcal{F} \exists \mathcal{C}$	$\forall \mathcal{S} \exists \mathcal{A}$
NN*	$\forall f_1, \dots, f_n \in \mathcal{F} \exists \mathcal{C}$	$\exists \mathcal{A} \forall \mathcal{S}$

Table 7: Classification of zero-knowledge for (ZK, AoK)-combiners.

D Non-black-box ZK/AoK and related breaks

In this section, we provide non-black-box variants of standard definitions for zero-knowledge and argument-of-knowledge, and show, in a similar fashion to Section 2.2, how to formalize security breaks by using the respective relations $\mathcal{R}_{\text{ZK}}$ (described in Definition 4) and $\mathcal{R}_{\text{AoK}}$ (described in Definition 19).

Definition 22 (Non-Black-Box Non-Interactive Zero-Knowledge). *Let $\Pi \in \mathcal{F}_{\text{NI}}$. Let $\text{REAL} = \{(pp, x, w, \pi) \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{P}(pp, x, w)\}_{\lambda \in \mathbb{N}}$ and $\text{IDEAL}_{\mathcal{S}} = \{(pp, x, w, \pi) \mid (pp, \tau) \xleftarrow{\$} \mathcal{S}^0(1^{|x|}, 1^\lambda); (x, w) \xleftarrow{\$} \mathcal{A}(pp); \pi \xleftarrow{\$} \mathcal{S}^1(pp, \tau, x)\}_{\lambda \in \mathbb{N}}$. We say that Π*

satisfies non-black-box zero-knowledge if, for all PPT algorithms $\mathcal{A}$, there exists a PPT algorithm $\mathcal{S} = (\mathcal{S}^0, \mathcal{S}^1)$ for which the following holds:

$$\Pr \left[b = b' \mid \begin{array}{l} b \xleftarrow{\$} \{0, 1\} \\ b' \leftarrow \mathcal{A}(pp, x, w, \pi) \end{array}; (pp, x, w, \pi) \xleftarrow{\$} \begin{cases} \text{REAL} & \text{if } b = 0 \\ \text{IDEAL}_{\mathcal{S}} & \text{if } b = 1 \end{cases} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Breaking non-black-box zero-knowledge. The standard definition of (computational) non-black-box zero-knowledge (reported above) states that a protocol Π is secure if, for every efficient adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ (*i.e.*, $\forall PPT \mathcal{A} \exists \mathcal{S}$ *s.t.* $(\Pi, (\mathcal{A}, \mathcal{S})) \notin \mathcal{R}_{\text{ZK}}$). Breaking non-black-box zero-knowledge therefore means that there exists an efficient adversary for which no secure simulator can be designed (*i.e.*, this adversary breaks *every* possible simulator). This is formalized as follows:

$$\exists PPT \mathcal{A} \text{ s.t. } \forall \mathcal{S}, (\Pi, (\mathcal{A}, \mathcal{S})) \in \mathcal{R}_{\text{ZK}}.$$

As discussed in Section 2.2, it is possible to obtain a statistical variant of the above by lifting the requirement of PPT adversaries.

Definition 23 (Non-Black-Box Argument-of-Knowledge). *Let $\Pi \in \mathcal{F}_{\text{NI}}$. We say that Π satisfies non-black-box argument-of-knowledge if, for all PPT algorithms $\mathcal{A}$, there exists a PPT algorithm $\mathcal{E} = (\mathcal{E}^0, \mathcal{E}^1)$ for which the following holds:*

$$\Pr \left[\mathcal{A}(pp) = 1 \mid pp \xleftarrow{\$} \text{setup}(1^{|x|}, 1^\lambda) \right] \approx_c \Pr \left[\mathcal{A}(pp) = 1 \mid (pp, \xi) \xleftarrow{\$} \mathcal{E}^0(1^{|x|}, 1^\lambda) \right],$$

and

$$\Pr \left[(x, w) \in \mathcal{R}_{\mathcal{L}}, \mathcal{V}(pp, x, \pi) = 1 \mid \begin{array}{l} (pp, \xi) \xleftarrow{\$} \mathcal{E}^0(1^n, 1^\lambda); (x, \pi) \leftarrow \mathcal{A}(pp) \\ w \leftarrow \mathcal{E}^1(pp, \xi, x, \pi) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Breaking non-black-box argument-of-knowledge. The standard definition of non-black-box argument-of-knowledge (reported above) states that a protocol Π is secure if, for every efficient adversary $\mathcal{A}$, there exists an extractor $\mathcal{E}$ (*i.e.*, $\forall PPT \mathcal{A} \exists \mathcal{E}$ *s.t.* $(\Pi, (\mathcal{A}, \mathcal{E})) \notin \mathcal{R}_{\text{AoK}}$). Breaking non-black-box argument-of-knowledge therefore means that there exists an efficient adversary for which no secure extractor can be designed (*i.e.*, this adversary breaks *every* possible extractor). This is formalized as follows:

$$\exists PPT \mathcal{A} \text{ s.t. } \forall \mathcal{E}, (\Pi, (\mathcal{A}, \mathcal{E})) \in \mathcal{R}_{\text{AoK}}.$$

Again, it is possible to obtain a definition applicable to proof-of-knowledge by considering unbounded adversaries.

E Instantiations of homomorphic languages and $\overline{\text{reconstruct}}_{\text{H}}$

In this section, we report concrete instantiations of $\overline{\text{reconstruct}}_{\text{H}}$ for some of the homomorphic languages presented in [Mau09].

We also note that, in the following examples, the witness is either in $\mathbb{Z}_q$ or $\mathbb{Z}_q \times \mathbb{Z}_q$. When the witness is in $\mathbb{Z}_q$, we know $\mathbb{Z}_q$ is actually a finite field, and the secret sharing scheme (share_{G} , $\text{reconstruct}_{\text{G}}$) in Section 4 satisfies correctness (as per Definition 14) and perfect security (as per Definition 15). Regarding $\mathbb{Z}_q \times \mathbb{Z}_q$, if we consider the addition operator as component-wise addition, and the multiplication operator as component-wise multiplication, then $\mathbb{Z}_q \times \mathbb{Z}_q$ is a finite field, and the correctness and perfect security of (share_{G} , $\text{reconstruct}_{\text{G}}$) also holds. Intuitively, for this case, we secret share each component of the witness, and perform the reconstruction component-wise.

DLOG. In DLOG, $(\mathbf{G}, \star) = (\mathbb{Z}_q, +)$, and $(\mathbf{H}, \otimes) = (\mathbf{H}, \cdot)$, with $\mathbf{H}$ being a group of prime order q . The homomorphic group operation is defined as:

$$\mathbf{G} \rightarrow \mathbf{H} : w \rightarrow f(w) = g^w$$

Therefore, $\{2\star\} = \cdot$, and $\{2\otimes\} = \exp$. More specifically:

- $f(w_1) \cdot f(w_2) = g^{w_1} \cdot g^{w_2} = g^{w_1+w_2} = f(w_1 + w_2)$
- $f(w) \exp v = g^w \exp v = \underbrace{g^w \cdot g^w \cdot \dots \cdot g^w}_{v \text{ copies of } g^w} = g^{w \cdot v} = f(w \cdot v)$

For completeness, we also instantiate the generic **share_G** of Section 4 with the operators defined above:

- **share_G**(w): $a_1, \dots, a_{t-1} \xleftarrow{\$} \text{Gen}_{\mathbf{G}}(1^\lambda)$. Output $\{w_i \leftarrow w + \sum_{j=1}^{t-1} (a_j \cdot i^j)\}_{i \in [n]}$
- **reconstruct_G**($\{s_i\}_{i \in I}, l$): Output $w_l \leftarrow \sum_{j \in I} (s_i \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k}))$

We expand **reconstruct_H** in terms of **reconstruct_G**, as in Section 4.

$$\begin{aligned} x'_l &= \bigotimes_{j \in I} (x_j \{2\otimes\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \prod_{j \in I} (g^{w_j} \exp(\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \prod_{j \in I} (g^{w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}) \\ &= g^{\sum_{j \in I} w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})} \\ &= g^{w_l} = f(\overline{\text{reconstruct}}_{\mathbf{G}}(\{w_j\}_{j \in I}, l)) = f(w_l) = x_l. \end{aligned}$$

DH tuples. In DH, $f(w) = (g^w, h^w)$, $(\mathbf{G}, \star) = (\mathbb{Z}_q, +)$, and $(\mathbf{H}, \otimes) = (\mathbf{H} \times \mathbf{H}, \odot)$, with $\mathbf{H}$ being a group of prime order q . The homomorphic group operation is defined as:

$$\mathbf{G} \rightarrow \mathbf{H} \times \mathbf{H} : w \rightarrow f(w) = (g^w, h^w)$$

The group operation of $\mathbf{H} \odot$ is the Hadamard product (*i.e.*, the component-wise product), with its matching hyperoperation $\{2\odot\}$ being the component-wise exponentiation. More specifically:

- $f(w_1) \odot f(w_2) = (g^{w_1} \cdot g^{w_2}, h^{w_1} \cdot h^{w_2}) = (g^{w_1+w_2}, h^{w_1+w_2}) = f(w_1 + w_2)$
- $f(w) \{2\odot\} v = \underbrace{f(w) \odot f(w) \odot \dots \odot f(w)}_{v \text{ copies of } f(w)} = (g^{w \cdot v}, h^{w \cdot v}) = f(w \cdot v)$

The algorithms **share_G**, **reconstruct_G** are exactly the same as in the previous example. We expand **reconstruct_H** in terms of **reconstruct_G**, as in Section 4.

$$\begin{aligned} x'_l &= \bigotimes_{j \in I} (x_j \{2\otimes\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \bigodot_{j \in I} ((g^{w_j}, h^{w_j}) \{2\odot\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \bigodot_{j \in I} (g^{w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}, h^{w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}) \\ &= (g^{\sum_{j \in I} w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}, h^{\sum_{j \in I} w_j \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}) \\ &= (g^{w_l}, h^{w_l}) = f(\overline{\text{reconstruct}}_{\mathbf{G}}(\{w_j\}_{j \in I}, l)) = f(w_l) = x_l. \end{aligned}$$

Pedersen commitments. In Pedersen commitments, $f(w_1, w_2) = h_1^{w_1} h_2^{w_2}$, $(G, \star) = (\mathbb{Z}_q \times \mathbb{Z}_q, \star)$, with $\star$ being the component-wise addition, with its matching hyperoperation $\{2\star\}$ being the component-wise multiplication, and $(H, \otimes) = (H, \cdot)$, with H being a group of prime order q . The homomorphic group operation is defined as:

$$\mathbb{Z}_q \times \mathbb{Z}_q \rightarrow H : (w_1, w_2) \rightarrow f(w_1, w_2) = h_1^{w_1} h_2^{w_2}$$

We expand the homomorphic operations as follows:

- $f(a) \otimes f(b) = (h_1^{a_1} h_2^{a_2}) \cdot (h_1^{b_1} h_2^{b_2}) = h_1^{a_1+b_1} h_2^{a_2+b_2} = f(a \star b)$
- $f(a)\{2\otimes\}v = \underbrace{(h_1^{a_1} h_2^{a_2}) \cdot (h_1^{a_1} h_2^{a_2}) \dots (h_1^{a_1} h_2^{a_2})}_{v \text{ copies of } a = (a_1, a_2)} = f(h_1^{va_1} h_2^{va_2}) = f(a\{2\star\}v)$

For completeness, we also report share_G instantiated with the operators defined above. Intuitively, we secret share each component of the witness on a different polynomial, and perform the reconstruction component-wise:

- $\text{share}_G(w = (w_1, w_2)) : (a_{1,1}, a_{2,1}) \dots, (a_{1,t-1}, a_{2,t-1}) \xleftarrow{\$} \text{Gen}_G(1^\lambda)$. Output $\{(w_{1,i}, w_{2,i}) \leftarrow (w_1 + \sum_{j=1}^{t-1} (a_{1,j} \cdot i^j), w_2 + \sum_{j=1}^{t-1} (a_{2,j} \cdot i^j))\}_{i \in [n]}$
- $\overline{\text{reconstruct}}_G(\{(s_{1,i}, s_{2,i})\}_{i \in I}, l)$:
Output $w = (w_1, w_2) \leftarrow (\sum_{j \in I} (s_{1,i} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})), \sum_{j \in I} (s_{2,i} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})))$

We expand $\overline{\text{reconstruct}}_H$ in terms of $\overline{\text{reconstruct}}_G$, as in Section 4.

$$\begin{aligned} x'_l &= \bigotimes_{j \in I} (x_j \{2\otimes\} (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \prod_{j \in I} (h_1^{a_{1,j}} h_2^{a_{2,j}} \exp(\prod_{k \in I, k \neq j} \frac{l-k}{j-k})) \\ &= \prod_{j \in I} (h_1^{a_{1,j} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})} h_2^{a_{2,j} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})}) \\ &= h_1^{\sum_{j \in I} a_{1,j} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})} h_2^{\sum_{j \in I} a_{2,j} \cdot (\prod_{k \in I, k \neq j} \frac{l-k}{j-k})} \\ &= f(\overline{\text{reconstruct}}_G(\{(a_{1,j}, a_{2,j})\}_{j \in I}, l)) = f(a_l) = x_l. \end{aligned}$$

F Combiner for homomorphic languages: formal treatment

In this section, we provide formal theorems and proofs for the combiner introduced in Section 4.

F.1 C_{HL} is a (ZK, Sound)-combiner

In order to claim that C_{HL} (as in Figure 4.1) is a t -out-of- n BB (ZK, Sound)-combiner, we need to argue the following:

- C_{HL} is a correct n -candidate $\mathcal{F}_{NI}$ -combiner for the language $\mathcal{L}_{HL}$, as per Definition 2.
- C_{HL} is a sound t -out-of- n BB (ZK, Sound)-combiner for the language $\mathcal{L}_{HL}$, as per Definition 5.
- C_{HL} is a zero-knowledge t -out-of- n BB (ZK, Sound)-combiner for the language $\mathcal{L}_{HL}$, as per Definition 6.

Below, we provide formal theorems and proofs for each of the aforementioned items.

Theorem 4. *The construction in Figure 4.1 is a correct n -candidate $\mathcal{F}_{NI}$ -combiner for the language $\mathcal{L}_{HL}$, as per Definition 2, for $n, t \in \text{poly}(\lambda)$.*

Proof. In order for the construction to be an n -candidate $\mathcal{F}_{\text{NI}}$ -combiner, the following has to hold:

$$\Pr \left[\mathsf{C}_{\text{vrfy}}(pp, x, \mathsf{C}_{\text{prove}}(pp, x, w)) = 1 \mid pp \xleftarrow{\$} \mathsf{C}_{\text{setup}}(1^{|x|}, 1^\lambda) \right] \geq 1 - \text{negl}(\lambda).$$

Intuitively, this happens if both (i) all sub-proofs are accepting, and (ii) all reconstructed values are consistent with the same polynomial, which evaluates to x in 0. We explore the two events separately.

Event 1. Let $\mathsf{E}_{\text{corr},k}$ as the event that the k -th sub-proof is accepting. We denote E_{corr} as the event that all sub-proofs are accepting, *i.e.*:

$$\Pr[\mathsf{E}_{\text{corr}}] = \prod_{k=1}^n \Pr[\mathsf{E}_{\text{corr},k}]$$

For $t \in \text{poly}(\lambda)$, the aforementioned quantity is overwhelming. By Bernoulli's inequality (*i.e.*, $(1+x)^n \geq 1+nx$), the following holds:

$$\begin{aligned} \Pr[\mathsf{E}_{\text{corr}}] &\geq (\min_k \{\Pr[\mathsf{E}_{\text{corr},k}]\})^n \\ &\geq (1 - \text{negl}(\lambda))^{\text{poly}(\lambda)} \\ &\geq 1 - \text{negl}(\lambda)\text{poly}(\lambda) \\ &\geq 1 - \text{negl}(\lambda). \end{aligned}$$

Event 2. The verifier computes $x'_l = \overline{\text{reconstruct}}_{\text{H}}(\{x_j\}_{j \in [t]}, l)$, for $l \in [n] \setminus [t] \cup \{0\}$. Following Section 4, this is equivalent to $f(\overline{\text{reconstruct}}_{\text{G}}(\{s_j\}_{j \in [t]}, l))$. By correctness of Shamir's secret sharing (as per Definition 14), the verifier successfully reconstructs the sub-statement $x'_l = f(w_l) = x_l$ with probability 1. $\square$

Theorem 5. *The construction in Figure 4.1 is a sound t -out-of- n BB (ZK, Sound)-combiner for the language $\mathcal{L}_{\text{HL}}$, as per Definition 5, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. Assume by contradiction that the construction C_{HL} is not sound. Then, there exists an algorithm $\mathcal{A}$ running in time τ for which $(\mathsf{C}_{\text{HL}}, \mathcal{A}) \in \mathcal{R}_{\text{Sound}}$ (as per Definition 5). This means that $\mathcal{A}$ outputs an accepting proof π for the combiner for a statement $x \notin \mathcal{L}_{\text{HL}}$ with non-negligible probability. The verifier parses (x, π) as $\{x_k, \pi_k\}_{k \in [n]}$, and behaves according to the protocol description. In particular, the verifier checks that (i) each sub-proof π_i verifies against the verifier of candidate i $\mathcal{V}_i$, and (ii) each sub-statement x_i is an evaluation of the same polynomial of degree $t-1$ in i .

In order to prove soundness for our combiner, it suffices to show that any accepting proof for an $x \notin \mathcal{L}_{\text{HL}}$ must yield a set of indices $I = \{i_1, \dots, i_{n-t+1}\}$ such that $(\Pi_{i_k}, \text{Red}_{i_k}) \in \mathcal{R}_{\text{Sound}}$ for all $i_k \in I$, where Red_{i_k} runs in time $\text{poly}(\tau)$. We prove that this fact holds in Lemma 2, and specify the behaviour of Red_{i_k} next.

Figure F.1: Reduction Red_{i_k}

- Receive public parameters pp'_{i_k} from the challenger $\mathcal{C}$.
- Compute $pp \xleftarrow{\$} \mathsf{C}_{\text{setup}}(1^l, 1^\lambda)$. Parse pp as $\{pp_j\}_{j \in [n]}$. Replace pp_{i_k} with pp'_{i_k} .
- Forward pp to $\mathcal{A}$.
- Upon receiving (x, π) from $\mathcal{A}$, Parse π as $\{\pi_k, x_k\}_{k \in [n]}$.
- Forward (x_{i_k}, π_{i_k}) to $\mathcal{C}$.

□

Lemma 2. For $t > \lfloor \frac{n}{2} \rfloor$, any accepting proof π for the combiner related to a statement $x \notin \mathcal{L}_{\text{HL}}$ must include at least $n-t+1$ distinct indices $I = \{i_1, \dots, i_{n-t+1}\}$ for which $(\Pi_{i_k}, \text{Red}_{i_k}) \in \mathcal{R}_{\text{Sound}}$ for all $i_k \in I$.

Proof. Assume by contradiction that the lemma statement does not hold. Then, there exists an accepting proof π for the combiner related to a statement $x \notin \mathcal{L}_{\text{HL}}$ that includes at most $n-t$ distinct indices $I = \{i_1, \dots, i_{n-t}\}$ for which π_{i_k} is an accepting proof for a statement $x_{i_k} \notin \mathcal{L}_{\text{HL}}$ for candidate Π_{i_k} , with $i_k \in I$.

Since the verifier of the combiner accepts proof π , all sub-proofs are accepting. In particular, this means that there exists a subset I' containing sub-proofs $\{\pi_{i_k}\}_{i_k \notin I}$ that do not yield reductions to soundness. Given that these proofs are accepting, they relate to sub-statements that are in the language. We observe that, by assumption, the cardinality of set I' is $n - (n-t) = t$. In what follows, we denote the set of shares identified by I' as $S = \{x_i\}_{i \in I'}$.

In order for the verifier of the combiner to accept proof π , it should be the case that $x = \overline{\text{reconstruct}}_{\text{H}}(\{x_k\}_{k \in I'}, 0)$, i.e.:

$$x = \bigotimes_{j \in I'} (x_j \{2 \otimes\} (\prod_{k \in I', k \neq j} \frac{k}{k-j}))$$

Since all the sub-statements in S are in the language (i.e., $\forall j \in I', x_j = f(w_j)$), the following holds:

$$x = \bigotimes_{j \in I'} (f(w_j) \{2 \otimes\} (\prod_{k \in I', k \neq j} \frac{k}{k-j}))$$

Then, we apply the homomorphism of f by invoking Fact 1 and Fact 2:

$$\begin{aligned} x &= \bigotimes_{j \in I'} (f(w_j) \{2 \otimes\} (\prod_{k \in I', k \neq j} \frac{k}{k-j})) \\ &= f(\bigstar_{j \in I'} (w_j \{2 \star\} (\prod_{k \in I', k \neq j} \frac{k}{k-j}))) \\ &= f(\overline{\text{reconstruct}}_{\text{G}}(\{w_j\}_{j \in I'}, 0)) \\ &= f(w). \end{aligned}$$

Note that, by the assumption of $t > \lfloor \frac{n}{2} \rfloor$, the reconstruction of the secret out of the n shares is uniquely determined by any set of size t . Hence, set S uniquely determines the secret $f(w)$. This is a contradiction to the assumption that $x \notin \mathcal{L}_{\text{HL}}$, as $x = f(w)$. Therefore, any accepting proof π for the combiner for a statement $x \notin \mathcal{L}_{\text{HL}}$ implies the existence of at least $n-t+1$ reductions to soundness of the underlying candidates.

□

Theorem 6. The construction in Figure 4.1 is a zero-knowledge t -out-of- n BB (ZK, Sound)-combiner for the language $\mathcal{L}_{\text{HL}}$, as per Definition 6, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.

Proof. Following the sketch provided in Section 4.3, we construct a simulator $\mathcal{S}^T$ and prove in Lemma 3 that either its output is indistinguishable from the output of the real prover, or there exists at least one reduction to zero-knowledge of one of the candidates whose indices are in T . Then, we argue in Lemma 7 that the existence of an adversary $\mathcal{A}$ who breaks zero-knowledge of the combiner implies the existence of sufficiently-many reductions to zero-knowledge of the underlying candidates, concluding the proof. □

Lemma 3. *There exists a simulator $\mathcal{S}^T$ for which either $\{\text{REAL}\} \approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$, or there exists a reduction to zero-knowledge of at least one of the candidates $\{\Pi_k\}_{k \in T}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. Moreover, we recall that we denote T_j as the set containing the first j indices of T , i.e., $T_j = \{i_1, \dots, i_j\}$. $\mathcal{S}^T = (\mathcal{S}^{T,0}, \mathcal{S}^{T,1})$ is constructed as follows:

Figure F.2: Simulator $\mathcal{S}^T$

$\mathcal{S}^{T,0}(1^l, 1^\lambda)$:

- For $l \in T$, compute $(pp_l, \tau_l) \xleftarrow{\$} \mathcal{S}_l^0(1^l, 1^\lambda)$.
- For $i \in [n] \setminus T$, compute $pp_i \leftarrow \text{setup}_i(1^l, 1^\lambda)$.
- Compute $pp \leftarrow \{pp_k\}_{k \in [n]}$, $\tau \leftarrow \{\tau_k\}_{k \in T}$.
- Return (pp, τ) .

$\mathcal{S}^{T,1}(pp, \tau, x)$:

- Sample $\{r_i\}_{i \in [n] \setminus T \cup T_{2t-n-1}} \xleftarrow{\$} \text{Gen}_G(1^\lambda)$.
- For $i \in [n] \setminus T \cup T_{2t-n-1}$, compute $x'_i \leftarrow f(r_i)$. Let $x'_0 \leftarrow x$.
- For $l \in T \setminus T_{2t-n-1}$, compute $x'_l \leftarrow \text{reconstruct}_H(\{x'_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$.
- For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, x'_i, r_i)$. For $l \in T$, compute $\pi_l \leftarrow \mathcal{S}_l^1(pp_l, \tau_l, x'_l)$.
- Return $(\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.

The proof of this lemma goes through $t + 3$ hybrid experiments, where hybrid $\mathcal{H}_0$ is the same as the real world, and hybrid $\mathcal{H}_{t+2}$ is the same as the ideal world. We denote the output of the adversary in hybrid $\mathcal{H}_i$ as $\text{out}^{\mathcal{H}_i}$, for $i \in \{0, 1, \dots, t+2\}$. Formally we prove that, for $j \in \{1, \dots, t+2\}$ and for any algorithm $\mathcal{A}$ running in time τ , the following holds:

$$\left| \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_{j-1}}) = 1] - \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_j}) = 1] \right| \leq \text{negl}(\lambda).$$

We define the hybrids as follows:

Figure F.3: Hybrid experiments $\mathcal{H}_0, \mathcal{H}_j, \mathcal{H}_{t+1}, \mathcal{H}_{t+2}$, for $1 \leq j \leq t$

$\mathcal{H}_0, \boxed{\mathcal{H}_j}, \boxed{\mathcal{H}_{t+1}}, \boxed{\mathcal{H}_{t+2}}$

1. $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $k \in [n]$. $pp \leftarrow \{pp_k\}_{k \in [n]}$.

1. $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_j$. $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. $pp \leftarrow \{pp_k\}_{k \in [n]}$.

2. Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$
3. Compute $\{w_i\}_{i \in [n]} \leftarrow \text{share}_G(w)$.
For $i \in [n]$, compute $x_i \leftarrow f(w_i)$.

3. Sample $\{r_i\}_{i \in [n] \setminus T \cup T_{2t-n-1}} \xleftarrow{\$} \text{Gen}_G(1^\lambda)$.
 For $i \in [n] \setminus T \cup T_{2t-n-1}$, compute $x'_i \leftarrow f(r_i)$. Let $r_0 \leftarrow w$.
 For $l \in T \setminus T_{2t-n-1}$, compute $w_l \leftarrow \text{reconstruct}_G(\{r_i\}_{i \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$. Compute $x_l \leftarrow f(w_l)$.
3. Sample $\{r_i\}_{i \in [n] \setminus T \cup T_{2t-n-1}} \xleftarrow{\$} \text{Gen}_G(1^\lambda)$.
 For $i \in [n] \setminus T \cup T_{2t-n-1}$, compute $x'_i \leftarrow f(r_i)$. Let $x'_0 \leftarrow x$.
 For $l \in T \setminus T_{2t-n-1}$, compute $x'_l \leftarrow \text{reconstruct}_H(\{x'_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$.
4. $\pi_k \leftarrow \mathcal{P}_k(pp_k, x_k, w_k)$ for $k \in [n]$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in [n]})$.
4. $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, x_k)$ for $k \in T_j$. $\pi_i \leftarrow \mathcal{P}_i(pp_i, x_i, w_i)$ for $i \in [n] \setminus T_j$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in [n]})$.
4. $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, x_k)$ for $k \in T$. $\pi_i \leftarrow \mathcal{P}_i(pp_i, x'_i, r_i)$ for $i \in [n] \setminus T$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in T \setminus T_{2t-n-1}} \parallel \{x'_k\}_{k \in [n] \setminus T \cup T_{2t-n-1}})$.
4. $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, x'_k)$ for $k \in T$. $\pi_i \leftarrow \mathcal{P}_i(pp_i, x'_i, r_i)$ for $i \in [n] \setminus T$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
5. Run $\mathcal{A}(pp, x, \pi)$.

Intuitively, in the first t hybrids, the real proofs generated by the candidates in T are gradually replaced with simulated proofs generated by their respective simulator. Note that, in principle, it could be the case that some of these candidates are not zero-knowledge. Hence, the adversary may be able to distinguish the output of two adjacent hybrids. This is acceptable as per the lemma statement, since in this case we would have a reduction to ZK of that specific candidate.

In $\mathcal{H}_{t+1}$, we replace $t - 1$ shares of the real witness w with $t - 1$ random group elements in G . This is possible as shares of the real witness are used in at most $t - 1$ of the prover algorithms of the candidates⁹. Hence, any adversary can leak at most $t - 1$ of the shares, and it is therefore unable to determine whether these are shares of w or random group elements. Since this argument is information theoretical, the two hybrids are identically distributed. Finally, the last hybrid is just a syntactic change that derives from homomorphic operations. Formally, this lemma is proved by having Lemma 4, Lemma 5, and Lemma 6. $\square$

Lemma 4 (Transition from $\mathcal{H}_{j-1}$ to $\mathcal{H}_j$, for $1 \leq j \leq t$). *If there exists an algorithm $\mathcal{A}$ running in time τ for which $\text{out}^{\mathcal{H}_{j-1}} \not\approx_c \text{out}^{\mathcal{H}_j}$, there exists an algorithm Red_{i_j} running in time $\text{poly}(\tau)$ that breaks zero-knowledge of the i_j -th scheme.*

Proof. Given $\mathcal{A}$ as in the theorem statement, we describe Red_{i_j} in what follows:

Figure F.4: Reduction Red_{i_j}

- Receive public parameters pp_{i_j} from the challenger $\mathcal{C}$.
- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_{j-1}$. Compute $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. Set $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$.

⁹More precisely, the real prover is used exactly in $n - t$ of the candidates. Therefore, $\mathcal{H}_1$ removes the dependency on the witness from all the proofs generated by real provers, as well as the first $t - 1 - (n - t) = 2t - n - 1$ elements of the set T (i.e., T_{2t-n-1}). From the honest majority assumption, the cardinality of T_{2t-n-1} is always greater or equal to 0. Looking ahead, this enables that the reconstruction procedure in $\mathcal{H}_2$ to be carried out with sufficiently-many shares.

- Compute $\{w_i\}_{i \in [n]} \leftarrow \text{share}_{\mathbf{G}}(w)$. Compute $\{x_i \leftarrow f(w_i)\}_{i \in [n]}$.
- Send $\{x_{i_j}, w_{i_j}\}$ to $\mathcal{C}$, and receive proof π_{i_j} . This proof is generated either as $\mathcal{P}_{i_j}(pp_{i_j}, x_{i_j}, w_{i_j})$ or $\mathcal{S}_{i_j}^1(pp_{i_j}, \tau_{i_j}, x_{i_j})$.
- Compute $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, x_k)$ for $k \in T_{j-1}$. Compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, x_i, w_i)$ for $i \in [n] \setminus T_j$.
- Set $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x_k\}_{k \in [n]})$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ provides a real proof, we are in hybrid $\mathcal{H}_{j-1}$. Otherwise, we are in hybrid $\mathcal{H}_j$. Therefore, Red_{i_j} retains the same distinguishing advantage of $\mathcal{A}$, breaking ZK of candidate Π_{i_j} . $\square$

Lemma 5 (Transition from $\mathcal{H}_t$ to $\mathcal{H}_{t+1}$). *out $^{\mathcal{H}_t}$ and out $^{\mathcal{H}_{t+1}}$ are identically distributed.*

Proof. Assume by contradiction there exists an unbounded algorithm $\mathcal{A}$ that can distinguish $\text{out}^{\mathcal{H}_t}$ from $\text{out}^{\mathcal{H}_{t+1}}$. Then, we can construct the following adversary $\overline{\mathcal{A}}^{\text{SS}}$ breaking security (as per Definition 15) of the t -out-of- n Shamir's secret sharing scheme:

Figure F.5: Adversary $\overline{\mathcal{A}}^{\text{SS}}$

- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T$. $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T$. $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$, and forward w to the challenger $\mathcal{C}$.
- Receive $t-1$ shares $\{s_i\}_{i \in [n] \setminus T \cup T_{2t-n-1}}$ from $\mathcal{C}$ which are either shares of the real witness w or random group elements $\{r_i\}_{i \in [n] \setminus T \cup T_{2t-n-1}} \xleftarrow{\$} \text{Gen}_{\mathbf{G}}(1^\lambda)$.
- Set $s_0 \leftarrow w$.
- For $l \in T \setminus T_{2t-n-1}$, compute $w_l \leftarrow \overline{\text{reconstruct}}_{\mathbf{G}}(\{s_i\}_{i \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$.
- For $l \in T \setminus T_{2t-n-1}$, compute $x_l \leftarrow f(w_l)$. For $i \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}$, compute $x_i \leftarrow f(s_i)$.
- For $l \in T$, compute $\pi_l \leftarrow \mathcal{S}_l^1(pp_l, \tau_l, x_l)$. For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, x_i, s_i)$.
- Set $\pi \leftarrow \{(\pi_k, x_k)\}_{k \in [n]}$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

First, observe that $\overline{\mathcal{A}}^{\text{SS}}$ uses w alongside the shares s_i output by the challenger $\mathcal{C}$ in order to reconstruct shares of the real w for $i \in T \setminus T_{2t-n-1}$. This is necessary, as both $\mathcal{H}_t$ and $\mathcal{H}_{t+1}$ simulate proofs for $i \in T \setminus T_{2t-n-1}$ by supplying $x_i = f(w_i)$ as the input to the simulators of the candidates whose indices are in $T \setminus T_{2t-n-1}$.

We now argue that $\overline{\mathcal{A}}^{\text{SS}}$ correctly reproduces the behavior of hybrids $\mathcal{H}_t$ and $\mathcal{H}_{t+1}$ according to the challenger's behaviour. If the challenger shares the real witness w , all the proofs in $[n] \setminus T \cup T_{2t-n-1}$ are generated exactly as in $\mathcal{H}_t$ (i.e., using shares of w). If the challenger generates random group elements in $\mathbf{G}$, all the proofs in $[n] \setminus T \cup T_{2t-n-1}$ are generated exactly as in $\mathcal{H}_{t+1}$ (i.e., using random group elements in $\mathbf{G}$). Therefore, $\overline{\mathcal{A}}^{\text{SS}}$ retains the same distinguishing advantage of $\mathcal{A}$, breaking security of Shamir's secret sharing scheme. This is a contradiction; therefore, $\text{out}^{\mathcal{H}_t} \equiv \text{out}^{\mathcal{H}_{t+1}}$. $\square$

Lemma 6 (Transition from $\mathcal{H}_{t+1}$ to $\mathcal{H}_{t+2}$). *out $^{\mathcal{H}_{t+1}}$ and out $^{\mathcal{H}_{t+2}}$ are identically distributed.*

Proof. The proof follows by comparing how the sub-statements are generated and by applying the homomorphism of the language. More precisely:

- **Comparing** $\{x_l\}_{l \in [n] \setminus T \cup T_{2t-n-1}}$. In both experiments, these sub-statements are generated as $f(r_l)$ for a random $r_l \in \mathbf{G}$.

- **Comparing x_0 .** In $\mathcal{H}_{t+1}$, $x_0 = f(r_0)$, with $r_0 = w$. In $\mathcal{H}_{t+2}$, $x_0 = x$. Since $f(w) = x$ by assumption, the two sub-statements are identical.
- **Comparing $\{x_l\}_{l \in T \setminus T_{2t-n-1}}$.** In $\mathcal{H}_{t+1}$, $x_l = f(w_l)$, with $w_l = \text{reconstruct}_G(\{r_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$. In $\mathcal{H}_{t+2}$, $x_l = \text{reconstruct}_H(\{x_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)$, with $x_j = f(r_j)$. We show that these sub-statements are identical by applying the homomorphic property of the language, similarly to the proof for Theorem 4:

$$\begin{aligned}
f(w_l) &= f(\overline{\text{reconstruct}_G}(\{r_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l)) \\
&= f(\bigstar_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}} (r_j \{2\star\} (\prod_{k \in T \setminus T_{2t-n-1}, k \neq j} \frac{l-k}{j-k}))) \\
&= \bigotimes_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}} (f(r_j) \{2\otimes\} (\prod_{k \in T \setminus T_{2t-n-1}, k \neq j} \frac{l-k}{j-k})) \\
&= \overline{\text{reconstruct}_H}(\{f(r_j)\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l) \\
&= \overline{\text{reconstruct}_H}(\{x_j\}_{j \in [n] \setminus T \cup T_{2t-n-1} \cup \{0\}}, l).
\end{aligned}$$

□

Lemma 7. *The existence of adversaries breaking zero-knowledge of all possible simulators $\mathcal{S}^T$, as described in Lemma 3, yields reductions to zero-knowledge of at least $n - t + 1$ distinct candidates.*

Proof. Let S_0 be a set of cardinality $\binom{n}{t}$ containing all the possible instantiations of $\mathcal{S}^T$ as described in Lemma 3 (i.e., by considering all the possible choices of T out of the n candidates). The fact that the combiner is not zero-knowledge (as per Definition 21) implies that no simulator in S_0 can be such that $\{\text{REAL}\} \approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$. Moreover, Lemma 3 prescribes that if $\{\text{REAL}\} \not\approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$, there exists a reduction to at least one of the candidate schemes identified by an index in T . Let $i_1 \in [n]$ be such an index. This means that there exists an adversary $\mathcal{A}$ that is able to break zero-knowledge for scheme Π_{i_1} , i.e., $\mathcal{A}$ breaks all possible simulators of Π_{i_1} . Observe that *any* simulator that is parameterized by a T containing i_1 is such that $\{\text{REAL}\} \not\approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$. However, there still exists a set S_1 of cardinality $\binom{n-1}{t}$ containing all the simulators parameterized by a T that do *not* contain i_1 . Since there should exist (a possibly different) $\mathcal{A}$ that breaks all the simulators in S_1 , an index $i_2 \in [n]$ distinct from i_1 exists. This means that $\mathcal{A}$ is able to break zero-knowledge for scheme Π_{i_2} . Generalizing the above, we denote I_j as the set of indices of cardinality j for which some $\mathcal{A}$ is able to break zero-knowledge of the respective candidate Π_{i_k} . We further denote S_j as the set of simulators that are parameterized by a T that does not contain any $i_k \in I_j$. The cardinality of S_j is $\binom{n-j}{t}$. Since $\mathcal{A}$ must also break all the simulators in S_j , there exists another index $i^* \in [n] \setminus I_j$ related to a scheme Π_{i^*} for which $\mathcal{A}$ breaks its ZK property.

In order to have $|S_j| = 0$, it suffices to show that $t > n - j$. The minimum j for which this condition holds is $j = n - t + 1$, as $\binom{t-1}{t} = 0$. Hence, the existence of adversaries breaking zero-knowledge of the combiner implies the existence of at least $n - t + 1$ reductions to zero-knowledge of distinct candidates, concluding the proof. □

F.2 C_{HL} is a (ZK, AoK)-combiner

In order to claim that C_{HL} (as in Figure 4.1) is a t -out-of- n BBB (ZK, AoK)-combiner, we need to argue the following:

- C_{HL} is a correct n -candidate $\mathcal{F}_{NI}$ -combiner for the language $\mathcal{L}_{HL}$, as per Definition 2.
- C_{HL} is an argument-of-knowledge t -out-of- n BBB (ZK, AoK)-combiner for the language $\mathcal{L}_{HL}$, as per Definition 20.
- C_{HL} is a zero-knowledge t -out-of- n BBB (ZK, AoK)-combiner for the language $\mathcal{L}_{HL}$, as per Definition 21.

Note that the first item already holds by Theorem 4. In what follows, we formalize the remaining items.

Theorem 7. *The construction in Figure 4.1 is an argument-of-knowledge t -out-of- n BBB (ZK, AoK)-combiner for the language $\mathcal{L}_{HL}$, as per Definition 20, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. Following Section 4.3, we construct an extractor $\mathcal{E}^T$ and prove in Lemma 8 that either it successfully extracts a witness, or there exists at least one reduction to AoK of one of the candidates whose indices are in T . From there, one can exhibit an argument similar to Lemma 7 to argue that the existence of adversaries who break AoK of the combiner implies the existence of sufficiently-many reductions to AoK of the underlying candidates, concluding the proof. This step is the same for all our combiners, and is formally proved in Lemma 10. $\square$

Lemma 8. *There exists an extractor $\mathcal{E}^T$ for which either a witness w is successfully extracted from an accepting proof, or there exists a reduction to argument-of-knowledge of at least one of the candidates $\{\Pi_k\}_{k \in T}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. The extractor $\mathcal{E}^T = (\mathcal{E}^{T,0}, \mathcal{E}^{T,1})$ is constructed as follows:

Figure F.6: Extractor $\mathcal{E}^T$
$\mathcal{E}^{T,0}(1^l, 1^\lambda)$: • For $k \in T$, compute $(pp_k, \xi_k) \xleftarrow{\\$} \mathcal{E}_k^0(1^l, 1^\lambda)$. • Compute $pp \leftarrow \{pp_k\}_{k \in [n]}$, $\xi \leftarrow \{\xi_k\}_{k \in T}$. • Return (pp, ξ). $\mathcal{E}^{T,1}(pp, \xi, x, \pi)$: • For $k \in T$, parse the k-th component of π as (x_k, π_k). Compute $w_k \leftarrow \mathcal{E}_k^1(pp_k, \xi_k, x_k, \pi_k)$. • Output $w \leftarrow \text{reconstruct}_G(\{w_j\}_{j \in T})$.

As per Definition 19, an adversary may break argument-of-knowledge either by detecting the introduction of trapdoor setups, or by outputting an accepting proof that causes any extractor to fail. If either occurs, we exhibit a reduction to AoK of that specific candidate.

Introducing trapdoor setups. We define t hybrid experiments: in hybrid $\mathcal{H}_0$ all public parameters for the candidates in T are generated by setup algorithms; in hybrid $\mathcal{H}_t$, all public parameters for the candidates in T are generated by trapdoor setup algorithms. We denote the output of the adversary in hybrid $\mathcal{H}_i$ as $\text{out}^{\mathcal{H}_i}$, for $i \in \{0, 1, \dots, t\}$. Formally, for $j \in \{1, \dots, t\}$ and for any algorithm $\mathcal{A}$ running in time τ , the following holds:

$$\left| \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_{j-1}}) = 1] - \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_j}) = 1] \right| \leq \text{negl}(\lambda).$$

We define the hybrids as follows, and prove that the above holds in Lemma 9:

Figure F.7: Hybrid experiments $\mathcal{H}_0, \mathcal{H}_j$, for $1 \leq j \leq t$

$\mathcal{H}_0, \boxed{\mathcal{H}_j}$

1. $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $k \in [n]$. $pp \leftarrow \{pp_k\}_{k \in [n]}$.

1. $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_j$. $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. $pp \leftarrow \{pp_k\}_{k \in [n]}$.

2. Run $\mathcal{A}(pp)$.

Lemma 9 (Transition from $\mathcal{H}_{j-1}$ to $\mathcal{H}_j$, for $1 \leq j \leq t$). *If there exists an algorithm $\mathcal{A}$ running in time τ for which $\text{out}^{\mathcal{H}_{j-1}} \not\approx_c \text{out}^{\mathcal{H}_j}$, there exists an algorithm Red_{i_j} running in time $\text{poly}(\tau)$ that breaks argument-of-knowledge of the i_j -th scheme.*

Proof. Given $\mathcal{A}$ as in the theorem statement, we describe Red_{i_j} in what follows:

Figure F.8: Reduction Red_{i_j}

- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{E}_k^0(1^l, 1^\lambda)$ for $k \in T_{j-1}$. Compute $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$.
- Receive public parameters pp_{i_j} from the challenger $\mathcal{C}$. These public parameters are generated either by using $\text{setup}_{i_j}(1^l, 1^\lambda)$ or $\mathcal{E}_{i_j}^0(1^l, 1^\lambda)$.
- Set $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- Run $\mathcal{A}(pp)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ generates the public parameters using the setup algorithm, we are in hybrid $\mathcal{H}_{j-1}$. Otherwise, we are in hybrid $\mathcal{H}_j$. Therefore, Red_{i_j} retains the same distinguishing advantage of $\mathcal{A}$, breaking AoK of candidate Π_{i_j} . $\square$

Extractor $\mathcal{E}^T$ fails to extract a witness. If any reduction occurred throughout the chain of hybrids, the existence of the extractor in Figure F.6 already implies a reduction to AoK for one of the candidates, as prescribed by the lemma statement. Indeed, if the adversary is able to distinguish whether any of the public parameters of candidates in T is generated by a trapdoor setup algorithm (*i.e.*, $\mathcal{E}_k^0$ for some $k \in T$), its behaviour may change arbitrarily when interacting with $\mathcal{E}^T$.

Conversely, if no reduction occurred so far, the adversary has at most an advantage of $t \cdot \text{negl}(\lambda)$ in discerning the generation algorithm used for $pp = \{pp_k\}_{k \in [n]}$. Hence, it will always output an accepting proof upon input pp except for negligible probability. Given that, the proof continues only if no reduction was possible in the chain of hybrids, as the adversary interacts with an extractor that controls the trapdoors used to generate pp .

We observe that the only way $\mathcal{E}^T$ can reconstruct w is by successfully extracting all the sub-witnesses in T and using the **reconstruct_G** algorithm of Shamir's secret sharing scheme. Since the verifier $\mathcal{C}_{\text{vrfy}}$ runs **reconstruct_H** as a consistency check, when all the checks pass, it means that all sub-statements are shares of the statement x . By the property of **reconstruct_H** we have in Section 4, all the t extracted shares are shares of w . Hence, by correctness of Shamir's secret sharing, we reconstruct w successfully.

Therefore, in order for $\mathcal{E}^T$ to fail, it suffices for one of the sub-extraction procedures to fail, as otherwise the reconstruction of Shamir's secret sharing would have been successful and $\mathcal{E}^T$ would not have failed. Let i^* be an index for which $\mathcal{E}_{i^*}^1$ fails to extract from (x_{i^*}, π_{i^*}) . Any adversary that outputs an accepting proof (x, π) for the combiner that contains (x_{i^*}, π_{i^*}) in

position i^* immediately causes $\mathcal{E}^T$ to fail, yielding a reduction to AoK of scheme Π_{i^*} . We describe Red_{i^*} in what follows:

Figure F.9: Reduction Red_{i^*}

- Receive public parameters pp'_{i^*} from the challenger $\mathcal{C}$.
- Compute $pp \xleftarrow{\$} \text{C}_{\text{setup}}(1^l, 1^\lambda)$. Parse pp as $\{pp_j\}_{j \in [n]}$. Replace pp_{i^*} with pp'_{i^*} .
- Forward pp to $\mathcal{A}$.
- Upon receiving (x, π) from $\mathcal{A}$, Parse π as $\{\pi_k, x_k\}_{k \in [n]}$.
- Forward (x_{i^*}, π_{i^*}) to $\mathcal{C}$.

□

Lemma 10. *The existence of adversaries breaking argument-of-knowledge of $\mathcal{E}^T$ as described in Lemma 8 yields reductions to argument-of-knowledge for at least $n - t + 1$ distinct candidates.*

Proof. The proof uses the same combinatorics argument described for the proof of Lemma 7. In Lemma 8, we define $\binom{n}{t}$ extractors, and the existence of adversaries breaking AoK for the combiner can be used to also break the security of all the extractors. In particular, there exists an index i^* associated with candidate Π_{i^*} for which an adversary either detects the introduction of the trapdoor generation algorithm, or it makes sub-extractor $\mathcal{E}_{i^*}^1$ fail upon input an accepting sub-proof. In order to break AoK for all the possible extractors, there should exist $n - t + 1$ such indices that are distinct, *i.e.*, there should exist adversaries breaking AoK for $n - t + 1$ of the underlying schemes. The existence of these adversaries implies the existence of at least $n - t + 1$ reductions to AoK of distinct candidates, concluding the proof. □

Theorem 8. *The construction in Figure 4.1 is a zero-knowledge t -out-of- n BBB (ZK, AoK)-combiner for the language $\mathcal{L}_{\text{HL}}$, as per Definition 21, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. Within the proof for Theorem 6, we showed that it is possible to break zero-knowledge of the combiner by solely relying on zero-knowledge of the candidates (*i.e.*, by obtaining $n - t + 1$ reductions to zero-knowledge of the candidates). Hence, we reuse exactly the same proof. □

G Recursive proof combiner

In this section we complete the formal treatment of Section 5.

G.1 C_{PoP} is a (ZK, AoK)-combiner

In order to claim that C_{PoP} (as in Figure 5.1) is a t -out-of- n NBB (ZK, AoK)-combiner, we need to argue the following:

- C_{PoP} is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner for any NP language $\mathcal{L}$, as per Definition 2.
- C_{PoP} is an argument-of-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 20.
- C_{PoP} is a zero-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 21.

Below, we provide formal theorems and proofs for each of the aforementioned items.

Theorem 9. *The construction in Figure 5.1 is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner for any NP language $\mathcal{L}$, as per Definition 2, for $n, t \in O(1)$.*

Proof. In order for the construction to be an n -candidate $\mathcal{F}_{\text{NI}}$ -combiner, the following has to hold:

$$\Pr \left[C_{\text{vrfy}}(pp, x, C_{\text{prove}}(pp, x, w)) = 1 \mid pp \xleftarrow{\$} C_{\text{setup}}(1^{|x|}, 1^\lambda) \right] \geq 1 - \text{negl}(\lambda).$$

The verifier of the combiner accepts a proof if all the sub-proofs are accepting. Denoting $E_{\text{corr},i}$ as the event that the i -th sub-proof is accepting, we first argue that $\Pr[E_{\text{corr},i}] \geq 1 - \text{negl}(\lambda)$ for all $i \in \binom{n}{t}$. Given that π_i is generated by nesting t candidates, the success probability of generating an accepting π_i depends on the event that all the sub-proofs generated in the chain of proofs are accepting. Let E_{corr,i_k} be the event that the i_k -th proof internally generated as part of π_i is accepting. Since $t = O(1)$, we apply Bernoulli's inequality (*i.e.*, $(1+x)^n \geq 1+nx$) to argue that $\Pr[E_{\text{corr},i}]$ is overwhelming:

$$\begin{aligned} \Pr[E_{\text{corr},i}] &= \prod_{k=1}^t \Pr[E_{\text{corr},i_k}] \\ &\geq (\min_k \{\Pr[E_{\text{corr},i_k}]\})^t \\ &\geq (1 - \text{negl}(\lambda))^{O(1)} \\ &\geq 1 - \text{negl}(\lambda) \cdot O(1) \\ &\geq 1 - \text{negl}(\lambda). \end{aligned}$$

Finally, we observe that, for $t, n \in O(1)$, $\binom{n}{t} \in O(1)$. Denoting E_{corr} as the event that all sub-proofs are accepting, we argue that $\Pr[E_{\text{corr}}]$ is overwhelming:

$$\begin{aligned} \Pr[E_{\text{corr}}] &= \prod_{i=1}^{\binom{n}{t}} \Pr[E_{\text{corr},i}] \\ &\geq (\min_i \{\Pr[E_{\text{corr},i}]\})^{\binom{n}{t}} \\ &\geq (1 - \text{negl}(\lambda))^{O(1)} \\ &\geq 1 - \text{negl}(\lambda). \end{aligned}$$

□

Theorem 10. *The construction in Figure 5.1 is a zero-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 21, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in O(1)$.*

Proof. The structure of this proof is the same as the one sketched in Section 4.3. First, we construct a simulator $\mathcal{S}^T$, and argue that either its output is indistinguishable from the output of the real prover, or there exists at least one reduction to zero-knowledge of one of the candidates whose indices are in T . We describe such a $\mathcal{S}^T$ in Lemma 11. With that, we can use exactly the same the combinatorics argument used to prove Lemma 7.

□

Lemma 11. *There exists a simulator $\mathcal{S}^T$ for which either $\{\text{REAL}\} \approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$, or there exists a reduction to zero-knowledge of at least one of the candidates $\{\Pi_k\}_{k \in T}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. $\mathcal{S}^T = (\mathcal{S}^{T,0}, \mathcal{S}^{T,1})$ is constructed as follows:

Figure G.1: Simulator $\mathcal{S}^T$

$\mathcal{S}^{T,0}(1^l, 1^\lambda)$:

- For $l \in T$, compute $(pp_l, \tau_l) \xleftarrow{\$} \mathcal{S}_l^0(1^l, 1^\lambda)$.
- For $i \in [n] \setminus T$, compute $pp_i \leftarrow \text{setup}_i(1^l, 1^\lambda)$.
- Compute $pp \leftarrow \{pp_k\}_{k \in [n]}$, $\tau \leftarrow \{\tau_k\}_{k \in [t]}$.
- Return (pp, τ) .

$\mathcal{S}^{T,1}(pp, \tau, x)$:

- Index all possible subsets containing t of the n candidates with set $S = \{S^i\}_{i \in \binom{n}{t}}$.
- Denote S^{T_k} as the set of all proofs containing candidate i_k but not candidates in T_{k-1} (i.e., $S^{T_k} = \{S^i \mid \forall i \in \binom{n}{t} \text{ s.t. } i_k \in S^i \wedge S^i \cap T_{k-1} = \emptyset\}$). Denote every $S^i \in S^{T_k}$ as $S^{T_k,i}$.
- For $k \in [t]$, for $S^{T_k,i} \in S^{T_k}$, simulate as follows:
 - Compute $\pi_{T_k,i} \leftarrow \mathcal{S}_{i_k}^1(pp_{i_k}, \tau_{i_k}, x_{T_k,i})$, where $x_{T_k,i} \leftarrow (\{\mathcal{V}_l, pp_l\}_{l \in S^{T_k,i} \wedge l < i_k}, x)$, and proceed as follows:
 - * If $\pi_{T_k,i}$ is the outermost proof, set $\pi_{T_k,i}^{\text{PoP}} \leftarrow \pi_{T_k,i}$.
 - * Otherwise, obtain $\pi_{T_k,i}^{\text{PoP}}$ by recursively running the prover algorithm $\{\mathcal{P}_l\}_{l \in S^{T_k,i} \wedge l > i_k}$, using $\pi_{T_k,i}$ as a valid witness (i.e., an accepting proof), as per the combiner description.

The proof of this lemma goes through $t + 1$ hybrid experiments, where hybrid $\mathcal{H}_0$ is the same as the real world, and hybrid $\mathcal{H}_t$ is the same as the ideal world. We denote the output of the adversary in hybrid $\mathcal{H}_i$ as $\text{out}^{\mathcal{H}_i}$, for $i \in \{0, 1, \dots, t\}$. Formally we prove that, for $j \in \{1, \dots, t\}$ and for any algorithm $\mathcal{A}$ running in time τ , the following holds:

$$\left| \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_{j-1}}) = 1] - \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_j}) = 1] \right| \leq \text{negl}(\lambda).$$

We define the hybrids as follows:

Figure G.2: Hybrid experiments $\mathcal{H}_0, \mathcal{H}_j$, for $1 \leq j \leq t$

$\mathcal{H}_0, \boxed{\mathcal{H}_j}$

1. $pp \xleftarrow{\$} \mathcal{C}_{\text{setup}}(1^l, 1^\lambda)$.

1. $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $i \in [j]$. $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus [j]$. $pp \leftarrow \{pp_i\}_{i \in [n]}$.

2. Index all possible subsets containing t of the n candidates with set $S = \{S^i\}_{i \in \binom{n}{t}}$.

3. For $S^i \in S$, compute π_i as the honest prover.

3. Denote $S^{T_k} = \{S^i \mid \forall i \in \binom{n}{t} \text{ s.t. } i_k \in S^i \wedge S^i \cap T_{k-1} = \emptyset\}$. We denote every $S^i \in S^{T_k}$ as $S^{T_k,i}$.

For $k \in [j]$, for $S^{T_k,i} \in S^{T_k}$, simulate as follows:

- Compute $\pi_{T_k,i} \leftarrow \mathcal{S}_{i_k}^1(pp_{i_k}, \tau_{i_k}, x_{T_k,i})$, where $x_{T_k,i} \leftarrow (\{\mathcal{V}_l, pp_l\}_{l \in S^{T_k,i} \wedge l < i_k}, x)$, and proceed as follows:
 - If $\pi_{T_k,i}$ is the outermost proof, set $\pi_{T_k,i}^{\text{PoP}} \leftarrow \pi_{T_k,i}$.
 - Otherwise, obtain $\pi_{T_k,i}^{\text{PoP}}$ by recursively running the prover algorithm $\{\mathcal{P}_l\}_{l \in S^{T_k,i} \wedge l > i_k}$, using $\pi_{T_k,i}$ as a valid witness (i.e., an accepting proof), as per the combiner description.

For $S^i \in S \setminus \{S^{T_k}\}_{k \in [j]}$, compute π_i as the honest prover of the combiner.

4. Set $\pi \leftarrow \{\pi_i^{\text{PoP}}\}_{i \in \binom{n}{t}}$.

5. Run $\mathcal{A}(pp, x, \pi)$.

Intuitively, in $\mathcal{H}_j$ we simulate all the proofs containing candidate i_j that were not already simulated in any previous hybrid. If the proof to be simulated is the outermost one, the dependency on the witness is removed by running the simulator for the outermost proof system. In any other case, the simulation generates an accepting intermediate proof without knowing its respective witness, which is used as a valid witness by recursively running the prover algorithm as per the combiner description.

Finally, we argue that the simulator is indeed independent from the witness (*i.e.*, that all proofs were simulated) through a simple combinatorics argument. Hence, if no reduction occurred throughout the hybrid chain, the simulator successfully removes the dependency of the witness from all the $\binom{n}{t}$ proofs. We formalize the above in Lemma 12 and Lemma 13.

On the indistinguishability of the hybrid chain. If no reduction occurred, each simulated proof yields an advantage to the adversary of $\text{negl}(\lambda)$. Hence, the advantage of the adversary from $\mathcal{H}_0$ to $\mathcal{H}_t$ is upper bounded by $\binom{n}{t}\text{negl}(\lambda)$. In order for this quantity to be negligible, it should be the case that $\binom{n}{t} \in \text{poly}(\lambda)$. Because we have $n, t \in O(1)$, $\binom{n}{t} \in O(1)$, satisfying our requirement. $\square$

Lemma 12 (Transition from $\mathcal{H}_{j-1}$ to $\mathcal{H}_j$, for $1 \leq j \leq t$). *If there exists an algorithm $\mathcal{A}$ running in time τ for which $\text{out}^{\mathcal{H}_{j-1}} \not\approx_c \text{out}^{\mathcal{H}_j}$, there exists an algorithm Red_{i_j} running in time $\text{poly}(\tau)$ that breaks zero-knowledge of the i_j -th scheme.*

Proof. We describe a reduction Red_{i_j} given $\mathcal{A}$ as in the theorem statement:

Figure G.3: Reduction Red_{i_j}

- Receive public parameters pp_{i_j} from the challenger $\mathcal{C}$.
- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_{j-1}$. Compute $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. Set $pp \leftarrow \{pp_k\}_{k \in [n]}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$.
- Index all possible subsets containing t of the n candidates with set $S = \{S^i\}_{i \in \binom{n}{t}}$.
- Denote $S^{T_k} = \{S^i \mid \forall i \in \binom{n}{t} \text{ s.t. } i_k \in S^i \wedge S^i \cap T_{k-1} = \emptyset\}$. We denote every $S^i \in S^{T_k}$ as $S^{T_k, i}$.
- For $k \in [j-1]$, for $S^{T_k, i} \in S^{T_k}$, simulate proofs as per the description of $\mathcal{H}_{j-1}$:
- Send (x, w) to $\mathcal{C}$, and receive proofs $\{\pi_l^{\text{PoP}}\}_{l \in S^{T_j}}$. These proofs are either generated by running the honest prover of the combiner, or by running $\mathcal{S}_{i_j}^1$, as described in $\mathcal{H}_j$.
- For $S^i \in S \setminus \{S^{T_k}\}_{k \in [j]}$, compute π_i as the honest prover of the combiner.
- Set $\pi \leftarrow \{\pi_i^{\text{PoP}}\}_{i \in \binom{n}{t}}$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ provides real proofs, we are in hybrid $\mathcal{H}_{j-1}$. Otherwise, we are in hybrid $\mathcal{H}_j$. Therefore, Red_{i_j} retains the same distinguishing advantage of $\mathcal{A}$, breaking multi-proof ZK of candidate Π_{i_j} .¹⁰ $\square$

¹⁰For convenience, we use multi-proof zero-knowledge. We remark that this is for compactness of notation in the setup generation algorithm of the combiner, and comes without loss of generality. Indeed, the same can be achieved without multi-proof ZK by generating a different trapdoor for each individual proof.

Lemma 13. *The output of $\mathcal{S}^T$ is independent from the witness.*

Proof. First, we observe that $\mathcal{S}^T$ exclusively simulates all the proofs that include any candidate $\{\Pi_k\}_{k \in T}$ as part of their generation procedure. In order to show that the output of $\mathcal{S}^T$ is independent from the witness, we define a set of proofs $\bar{S}$ that do not include any of the candidates in T as part of their generation procedure. The cardinality of $\bar{S}$ is $\binom{n-t}{t}$. Since $t > \lfloor \frac{n}{2} \rfloor$ by assumption, it trivially holds that $t > n - t$. This implies that $|\bar{S}| = 0$. Therefore, all proofs are simulated. $\square$

Theorem 11. *The construction in Figure 5.1 is an argument-of-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 20, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in O(1)$.*

Proof. Following Section 4.3, we construct an extractor $\mathcal{E}^T$ and prove in Lemma 14 that either it successfully extracts a witness, or there exists at least one reduction to AoK of one of the candidates whose indices are in T . With that, we can use exactly the same the combinatorics argument used to prove Lemma 10. $\square$

Lemma 14. *There exists an extractor $\mathcal{E}^T$ for which either a witness w is successfully extracted from an accepting proof, or there exists a reduction to argument-of-knowledge of at least one of the candidates $\{\Pi_k\}_{k \in T}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. The extractor $\mathcal{E}^T = (\mathcal{E}^{T,0}, \mathcal{E}^{T,1})$ is constructed as follows:

Figure G.4: Extractor $\mathcal{E}^T$
$\mathcal{E}^{T,0}(1^l, 1^\lambda)$: - For $k \in T$, compute $(pp_k, \xi_k) \xleftarrow{\\$} \mathcal{E}_k^0(1^l, 1^\lambda)$. - Compute $pp \leftarrow \{pp_k\}_{k \in [n]}$, $\xi \leftarrow \{\xi_k\}_{k \in T}$. - Return (pp, ξ). $\mathcal{E}^{T,1}(pp, \xi, x, \pi)$: - Parse π as $\{\pi_i^{\text{PoP}}\}_{i \in \binom{n}{t}}$. - Index all possible subsets containing t of the n candidates with set $S = \{S_i\}_{i \in \binom{n}{t}}$. - Select the proof π for which $S_i = T$, and set $\pi_{i_t} \leftarrow \pi$. - For $k \in \{0, \dots, t-2\}$, compute $\pi_{i_{t-k-1}} = \mathcal{E}_{i_{t-k}}^1(pp_{i_{t-k}}, \xi_{i_{t-k}}, x_{i_{t-k}}, \pi_{i_{t-k}})$. - Output $w = \mathcal{E}_{i_1}^1(pp_{i_1}, \xi_{i_1}, x, \pi_{i_1})$

Intuitively, the extractor picks the specific proof that was generated using all the candidates in T , and runs the extractors in T recursively until a witness is extracted. Since π is parsed as a set of $\binom{n}{t}$ proofs, and these proofs are enumerated by considering all the subsets of size t out of n , such a proof always exists.

As per Definition 19, an adversary may break argument-of-knowledge either by detecting the introduction of trapdoor setups, or by outputting an accepting proof that causes any extractor to fail. If either occurs, we exhibit a reduction to AoK of that specific candidate. The former step is the same for all our combiners, and is formally proved in Lemma 9. Intuitively, we replace one setup at a time, and if the adversary distinguishes two adjacent hybrids we have a reduction.

Given that no reduction occurred after introducing the trapdoor setups, we proceed with analyzing $\mathcal{E}^T$. The only way $\mathcal{E}^T$ can output w is by recursively extracting from the proof π that was generated by running all the schemes in T . Because all candidates work for any NP

language $\mathcal{L}$, they also work with the specific languages we have in Figure 5.1. Therefore, in order for $\mathcal{E}^T$ to fail, it suffices for one of the sub-extraction procedures to fail. Let i^* be an index for which $\mathcal{E}_{i^*}^1$ fails to extract from (x_{i^*}, π_{i^*}) . Any adversary that outputs an accepting proof (x, π) for the combiner for which the extraction procedure, upon reaching scheme Π_{i^*} , fails to extract from (x_{i^*}, π_{i^*}) , immediately causes $\mathcal{E}^T$ to fail. This yields a reduction to AoK of scheme Π_{i^*} . $\square$

H Combiner from MPC-in-the-head

In this section, we present formal theorems and proofs for our combiner presented in Section 6.

H.1 $\mathbf{C}_{\text{MPC}}$ is a (ZK, Sound)-combiner

In order to claim that $\mathbf{C}_{\text{MPC}}$ (as in Figure 6.1) is a t -out-of- n NB (ZK, Sound)-combiner, we need to argue the following:

- $\mathbf{C}_{\text{MPC}}$ is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner for any NP language $\mathcal{L}$, as per Definition 2.
- $\mathbf{C}_{\text{MPC}}$ is a sound t -out-of- n NB (ZK, Sound)-combiner for any NP language $\mathcal{L}$, as per Definition 5.
- $\mathbf{C}_{\text{MPC}}$ is a zero-knowledge t -out-of- n NB (ZK, Sound)-combiner for any NP language $\mathcal{L}$, as per Definition 6.

Below, we provide formal theorems and proofs for each of the aforementioned items.

Theorem 12. *The construction in Figure 6.1 is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner for any NP language $\mathcal{L}$, as per Definition 2, for $n, t \in \text{poly}(\lambda)$.*

Proof. In order for the construction to be an n -candidate $\mathcal{F}_{\text{NI}}$ -combiner, the following has to hold:

$$\Pr \left[\mathbf{C}_{\text{vrfy}}(pp, x, \mathbf{C}_{\text{prove}}(pp, x, w)) = 1 \mid pp \xleftarrow{\$} \mathbf{C}_{\text{setup}}(1^{|x|}, 1^\lambda) \right] \geq 1 - \text{negl}(\lambda).$$

Intuitively, this happens if (i) all sub-proofs are accepting, and (ii) the commitment corresponding to the outgoing messages are consistent with the commitment corresponding to the inbound messages. Because Π_{MPC} is honestly executed and commitments are computed honestly, then (ii) holds. Hence, it only remains to show that all the sub-proofs are accepting:

1. By perfect correctness of Π_{MPC} , f_x (as described in Section 6) obtains $w' \leftarrow \text{reconstruct}(\{w_j\}_{j \in [t]})$, where $(w_1, \dots, w_n) \xleftarrow{\$} \text{share}(w)$. By correctness of secret sharing scheme SS , we have $w' = w$. When $(x, w) \in \mathcal{R}_{\mathcal{L}}$, f_x outputs 1.
2. By Definition 1, for $k \in [n]$, $\mathcal{V}_k$ will output 1 with probability at least $1 - \text{negl}(\lambda)$ if and only if $(x'_k, w'_k) \in \mathcal{R}_{k, \mathcal{L}_{\text{MPC}}}$ and Π_k is for $\mathcal{L}_{k, \text{MPC}}$. Since $\Pi_k \in \mathcal{F}_{\text{NI}}$ works for any NP language, it will also work for $\mathcal{L}_{k, \text{MPC}}$. By correctness of Π_{MPC} , $(x'_k, w'_k) \in \mathcal{R}_{\mathcal{L}_{k, \text{MPC}}}$. Therefore, all $\mathcal{V}_k$ will output 1 with probability at least $1 - \text{negl}(\lambda)$.

Hence, when $(x, w) \in \mathcal{R}_{\mathcal{L}}$, the probability $\mathbf{C}_{\text{vrfy}}$ accepts is the same as the probability when all sub-proofs are accepting, which is exactly the same as Event 1 in the proof of Theorem 4. $\square$

Theorem 13. *The construction in Figure 6.1 is a sound t -out-of- n NB (ZK, Sound)-combiner for any NP language $\mathcal{L}$ for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. The proof is very similar to the proof of Theorem 5. The only difference is that we need to argue that there must be $n - t + 1$ reductions to the soundness of the candidates by relying on the statistical binding of the non-interactive commitment scheme and on the $\mathcal{K}$ -robustness of the MPC protocol.

Formally, assume by contradiction that there are at most $n - t$ reductions. This means there will be $n - (n - t) = t$ sub-statements x'_k that are in the language $\mathcal{L}_{k,\text{MPC}}$, which means that there are t parties that output 1. By assumption, we know $x \notin \mathcal{L}$. This means that $(x, w') \notin \mathcal{R}_{\mathcal{L}}$ holds for any w' . In this case, $f_x(w_1, \dots, w_n)$ must output 0.

Because the verifier accepts, the commitments corresponding to the outgoing messages are consistent with the commitments corresponding to the inbound messages. In this case, we claim that all pairs of views are consistent. Let E_{bad} be the event for an unbounded malicious prover $\tilde{\mathcal{P}}$ produces two consistent commitments that open to inconsistent views. This means that there exist two openings o_0, o_1 for the same commitment that open to the same value. Given that NIC is statistically binding, this event happens only with negligible probability.

Hence, conditioning on E_{bad} not happening, all views come from the same execution of Π_{MPC} realizing f_x . In such an execution, by perfect $\mathcal{K}$ -robustness, when there are at most $\mathcal{K}$ corrupted parties (controlled by a malicious adversary) that output 1, the other t parties must output 0. Since perfect $\mathcal{K}$ -robustness holds unconditionally, we have a contradiction, as in this case the verification procedure of the combiner must have been rejecting, because some parties are outputting 0. This is the case as, in order to break $\mathcal{K}$ -robustness, at least one more party should output 1. But the existence such a party would imply the existence of more reductions to the underlying primitives, contradicting the assumption of having at most $n - t$ reductions.

Therefore, any accepting proof π for the combiner for a statement $x \notin \mathcal{L}$ implies the existence of at least $n - t + 1$ reductions to soundness of the underlying candidates. Once we have at least $n - t + 1$ reductions, the proof is the same as the proof of Theorem 5. $\square$

Theorem 14. *The construction in Figure 6.1 is a zero-knowledge t -out-of- n NB (ZK, Sound)-combiner for any NP language $\mathcal{L}$, as per Definition 6, for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. The structure of this proof is nearly the same as the one of Theorem 6: in order to obtain $n - t + 1$ reductions to zero-knowledge of the candidates, we first construct a simulator $\mathcal{S}^T$ and then use exactly the same the combinatorics argument used to prove Lemma 7. Again, $\mathcal{S}^T$ should be such that either its output is indistinguishable from the output of the real prover, or there exists at least one reduction to zero-knowledge of one of the candidates whose indices are in T . However, this construction additionally requires computational hiding of the non-interactive commitment scheme, which relies both the soundness as well as the zero-knowledge of $n - t + 1$ of the candidates. Hence, within the proof, we additionally argue that, if the computational hiding of the commitment is violated, we would have at least $n - t + 1$ reductions to either soundness or zero-knowledge of the input candidates. We expand this argument and describe $\mathcal{S}^T$ in Lemma 15. $\square$

Lemma 15. *There exists a simulator $\mathcal{S}^T$ of C_{MPC} for which either $\{\text{REAL}\} \approx_c \{\text{IDEAL}_{\mathcal{S}^T}\}$, or there exists a reduction to zero-knowledge of at least one of the candidates $\{\Pi_k\}_{k \in [T]}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. Assume Gen is an algorithm that samples uniformly random elements in the space of shares of w . $\mathcal{S}^T = (\mathcal{S}^{T,0}, \mathcal{S}^{T,1})$ is constructed as follows:

Figure H.1: Simulator $\mathcal{S}^T$

$\mathcal{S}^{T,0}(1^l, 1^\lambda)$:

- For $l \in T$, compute $(pp_l, \tau_l) \xleftarrow{\$} \mathcal{S}_l^0(1^l, 1^\lambda)$.
- For $i \in [n] \setminus T$, compute $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$.
- Compute $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$.
- Compute $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$, $\tau \leftarrow \{\tau_k\}_{k \in T}$.
- Return (pp, τ) .

$\mathcal{S}^{T,1}(pp, \tau, x)$:

- Sample $\{w_j\}_{j \in [n] \setminus T} \xleftarrow{\$} \text{Gen}(1^\lambda)$. Run the $\mathcal{K}$ -privacy simulator $\mathcal{S}_{\text{MPC}}([n] \setminus T, x, \{w_j\}_{j \in [n] \setminus T}, 1)$ of Π_{MPC} to obtain the simulated views $\{\text{view}_j\}_{j \in [n] \setminus T}$.
- For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j . Complete view_i to be consistent with $\{\text{view}_j\}_{j \in [n] \setminus T}$.
- For $i \in [n]$, $j \in [n]$, compute $(\text{com}_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(\text{com}_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
- For $i \in [n]$, set $x'_i \leftarrow (\{\text{com}_{i,j}\}_{j \in [n]}, \{\text{com}_{j,i}\}_{j \in [n]})$. For $l \in [n] \setminus T$, $w'_l \leftarrow (\text{view}_l, \{o_{l,j}\}_{j \in [n]}, \{o_{j,l}\}_{j \in [n]})$.
- For $l \in T$, compute $\pi_l \leftarrow \mathcal{S}_l^1(pp_l, \tau_l, (pp_{\text{NIC}}, x'_l))$. For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$.
- Return $(\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.

The proof of this lemma goes through $t + 4$ hybrid experiments, where hybrid $\mathcal{H}_0$ is the same as the real world, and hybrid $\mathcal{H}_{t+3}$ is the same as the ideal world. We denote the output of the adversary in hybrid $\mathcal{H}_i$ as $\text{out}^{\mathcal{H}_i}$, for $i \in \{0, 1, \dots, t + 3\}$. Formally we prove that, for $j \in \{1, \dots, t + 3\}$ and for any algorithm $\mathcal{A}$ running in time τ , the following holds:

$$\left| \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_{j-1}}) = 1] - \Pr[\mathcal{A}(\text{out}^{\mathcal{H}_j}) = 1] \right| \leq \text{negl}(\lambda).$$

We define the hybrids as follows:

Figure H.2: Hybrid experiments $\mathcal{H}_0, \mathcal{H}_j, \mathcal{H}_{t+1}, \mathcal{H}_{t+2}, \mathcal{H}_{t+3}$, for $1 \leq j \leq t$

$\mathcal{H}_0, \boxed{\mathcal{H}_j}, \boxed{\mathcal{H}_{t+1}}, \boxed{\mathcal{H}_{t+2}}, \boxed{\mathcal{H}_{t+3}}$

1. $pp_k \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $k \in [n]$. $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$. $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.

1. $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_j$. $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$.
 $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.

2. Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$

3. $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Run MPC protocol Π_{MPC} with input $(w_1, \dots, w_n)$ to obtain $\{\text{view}_i\}_{i \in [n]}$.

3.
 - $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Run MPC protocol Π_{MPC} with input $(w_1, \dots, w_n)$ to obtain $\{\text{view}_i\}_{i \in [n]}$.
 - For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j .
3.
 - $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Run the $\mathcal{K}$ -privacy simulator $\mathcal{S}_{\text{MPC}}([n] \setminus T, x, \{s_j\}_{j \in [n] \setminus T}, 1)$ of Π_{MPC} to obtain the simulated views $\{\text{view}_j\}_{j \in [n] \setminus T}$.
 - For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j . Complete view_i to be consistent with $\{\text{view}_j\}_{j \in [n] \setminus T}$.
3.
 - Sample $\{w_j\}_{j \in [n] \setminus T} \xleftarrow{\$} \text{Gen}(1^\lambda)$. Run the $\mathcal{K}$ -privacy simulator $\mathcal{S}_{\text{MPC}}([n] \setminus T, x, \{s_j\}_{j \in [n] \setminus T}, 1)$ of Π_{MPC} to obtain the simulated views $\{\text{view}_j\}_{j \in [n] \setminus T}$.
 - For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j . Complete view_i to be consistent with $\{\text{view}_j\}_{j \in [n] \setminus T}$.
4. For $i \in [n]$, $j \in [n]$, compute $(\text{com}_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(\text{com}_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
5. For $i \in [n]$, set $x'_i \leftarrow (\{\text{com}_{i,j}\}_{j \in [n]}, \{\text{com}_{j,i}\}_{j \in [n]})$ and $w'_i \leftarrow (\text{view}_i, \{o_{i,j}\}_{j \in [n]}, \{o_{j,i}\}_{j \in [n]})$.
6. $\pi_k \leftarrow \mathcal{P}_k(pp_k, (pp_{\text{NIC}}, x'_k), w'_k)$ for $k \in [n]$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
6. $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, (pp_{\text{NIC}}, x'_k))$ for $k \in T_j$. $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$ for $i \in [n] \setminus T_j$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
6. $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, (pp_{\text{NIC}}, x'_k))$ for $k \in T$. $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$ for $i \in [n] \setminus T$. $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
7. Run $\mathcal{A}(pp, x, \pi)$.

Intuitively, in the first t hybrids, the real proofs generated by the candidates in T are gradually replaced with simulated proofs generated by their respective simulator. In $\mathcal{H}_{t+1}$, we use the computational hiding property of NIC to replace the messages sent by honest parties with the message 0, and update the views accordingly so the commitments remain consistent. In $\mathcal{H}_{t+2}$, we run $\mathcal{S}_{\text{MPC}}([n] \setminus T, x, \{w_j\}_{j \in [n] \setminus T}, 1)$ in place of the MPC protocol to obtain simulated views for the semi-honest parties. By $\mathcal{K}$ -privacy of Π_{MPC} , such a simulator exists. Finally, in $\mathcal{H}_{t+3}$, we replace $n - t$ shares of the real witness w with $n - t$ random elements in the same space of shares of w . This is possible as shares of the real witness are used in at most $n - t$ of the prover algorithms of the candidates. Because $t > \lfloor \frac{n}{2} \rfloor$, the adversary can leak at most $\lfloor \frac{n}{2} \rfloor$ of the shares, and is unable to determine whether these are shares of w or random elements. Formally, this lemma is proved in Lemma 16, Lemma 17, Lemma 18, and Lemma 19. $\square$

Lemma 16 (Transition from $\mathcal{H}_{j-1}$ to $\mathcal{H}_j$, for $1 \leq j \leq t$). *If there exists an algorithm $\mathcal{A}$ running in time τ for which $\text{out}^{\mathcal{H}_{j-1}} \not\approx_c \text{out}^{\mathcal{H}_j}$, there exists an algorithm Red_{i_j} running in time $\text{poly}(\tau)$ that breaks zero-knowledge of the i_j -th scheme.*

Proof. Given $\mathcal{A}$ as in the theorem statement, we describe Red_{i_j} in what follows:

Figure H.3: Reduction Red_{i_j}

- Receive public parameters pp_{i_j} from the challenger $\mathcal{C}$.
- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T_{j-1}$. Compute $pp_i \xleftarrow{\$} \text{setup}_i(1^l, 1^\lambda)$ for $i \in [n] \setminus T_j$. Compute $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$. Set $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.

- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$.
- Compute $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Run MPC protocol Π_{MPC} with input $(w_1, \dots, w_n)$ to obtain $\{\text{view}_i\}_{i \in [n]}$.
- For $i \in [n], j \in [n]$, compute $(com_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(com_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
- For $i \in [n]$, set $x'_i \leftarrow (\{com_{i,j}\}_{j \in [n]}, \{com_{j,i}\}_{j \in [n]})$ and $w'_i \leftarrow (\text{view}_i, \{o_{i,j}\}_{j \in [n]}, \{o_{j,i}\}_{j \in [n]})$.
- Send $\{x'_i, w'_i\}$ to the challenger $\mathcal{C}$, and receive proof π_{i_j} . This proof is generated by the challenger $\mathcal{C}$ either as $\mathcal{P}_{i_j}(pp_{i_j}, (pp_{\text{NIC}}, x'_{i_j}), w'_{i_j})$ or $\mathcal{S}_{i_j}^1(pp_{i_j}, \tau_{i_j}, (pp_{\text{NIC}}, x'_{i_j}))$.
- For $k \in T_{j-1}$, compute $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, (pp_{\text{NIC}}, x'_k))$. For $i \in [n] \setminus T_j$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$.
- Set $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ provides a real proof, we are in hybrid $\mathcal{H}_{j-1}$. Otherwise, we are in hybrid $\mathcal{H}_j$. Therefore, Red_{i_j} retains the same distinguishing advantage of $\mathcal{A}$, breaking ZK of candidate Π_{i_j} . $\square$

Lemma 17 (Transition from $\mathcal{H}_t$ to $\mathcal{H}_{t+1}$). $\text{out}^{\mathcal{H}_t} \approx_c \text{out}^{\mathcal{H}_{t+1}}$.

Proof. The indistinguishability goes through nt intermediate hybrids $\mathcal{H}_{s,e}$, with $s \in [t]$ and $e \in [n]$. The difference between two intermediate hybrids is that in $\mathcal{H}_{s,e-1}$, $(com_{i_s, i_e}, o_{i_s, i_e}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i_s \rightarrow i_e}^k\}_{k \in [R]})$, whereas in $\mathcal{H}_{s,e}$, $(com_{i_s, i_e}, o_{i_s, i_e}) \leftarrow \text{commit}(pp_{\text{NIC}}, 0)$. This way, we effectively replace the set of messages from party i_s to party i_e (i.e., $\{\text{msg}_{i_s \rightarrow i_e}^k\}$) with 0. We note that, for $s \neq 1$, $\mathcal{H}_{s,0}$ is $\mathcal{H}_{s-1,n}$. We also note that $\mathcal{H}_{1,0}$ is $\mathcal{H}_t$, and $\mathcal{H}_{t,n}$ is $\mathcal{H}_{t+1}$.

Assume by contradiction there exists a PPT algorithm $\mathcal{A}$ that can distinguish $\text{out}^{\mathcal{H}_{s,e-1}}$ from $\text{out}^{\mathcal{H}_{s,e}}$. Then, we can construct the following adversary $\overline{\mathcal{A}}_{s,e}^{\text{NIC}}$ breaking computational hiding of NIC.

Figure H.4: Adversary $\overline{\mathcal{A}}_{s,e}^{\text{NIC}}$

- Receive public parameters $\overline{pp}$ from the challenger $\mathcal{C}$.
- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T$. $pp_i \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $i \in [n] \setminus T$. Set $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{\overline{pp}\}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$.
- Compute $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Run MPC protocol Π_{MPC} with input $(w_1, \dots, w_n)$ to obtain $\{\text{view}_i\}_{i \in [n]}$.
- For $i \in T_s$:
 - if $i \neq i_s$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j .
 - if $i = i_s$, for $j \in T_{e-1}$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j .
- For $i \in [n], j \in [n]$, compute $(com_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(com_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
- Send 0 and $\{\text{msg}_{i_s \rightarrow i_e}^k\}_{k \in [R]}$ to the challenger $\mathcal{C}$, and receive the commitment com' that could either be a commitment of 0 or a commitment of the messages. Set $com_{i_s, i_e} \leftarrow com'$.
- For $i \in [n]$, set $x'_i \leftarrow (\{com_{i,j}\}_{j \in [n]}, \{com_{j,i}\}_{j \in [n]})$ and $w'_i \leftarrow (\text{view}_i, \{o_{i,j}\}_{j \in [n]}, \{o_{j,i}\}_{j \in [n]})$.
- For $k \in T$, compute $\pi_k \leftarrow \mathcal{S}_k^1(pp_k, \tau_k, (pp_{\text{NIC}}, x'_k))$. For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$.
- Set $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ provides a real commitment, we are in intermediate hybrid $\mathcal{H}_{s,e-1}$. Otherwise, we are in intermediate hybrid $\mathcal{H}_{s,e}$. Therefore, $\overline{\mathcal{A}}_{s,e}^{\text{NIC}}$ retains the same distinguishing advantage of $\mathcal{A}$, breaking the computational hiding of NIC. This is a contradiction; therefore, $\text{out}^{\mathcal{H}_{s,e-1}} \approx_c \text{out}^{\mathcal{H}_{s,e}}$. By applying nt intermediate hybrids, we have $\text{out}^{\mathcal{H}_t} \approx_c \text{out}^{\mathcal{H}_{t+1}}$.

Finally we observe that, by Lemma 1, there exists a construction of computationally hiding commitments that exclusively relies on the soundness and zero-knowledge of at least $t > \lfloor \frac{n}{2} \rfloor$ candidates. More specifically: in order to break the computational hiding property of the commitment, it must be that the security of the OWF obtained from the 1-out-of- n combiner from [Her05, Lemma 4.1] is broken. This is possible if an adversary is able to break the security of all OWF candidates. By the construction in Lemma 1, breaking a OWF candidate implies having a reduction either to the soundness of the corresponding input NIZK candidate, or the zero-knowledge of the corresponding input NIZK candidate, or both. Given that $t > \lfloor \frac{n}{2} \rfloor$, it must be the case that there exist either $n - t + 1$ reductions to soundness of distinct NIZK candidates, or $n - t + 1$ reductions to zero-knowledge of distinct NIZK candidates, or both. Therefore, in any of the cases above, we have enough reductions as desired. We argue this fact in what follows.

Assume by contradiction the above is not true. Then, there would exist an adversary breaking the security of all OWF candidates by using at most $n - t$ reductions to soundness and $n - t$ reductions to zero-knowledge of distinct candidates. In particular, the adversary could attack the security of each OWF candidate by using either a reduction to soundness or a reduction to zero-knowledge. This yields a set of broken OWF candidates of size at most $2n - 2t$. Since we assumed that $t > \lfloor \frac{n}{2} \rfloor$, we have that $2n - 2t < n$. But this implies that there still exists a OWF for which both soundness and ZK hold (*i.e.*, a OWF that is secure). $\square$

Lemma 18 (Transition from $\mathcal{H}_{t+1}$ to $\mathcal{H}_{t+2}$). *$\text{out}^{\mathcal{H}_{t+1}}$ and $\text{out}^{\mathcal{H}_{t+2}}$ are identically distributed, if Π_{MPC} has perfect $\mathcal{K}$ -privacy.*

Proof. Assume by contradiction there exists an unbounded algorithm $\mathcal{A}$ that can distinguish $\text{out}^{\mathcal{H}_{t+1}}$ from $\text{out}^{\mathcal{H}_{t+2}}$. Then, we can construct the following adversary $\overline{\mathcal{A}}^{\text{MPC}}$ breaking perfect $\mathcal{K}$ -privacy of Π_{MPC} :

Figure H.5: Adversary $\overline{\mathcal{A}}^{\text{MPC}}$

- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T$. $pp_i \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $i \in [n] \setminus T$. Compute $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$. Set $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$.
- Compute $\{w_i\}_{i \in [n]} \leftarrow \text{share}(w)$. Send $\{w_i\}_{i \in [n]}$ to the challenger $\mathcal{C}$.
- The challenger $\mathcal{C}$ for $\mathcal{K}$ -privacy computes $\{\text{view}_j\}_{j \in [n] \setminus T}$ either by running a real execution of the protocol or by using the $\mathcal{K}$ -privacy simulator $\mathcal{S}(\mathcal{I}, x, \{w_k\}_{k \in [\mathcal{I}]}, 1)$, with $\mathcal{I} = [n] \setminus T$.
- For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j . Complete view_i to be consistent with $\{\text{view}_j\}_{j \in [n] \setminus T}$.
- For $i \in [n]$, $j \in [n]$, compute $(com_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(com_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
- For $i \in [n]$, set $x'_i \leftarrow (\{com_{i,j}\}_{j \in [n]}, \{com_{j,i}\}_{j \in [n]})$. For $l \in [n] \setminus T$, $w'_l \leftarrow (\text{view}_l, \{o_{l,j}\}_{j \in [n]}, \{o_{j,l}\}_{j \in [n]})$.
- For $l \in T$, compute $\pi_l \leftarrow \mathcal{S}_l^1(pp_l, \tau_l, (pp_{\text{NIC}}, x'_l))$. For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$.

- Set $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

If $\mathcal{C}$ produces views coming from real parties, we are in hybrid $\mathcal{H}_{t+1}$. Otherwise, we are in hybrid $\mathcal{H}_{t+2}$. Therefore, $\overline{\mathcal{A}}^{\text{MPC}}$ retains the same distinguishing advantage of $\mathcal{A}$, breaking perfect $\mathcal{K}$ -privacy of Π_{MPC} . $\square$

Lemma 19 (Transition from $\mathcal{H}_{t+2}$ to $\mathcal{H}_{t+3}$). *$\text{out}^{\mathcal{H}_{t+2}}$ and $\text{out}^{\mathcal{H}_{t+3}}$ are identically distributed.*

Proof. Assume by contradiction there exists an unbounded algorithm $\mathcal{A}$ that can distinguish $\text{out}^{\mathcal{H}_{t+2}}$ from $\text{out}^{\mathcal{H}_{t+3}}$. Then, we can construct the following adversary $\overline{\mathcal{A}}^{\text{SS}}$ breaking perfect security of the t -out-of- n secret sharing scheme:

Figure H.6: Adversary $\overline{\mathcal{A}}^{\text{SS}}$

- Compute $(pp_k, \tau_k) \xleftarrow{\$} \mathcal{S}_k^0(1^l, 1^\lambda)$ for $k \in T$. $pp_i \xleftarrow{\$} \text{setup}_k(1^l, 1^\lambda)$ for $i \in [n] \setminus T$. Compute $pp_{\text{NIC}} \xleftarrow{\$} \text{setup}(1^\lambda)$. Set $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$.
- Compute $(x, w) \xleftarrow{\$} \mathcal{A}(pp)$, and forward w to the challenger $\mathcal{C}$.
- Receive $n - t$ shares $\{s_j\}_{j \in [n] \setminus T}$ from the challenger $\mathcal{C}$ which are either shares of the real witness w or random elements $\{r_j\}_{j \in [n] \setminus T} \xleftarrow{\$} \text{Gen}(1^\lambda)$.
- Run the $\mathcal{K}$ -privacy simulator $\mathcal{S}_{\text{MPC}}([n] \setminus T, x, \{s_j\}_{j \in [n] \setminus T}, 1)$ of Π_{MPC} to obtain the simulated views $\{\text{view}_j\}_{j \in [n] \setminus T}$.
- For $i \in T$, for $j \in [n]$, set $\{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]} = 0$ in view_i and view_j . Complete view_i to be consistent with $\{\text{view}_j\}_{j \in [n] \setminus T}$.
- For $i \in [n]$, $j \in [n]$, compute $(\text{com}_{i,j}, o_{i,j}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{i \rightarrow j}^k\}_{k \in [R]})$, and $(\text{com}_{j,i}, o_{j,i}) \leftarrow \text{commit}(pp_{\text{NIC}}, \{\text{msg}_{j \rightarrow i}^k\}_{k \in [R]})$.
- For $i \in [n]$, set $x'_i \leftarrow (\{\text{com}_{i,j}\}_{j \in [n]}, \{\text{com}_{j,i}\}_{j \in [n]})$. For $l \in [n] \setminus T$, $w'_l \leftarrow (\text{view}_l, \{o_{l,j}\}_{j \in [n]}, \{o_{j,l}\}_{j \in [n]})$.
- For $l \in T$, compute $\pi_l \leftarrow \mathcal{S}_l^1(pp_l, \tau_l, (pp_{\text{NIC}}, x'_l))$. For $i \in [n] \setminus T$, compute $\pi_i \leftarrow \mathcal{P}_i(pp_i, (pp_{\text{NIC}}, x'_i), w'_i)$.
- Set $\pi \leftarrow (\{\pi_k\}_{k \in [n]}, \{x'_k\}_{k \in [n]})$.
- Run $\mathcal{A}(pp, x, \pi)$ and output whatever $\mathcal{A}$ outputs.

We now argue that $\overline{\mathcal{A}}^{\text{SS}}$ correctly reproduces the behavior of hybrids $\mathcal{H}_{t+2}$ and $\mathcal{H}_{t+3}$ according to the challenger's behaviour. If the challenger shares the real witness w , all the proofs in $[n] \setminus T$ are generated exactly as in $\mathcal{H}_{t+2}$ (i.e., using shares of w). If the challenger generates random elements by using Gen , all the proofs in $[n] \setminus T$ are generated exactly as in $\mathcal{H}_{t+3}$. Also, because we have $t > \lfloor \frac{n}{2} \rfloor$, $n - t \leq t - 1$. Therefore, $\overline{\mathcal{A}}^{\text{SS}}$ retains the same distinguishing advantage of $\mathcal{A}$, breaking perfect security of the t -out-of- n secret sharing scheme. This is a contradiction; therefore, $\text{out}^{\mathcal{H}_{t+2}}$ and $\text{out}^{\mathcal{H}_{t+3}}$ are identically distributed. $\square$

H.2 C_{MPC} is a (ZK, AoK)-combiner

In order to claim that C_{MPC} (as in Figure 6.1) is a t -out-of- n NBB (ZK, AoK)-combiner, we need to argue the following:

- C_{MPC} is a correct n -candidate $\mathcal{F}_{\text{NI}}$ -combiner for any NP language $\mathcal{L}$, as per Definition 2.
- C_{MPC} is an argument-of-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 20.

- C_{MPC} is a zero-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$, as per Definition 21.

Note that the first item already holds by Theorem 12. In what follows, we formalize the remaining items.

Theorem 15. *The construction in Figure 6.1 is an argument-of-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$ for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. The structure of this proof is the same as the one of Theorem 7 and uses exactly the same the combinatorics argument used to prove Lemma 10. It therefore suffices to construct an extractor $\mathcal{E}^T$, and argue that either $\mathcal{E}^T$ successfully extracts a witness, or there exists at least one reduction to AoK of one of the candidates whose indices are in T . We describe such an $\mathcal{E}^T$ in Lemma 20. $\square$

Lemma 20. *There exists an extractor $\mathcal{E}^T$ for which either a witness w is successfully extracted from an accepting proof, or there exists a reduction to AoK of at least one of the candidates $\{\Pi_k\}_{k \in T}$.*

Proof. Let $T = \{i_1, \dots, i_t\}$ be a set of indices sorted in lexicographic order identifying candidates $\Pi_{i_1}, \dots, \Pi_{i_t}$. The extractor $\mathcal{E}^T = (\mathcal{E}^{T,0}, \mathcal{E}^{T,1})$ is constructed as follows:

Figure H.7: Extractor $\mathcal{E}^T$
$\mathcal{E}^{T,0}(1^l, 1^\lambda)$: • For $k \in T$, compute $(pp_k, \xi_k) \xleftarrow{\\$} \mathcal{E}_k^0(1^l, 1^\lambda)$. • Compute $pp_{\text{NIC}} \xleftarrow{\\$} \text{setup}(1^\lambda)$. • Compute $pp \leftarrow \{pp_k\}_{k \in [n]} \cup \{pp_{\text{NIC}}\}$, $\xi \leftarrow \{\xi_k\}_{k \in T}$. • Return (pp, ξ). $\mathcal{E}^{T,1}(pp, \xi, x, \pi)$: • For $k \in T$, parse the k-th component of π as (x'_k, π_k). Compute $w'_k \leftarrow \mathcal{E}_k^1(pp_k, \xi_k, (pp_{\text{NIC}}, x'_k), \pi_k)$. Parse w'_k as $(\text{view}_k, \{o_{k,j}\}_{j \in [n]}, \{o_{j,k}\}_{j \in [n]})$. Take w_k from view_k. • Output $w \leftarrow \text{reconstruct}(\{w_k\}_{k \in T})$.

An adversary may break AoK either by detecting the introduction of trapdoor setups, or by outputting an accepting proof that causes any extractor to fail. If either occurs, we exhibit a reduction to AoK of that specific candidate.

Introducing trapdoor setups. This step is the same as the proof of Lemma 8, and is therefore omitted.

Extractor $\mathcal{E}^T$ fails to extract a witness. We conclude the proof exactly as the proof of Lemma 8: given that no reduction occurred, we proceed with analyzing $\mathcal{E}^T$.

We observe that the only way $\mathcal{E}^T$ can reconstruct w is by successfully extracting all the sub-witnesses in T , and using the **reconstruct** algorithm of the t -out-of- n secret sharing scheme. Because the verifier C_{verfy} performs checks for the consistency of the commitments, when all the checks pass, it means that all pair of views are consistent with the execution of Π_{MPC} because NIC is statistically binding. The proof is the same as what we have in the proof of Theorem 13.

When all the sub-extraction procedures succeed, we can extract sub-witnesses from t sub-proofs successfully. This implies that these sub-statements x'_k are in the language $\mathcal{L}_{k, \text{MPC}}$, meaning that the output of parties $\{P_k\}_{k \in T}$ is 1. It also means there are at most $n - t$ parties

that output 0, satisfying $\mathcal{K}$ -robustness. Hence, the statement x is in the language, and using these t sub-witnesses as the input of the t -out-of- n secret sharing scheme allows a successful reconstruction of w .

Therefore, in order for $\mathcal{E}^T$ to fail, it suffices for one of the sub-extraction procedures to fail. Let i^* be an index for which $\mathcal{E}_{i^*}^1$ fails to extract from (x'_{i^*}, π_{i^*}) . Any adversary that outputs an accepting proof (x, π) for the combiner that contains (x'_{i^*}, π_{i^*}) in position i^* immediately causes $\mathcal{E}^T$ to fail. As for the proof of Lemma 8, this yields a reduction to AoK of scheme Π_{i^*} . $\square$

Theorem 16. *The construction in Figure 6.1 is a zero-knowledge t -out-of- n NBB (ZK, AoK)-combiner for any NP language $\mathcal{L}$ for $t > \lfloor \frac{n}{2} \rfloor$ and $n, t \in \text{poly}(\lambda)$.*

Proof. The proof for zero-knowledge of Theorem 14 relies both on soundness as well as zero-knowledge of the candidates (due to the presence of the commitment scheme). However, given that AoK implies soundness, we can reuse the same proof in this setting. $\square$